



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΕΦΑΡΜΟΣΜΕΝΩΝ ΜΑΘΗΜΑΤΙΚΩΝ ΚΑΙ ΦΥΣΙΚΩΝ ΕΠΙΣΤΗΜΩΝ

Χρήση εργαλείων Παραγωγικής Τεχνητής Νοημοσύνης για την Ανάλυση Δεδομένων από Κατανεμημένα Εκπαιδευτικά Περιβάλλοντα

Διπλωματική Εργασία

Γιόννη Ζάτσε

Επιβλέπων καθηγητής: Πέτρος Στεφανέας

Αθήνα, Φεβρουάριος 2026

Ευχαριστίες

Θα ήθελα να ευχαριστήσω αρχικά τον καθηγητή μου, κ. Πέτρο Στεφανέα, για την διάθεση και την καθοδήγηση που μου παρείχε κατά την εκπόνηση της διπλωματικής μου εργασίας. Ιδιαίτερη εκτίμηση οφείλω να εκφράσω στον υποψήφιο διδάκτορα της σχολής ΗΜΜΥ Δημήτριο Πανταζάτο, καθώς δίχως την βοήθειά του δεν θα ήταν εφικτή η πραγματοποίηση της παρούσας εργασίας, όπως και φυσικά στη δρ Μαρία Γραμματικού, ΕΔΙΠ της Σχολής Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών.

Ένα μεγάλο ευχαριστώ οφείλω στους φίλους μου με τους οποίους εξελίχθηκα και ωρίμασα ως άνθρωπος. Δίχως την βοήθεια και την στήριξη τους κατάφερα να φέρω εις πέρας τις προπτυχιακές μου σπουδές και να ζήσω τα καλύτερα χρόνια της ζωής μου.

Τέλος, την μεγαλύτερη ευγνωμοσύνη εκφράζω στην οικογένειά μου λόγω της οποίας ήταν όλα αυτά εφικτά. Ο πατέρας μου, με την καθοδήγηση και τους κόπους του οποίου είχα όλα όσα μου χρειάστηκαν κατά την φοίτησή μου. Η μητέρα μου, όπου με την στήριξη και την υπομονή της με βοήθησε όπου χρειαζόταν. Ο αδελφός μου ο οποίος ήταν εκεί για εμένα σε κάθε εύκολη και δύσκολη στιγμή. Η διπλωματική μου εργασία είναι αφιερωμένη σε αυτούς.



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΕΦΑΡΜΟΣΜΕΝΩΝ ΜΑΘΗΜΑΤΙΚΩΝ ΚΑΙ ΦΥΣΙΚΩΝ
ΕΠΙΣΤΗΜΩΝ

ΠΕΡΙΛΗΨΗ

Σκοπός της παρούσας διπλωματικής εργασίας είναι η αξιοποίηση εργαλείων Παραγωγικής Τεχνητής Νοημοσύνης για την υποστήριξη ανάλυσης δεδομένων από κατανεμημένα εκπαιδευτικά περιβάλλοντα, μέσω μιας αυτοματοποιημένης διαδικασίας κατά την οποία ο χρήστης διατυπώνει ερωτήματα σε **φυσική γλώσσα** και το σύστημα τα μετατρέπει σε κατάλληλες δομημένες αναζητήσεις που εκτελούνται σε **σχεσιακή βάση δεδομένων**, ώστε να επιστρέφονται συνοπτικές και αξιοποιήσιμες **απαντήσεις** πάνω στα **Moodle Logs**. Η αρχιτεκτονική της λύσης βασίστηκε σε διακριτά επίπεδα: αποθήκευση/ερωτήματα στη PostgreSQL, σύνδεση με LLMs μέσω LM Studio (OpenAI-compatible API) και οργάνωση παραγωγής ερωτημάτων μέσω prompt templates, με μηχανισμό ελέγχου/διόρθωσης σφαλμάτων ώστε να βελτιώνεται η πρακτική αξιοπιστία της παραγωγής SQL. Η αξιολόγηση πραγματοποιήθηκε με τρία τοπικά LLMs (Hermes Llama 3.2 3B, Phi-4 mini instruct 3B, Deepseek chat 7B), δύο τιμές θερμοκρασίας (0.1 και 0.5) και τέσσερα διακριτά prompts, με μετρικές τον χρόνο ολοκλήρωσης και το ποσοστό επιτυχίας (εκτελέσιμο και σημασιολογικά ορθό SQL που επιστρέφει αποτέλεσμα χωρίς σφάλμα). Τα αποτελέσματα ανέδειξαν σαφείς διαφοροποιήσεις ανά μοντέλο: το Hermes Llama παρουσίασε την υψηλότερη σταθερότητα και ακρίβεια, το Phi-4 πέτυχε ικανοποιητικές επιδόσεις αλλά με μεγαλύτερη εξάρτηση από διαδικασίες διόρθωσης/ανατροφοδότησης, ενώ το Deepseek εμφάνισε σημαντικά χαμηλότερη αξιοπιστία στο συγκεκριμένο σχήμα δεδομένων και στα χρησιμοποιούμενα prompts. Συνολικά, η εργασία τεκμηριώνει ότι η ροή από φυσική γλώσσα σε σχεσιακή βάση δεδομένων είναι εφαρμόσιμη και αποδοτική για ανακτήσεις και συνοπτικές απαντήσεις πάνω σε Moodle Logs, υπό την προϋπόθεση προσεκτικού σχεδιασμού prompts, ελέγχων εγκυρότητας και κατάλληλων πρακτικών ασφάλειας/ιδιωτικότητας.

Λέξεις-κλειδιά: Moodle, Large Language Models (LLMs), Natural language (NL), Learning Analytics, Learning Analytics, LM Studio, LangChain



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΕΦΑΡΜΟΣΜΕΝΩΝ ΜΑΘΗΜΑΤΙΚΩΝ ΚΑΙ ΦΥΣΙΚΩΝ
ΕΠΙΣΤΗΜΩΝ

ABSTRACT

The purpose of this thesis is to leverage Generative Artificial Intelligence tools to support data analysis in distributed educational environments, through an automated process in which users pose questions in natural language and the system translates them into structured queries executed on a relational database, returning concise, data-grounded answers based on Moodle logs. The solution's architecture was based on distinct layers: storage/querying in PostgreSQL, connection to LLMs via LM Studio (OpenAI-compatible API), and orchestration of query generation through prompt templates, along with an error checking/correction mechanism to improve the practical reliability of SQL generation. The evaluation was conducted using three local LLMs (Hermes Llama 3.2 3B, Phi-4 mini instruct 3B, Deepseek chat 7B), two temperature values (0.1 and 0.5), and four distinct prompts, with completion time and success rate as metrics (defined as producing syntactically executable and semantically correct SQL that returns results without errors). The results revealed clear differences across models: Hermes Llama demonstrated the highest stability and accuracy; Phi-4 achieved satisfactory performance but relied more heavily on correction/feedback procedures; and Deepseek showed significantly lower reliability for the specific data schema and the prompts used. Overall, the thesis demonstrates that the natural language to rational database pipeline is feasible and effective for retrieval and concise answering over Moodle Logs, provided that prompts are carefully designed, validity checks are applied, and appropriate security/privacy practices are followed.

Keywords: Moodle, Large Language Models (LLMs), Natural language (NL), Learning Analytics, Learning Analytics, LM Studio, LangChain

ΠΕΡΙΕΧΟΜΕΝΑ

Κεφάλαιο 1ο: Εισαγωγή	9
1.1 Στόχοι	10
1.2 Υπηρεσίες	10
1.3 Αρχές	11
1.4 Αρχιτεκτονική και Σχεδιασμός	12
Κεφάλαιο 2ο: Θεωρητικό Υπόβαθρο	13
2.1 Learning Analytics	13
2.2 Moodle Logs	13
2.2.1 Πώς χρησιμοποιούνται τα Moodle Logs	15
2.2.2 Παράμετροι Moodle Logs	17
2.3 Καθαρισμός και Προεπεξεργασία Δεδομένων	21
2.3.1 Διαδικασίες Καθαρισμού	21
2.3.2 Τελική Μορφή Δεδομένων	23
2.3.3 Συμπεράσματα Καθαρισμού	24
2.4 PostgreSQL	25
2.4.1 Χαρακτηριστικά PostgreSQL	25
2.4.2 Πλεονεκτήματα της χρήσης PostgreSQL	26
2.4.3 PostgREST API	27
2.5 LLMs	27
2.5.1 Πώς λειτουργούν τα LLMs	27
2.5.2 LM Studio και LLMs που χρησιμοποιήθηκαν	28
2.5.3 Prompt Engineering	30
2.6 LangChain	31
2.6.1 Τι είναι η LangChain, μια σύντομη επισκόπηση	32
Κεφάλαιο 3ο: Επεξεργασία και Καθαρισμός Δεδομένων	33
3.1 Αρχική Φόρτωση και Βασικός Χαρακτηρισμός	33
3.2 Κανόνες Καθαρισμού και Μετασχηματισμοί	34
3.3 Τελικό Schema και Εξαγωγή	36
3.4 Ενδεικτικοί Έλεγχοι Εγκυρότητας	37
3.5 Συμπεράσματα	37
Κεφάλαιο 4ο: Υλοποίηση Εφαρμογής	38
4.1 Περιβαλλον Υλοποίησης	39
4.2 Υλοποίηση	41
4.2.1 Επεξεργασία και προετοιμασία δεδομένων στην PostgreSQL	41
4.2.2 PostgREST API - Μεθοδολογία και βήματα υλοποίησης	43
4.2.3 LangChain - Μεθοδολογία και βήματα υλοποίησης	45
4.2.4 Υλοποίηση κώδικα	49
4.2.5 Υλοποίηση prompt engineering	54
Κεφάλαιο 5ο: Αποτελέσματα	56
5.1 Hermes Llama 3.2 (3B)	58
5.2 Phi-4 mini instruct (3B)	63

5.3 DeepSeek chat (7B)	69
5.4 Συγκεντρωτικά Αποτελέσματα	75
5.5 Συζήτηση Αποτελεσμάτων	77
5.6 Περιορισμοί	79
Κεφάλαιο 6ο: Επίλογος	81
6.1 Σύνοψη	81
6.2 Προοπτικές Μελλοντικών Επεκτάσεων	82
Παραρτήματα	83
Βιβλιογραφία	93

Κεφάλαιο 1ο: Εισαγωγή

Η αξιοποίηση δεδομένων από καταναεμημένα εκπαιδευτικά περιβάλλοντα και πλατφόρμες τύπου Learning Management System (LMS) αποτελεί βασικό άξονα της Learning Analytics (LA) και της Educational Data Mining (EDM), καθώς υποστηρίζει την κατανόηση της μαθησιακής δραστηριότητας και τη λήψη τεκμηριωμένων εκπαιδευτικών αποφάσεων μέσω ανάλυσης δεδομένων που παράγονται κατά τη χρήση των συστημάτων μάθησης. [4] Στο πλαίσιο αυτό, το Moodle (ένα ευρέως χρησιμοποιούμενο LMS) καταγράφει τη δραστηριότητα μέσω των Moodle Logs/Events, δηλαδή λεπτομερών συμβάντων που αφορούν ενέργειες χρηστών και λειτουργίες της πλατφόρμας, δημιουργώντας ένα πλούσιο αλλά συχνά ιδιαίτερα ογκώδες σύνολο δεδομένων. [20] Η πρακτική αξιοποίηση τέτοιων καταγραφών προϋποθέτει προεπεξεργασία (καθαρισμό) ώστε να μειωθεί ο θόρυβος και να διασφαλιστεί η συνέπεια των εγγραφών, καθώς και μεθοδολογίες που καθιστούν τα log δεδομένα ερμηνεύσιμα για εκπαιδευτικούς σκοπούς [1].

Στην καθημερινή πρακτική, οι εκπαιδευτικοί έχουν πρόσβαση στα δεδομένα κυρίως μέσω των ενσωματωμένων οθονών/αναφορών και των δυνατοτήτων φιλτραρίσματος της πλατφόρμας [20]. Ωστόσο, η εξαγωγή ουσιαστικής πληροφορίας συχνά απαιτεί πιο σύνθετες συσχετίσεις και ομαδοποιήσεις (π.χ. «ποιοι φοιτητές έχουν απότομη πτώση δραστηριότητας;», «πώς μεταβάλλεται η συμμετοχή ανά εβδομάδα/μάθημα;», «ποιες δραστηριότητες συγκεντρώνουν τη μεγαλύτερη αλληλεπίδραση;»), οι οποίες δεν είναι πάντα εύκολο να προκύψουν μόνο με βασικά φίλτρα. [1], [2] Σε αυτές τις περιπτώσεις, η ανάλυση γίνεται είτε με χειροκίνητες εξαγωγές/επεξεργασία είτε με τεχνικές λύσεις που προϋποθέτουν πρόσβαση σε βάση δεδομένων και χρήση SQL, κάτι που δεν είναι δεδομένο για όλους τους εκπαιδευτικούς και περιορίζει την άμεση, ευέλικτη διερεύνηση ερωτημάτων πάνω στη δραστηριότητα των φοιτητών. [1], [2]

Τα Large Language Models (LLMs) και η Παραγωγική Τεχνητή Νοημοσύνη εισάγουν νέες δυνατότητες αλληλεπίδρασης με δεδομένα μέσω φυσικής γλώσσας, επιτρέποντας τη μετατροπή ερωτήσεων σε δομημένα ερωτήματα πάνω σε σχεσιακές βάσεις. [3], [7], [8], [11] Ειδικότερα, η περιοχή Natural Language to SQL (Text-to-SQL) στοχεύει στη γεφύρωση του κενού μεταξύ «ερωτήσεων ανθρώπου» και «ερωτημάτων βάσης», μειώνοντας το απαιτούμενο τεχνικό υπόβαθρο για την ανάκτηση πληροφορίας. [9], [10], [11] Με αυτό τον τρόπο, μια λύση φυσικής γλώσσας προς SQL μπορεί να λειτουργήσει ως «στρώμα διευκόλυνσης» ανάμεσα στον εκπαιδευτικό και τη βάση δεδομένων, επιτρέποντας πιο επαναληπτική και διερευνητική ανάλυση (με διαδοχικές ερωτήσεις και παραμετροποίηση) χωρίς άμεση συγγραφή SQL από τον χρήστη. [7], [8], [11]

Παρά τα οφέλη, η χρήση LLMs σε σενάρια παραγωγής SQL απαιτεί προσεκτικό σχεδιασμό ώστε να περιορίζονται σφάλματα/ασάφειες και να διασφαλίζεται ότι τα παραγόμενα ερωτήματα είναι εκτελέσιμα και ασφαλή. [10], [11] Κρίσιμος παράγοντας είναι η σωστή αντιστοίχιση στο σχήμα (schema) της βάσης, δηλαδή στη δομή της (πίνακες, πεδία/στήλες, τύποι δεδομένων και σχέσεις μεταξύ πινάκων). [9], [11] Επιπλέον, η "ασφάλεια" δεν αφορά μόνο τη συντακτική ορθότητα, αλλά και τον έλεγχο του τι επιτρέπεται να ζητηθεί/επιστραφεί (π.χ. περιορισμοί πρόσβασης και πολιτικές ανά ρόλο) στο επίπεδο της βάσης και του API. [16]

Στο πλαίσιο της παρούσας εργασίας υλοποιείται και αξιολογείται μια αυτοματοποιημένη ροή Φυσική Γλώσσα (NL) → SQL → Εκτέλεση → Απάντηση πάνω σε καθαρισμένο σύνολο Moodle Logs, με στόχο την αξιόπιστη ανάκτηση πληροφορίας και την επιστροφή σύντομων, αξιοποιήσιμων απαντήσεων που μπορούν να υποστηρίξουν εκπαιδευτικές αποφάσεις. [4], [20] Η υλοποίηση βασίζεται σε βάση PostgreSQL που εκτίθεται ως RESTful υπηρεσία μέσω PostgREST [16], ώστε η πρόσβαση/εκτέλεση ερωτημάτων να γίνεται με τυποποιημένο τρόπο. [16] Τα LLMs εκτελούνται τοπικά μέσω LM Studio με OpenAI-compatible endpoints, υποστηρίζοντας πειρατισμό με διαφορετικά μοντέλα σε τοπικό περιβάλλον. [12], [22] Η ενορχήστρωση της διαδικασίας (δημιουργία prompt, κλήση μοντέλου, εκτέλεση SQL, επιστροφή αποτελέσματος) πραγματοποιείται μέσω LangChain. [17], [18], [19] Για να αυξηθεί η αξιοπιστία, η παραγωγή SQL στηρίζεται σε αυστηρά prompt templates που ενσωματώνουν το schema της βάσης και κανόνες

παραγωγής SQL, καθώς και σε μηχανισμό ελέγχου και ανατροφοδότησης σφαλμάτων (feedback) για διόρθωση και επανεκτέλεση όταν προκύπτουν αποτυχίες. [7], [8], [11] Τέλος, η προσέγγιση αξιολογείται πειραματικά με τρία LLMs που εκτελέστηκαν τοπικά μέσω LM Studio, σε δύο ρυθμίσεις θερμοκρασίας και με τέσσερα διακριτά prompts, καταγράφοντας χρόνο ολοκλήρωσης και ποσοστό επιτυχίας. [12], [13], [14], [15]

1.1 Στόχοι

Κεντρικός στόχος του προγράμματος (συστήματος) που αναπτύχθηκε είναι η υποστήριξη ερωτημάτων φυσικής γλώσσας πάνω σε Moodle Logs, με αυτοματοποιημένη παραγωγή και εκτέλεση SQL, ώστε να επιστρέφεται τεκμηριωμένη και συνοπτική απάντηση.

Ειδικότερα, οι επιμέρους στόχοι συνοψίζονται ως εξής:

- **Διαμόρφωση κατάλληλου dataset** από Moodle Logs με διαδικασίες καθαρισμού/προεπεξεργασίας, ώστε να ελαχιστοποιηθεί ο θόρυβος και να ενισχυθεί η αναλυτική αξιοπιστία.
- **Οργάνωση των δεδομένων σε σχεσιακή βάση PostgreSQL** για αποδοτική αναζήτηση, τυποποιημένο σχήμα και έλεγχο συνέπειας.
- **Σχεδιασμός ελεγχόμενων prompts** που «δένουν» τη συμπεριφορά του μοντέλου με το πραγματικό schema και μειώνουν την πιθανότητα παραγωγής μη εκτελέσιμων ή άστοχων ερωτημάτων.
- **Πειραματική αξιολόγηση LLMs** (με συγκρίσιμες συνθήκες) ως προς την επιτυχία παραγωγής ορθού SQL και τον χρόνο απόκρισης, ώστε να εξαχθούν πρακτικά συμπεράσματα για το ποιο μοντέλο είναι καταλληλότερο στο συγκεκριμένο πλαίσιο.

1.2 Υπηρεσίες

Το πρόγραμμα υλοποιεί μια αλληλουχία υπηρεσιών που καλύπτει πλήρως τη μετάβαση από ερώτημα φυσικής γλώσσας σε απάντηση βασισμένη σε δεδομένα βάσης, με έμφαση στην καθοδήγηση του μοντέλου, την εκτελεσιμότητα των παραγόμενων SQL ερωτημάτων και τη διαχείριση σφαλμάτων.

- **Υποβολή ερωτήματος φυσικής γλώσσας (NL)**
Ο χρήστης διατυπώνει το ερώτημά του σε φυσική γλώσσα, χωρίς να απαιτείται γνώση SQL ή γνώση του υποκείμενου σχήματος της βάσης. Η είσοδος μπορεί να περιλαμβάνει παραμέτρους όπως χρονικό διάστημα, μάθημα/ενότητα ή συγκεκριμένη κατηγορία δραστηριότητας, με τρόπο που αντανακλά το λεξιλόγιο και τις ανάγκες του εκπαιδευτικού.
- **Παραγωγή SQL από LLM με αυστηρό πλαίσιο κανόνων**
Το LLM καλείται να παράγει SQL ερώτημα μέσω prompt templates, τα οποία ενσωματώνουν (α) το schema της βάσης (πίνακες, πεδία/στήλες, σχέσεις) και (β) ρητούς περιορισμούς παραγωγής (π.χ. επιστροφή αποκλειστικά ερωτήσεων SQL δίχως κείμενο φυσικής γλώσσας, αποφυγή μη σχετικών πινάκων/πεδίων). Η χρήση prompt engineering στοχεύει στη μείωση ασάφειας και στη βελτίωση της συνέπειας των παραγόμενων ερωτημάτων. [7], [8]

- **Εκτέλεση SQL στη PostgreSQL και ανάκτηση αποτελεσμάτων**

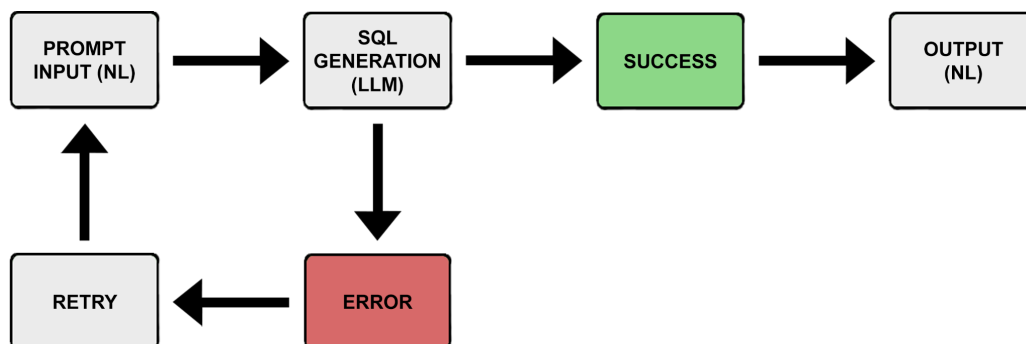
Το παραγόμενο SQL εκτελείται στη βάση PostgreSQL, πάνω στον πίνακα moodle_data (οργανωμένο και καθαρισμένο για τις ανάγκες της εργασίας). Το αποτέλεσμα επιστρέφεται στο σύστημα ως δομημένο σύνολο εγγραφών/τιμών.

- **Ανίχνευση σφαλμάτων και ανατροφοδότηση (feedback loop)**

Σε περίπτωση αποτυχίας εκτέλεσης (π.χ. μη υπαρκτές στήλες, συντακτικά σφάλματα), το σύστημα καταγράφει το σφάλμα και δημιουργεί στοχευμένη ανατροφοδότηση προς το μοντέλο (π.χ. «η στήλη X δεν υπάρχει», «χρησιμοποίησε τη στήλη Y», «διόρθωσε τη σύνταξη στο GROUP BY»). Στη συνέχεια, το LLM καλείται να αναθεωρήσει το SQL και να το επαναπαράγει, αυξάνοντας την πιθανότητα επιτυχούς εκτέλεσης σε επόμενο βήμα.

- **Παραγωγή συνοπτικής απάντησης**

Όταν υπάρξει έγκυρο αποτέλεσμα, το LLM παράγει σύντομη απάντηση σε φυσική γλώσσα, βασισμένη στα επιστρεφόμενα δεδομένα. Στόχος είναι η απάντηση να είναι κατανοητή και αξιοποιήσιμη (π.χ. να περιέχει αριθμητικά ευρήματα, συγκρίσεις ή βασικά συμπεράσματα), χωρίς να απαιτείται από τον χρήστη να ερμηνεύσει απευθείας τα "ωμά" αποτελέσματα SQL.



Εικόνα 1: Μια απλή αναπαράσταση του workflow του προγράμματος

1.3 Αρχές

Ο σχεδιασμός του συστήματος βασίστηκε σε αρχές που στοχεύουν στην αξιοπιστία, την αναπαραγωγικότητα και την ασφαλή χρήση:

- **«Grounding» στο schema (schema grounding):** η παραγωγή SQL περιορίζεται ρητά σε υπαρκτές στήλες/πίνακες, ώστε να μειώνονται «εφευρετικές» αλλά λανθασμένες απαντήσεις.
- **Ελεγχόμενη στοχαστικότητα:** η ρύθμιση παραμέτρων όπως η θερμοκρασία και το prompting χρησιμοποιείται ώστε να εξισορροπείται η σταθερότητα (ιδίως στη φάση παραγωγής SQL) με την ευελιξία στη διατύπωση.
- **Μηχανισμοί διόρθωσης:** η ύπαρξη feedback loop επιτρέπει την ανάκαμψη από συχνά σφάλματα παραγωγής, ενισχύοντας την πρακτική χρησιμότητα του συστήματος.

1.4 Αρχιτεκτονική και Σχεδιασμός

Η αρχιτεκτονική οργανώνεται σε διακριτά επίπεδα, με στόχο να μειωθεί η πολυπλοκότητα της συνολικής λύσης και να αυξηθεί η αξιοπιστία της ροής $NL \rightarrow SQL \rightarrow \text{Εκτέλεση} \rightarrow \text{Απάντηση}$, μέσω σαφούς διαχωρισμού λειτουργιών: αποθήκευση/ποιότητα δεδομένων, τυποποιημένη πρόσβαση, παραγωγή SQL και απάντησης από το μοντέλο, καθώς και μηχανισμών ελέγχου, επικύρωσης και ανατροφοδότησης.

- **Επίπεδο Δεδομένων (PostgreSQL):** τα καθαρισμένα Moodle Logs αποθηκεύονται στη βάση και διατίθενται μέσω ενός πίνακα, ώστε να υποστηρίζονται αποδοτικά ερωτήματα και έλεγχοι εγκυρότητας
- **Επίπεδο Πρόσβασης/Διασύνδεσης:** όπου απαιτείται, μπορεί να αξιοποιηθεί πρόσβαση μέσω API (PostgREST) για τυποποιημένη ανάκτηση δεδομένων και ενιαίο σημείο κατανάλωσης από εξωτερικές ροές
- **Επίπεδο Γλωσσικού Μοντέλου (LLM):** τα γλωσσικά μοντέλα εκτελούνται τοπικά μέσω LM Studio (OpenAI-compatible API), επιτρέποντας κλήσεις για παραγωγή SQL και για παραγωγή σύντομης περίληψης αποτελεσμάτων
- **Επίπεδο Λογικής Ροής (LangChain / templates / έλεγχοι):** ορίζονται prompts, κανόνες και μηχανισμοί ανατροφοδότησης σφαλμάτων ώστε η έξοδος του μοντέλου να είναι εκτελέσιμη και συμβατή με το schema

Κεφάλαιο 2ο: Θεωρητικό Υπόβαθρο

2.1 Learning Analytics

Η ανάλυση δεδομένων σε εκπαιδευτικά περιβάλλοντα εντάσσεται στο ευρύτερο πεδίο του **Learning Analytics (LA)** και του **Educational Data Mining (EDM)**. Στόχος αυτών των προσεγγίσεων είναι η συλλογή, επεξεργασία και αξιοποίηση δεδομένων που παράγονται κατά την αλληλεπίδραση εκπαιδευομένων και εκπαιδευτών με ψηφιακές πλατφόρμες μάθησης, ώστε να εξαχθούν συμπεράσματα για τη μαθησιακή διαδικασία και να υποστηριχθούν τεκμηριωμένες παρεμβάσεις και αποφάσεις [4]. Στο πλαίσιο αυτό, κρίσιμο ρόλο παίζουν τα "ίχνη δραστηριότητας" (digital traces), δηλαδή τα καταγεγραμμένα γεγονότα που αποτυπώνουν ενέργειες χρηστών μέσα στο σύστημα (π.χ. προβολές περιεχομένου, υποβολές εργασιών, συμμετοχή σε δραστηριότητες) [1], [20].

Σε πλατφόρμες τύπου Learning Management System (LMS), το βασικότερο σύνολο τέτοιων ιχνών είναι οι καταγραφές συμβάντων (logs/events). Οι καταγραφές αυτές λειτουργούν ως πρωτογενής πηγή δεδομένων για την παρακολούθηση συμμετοχής, την ανάλυση προτύπων αλληλεπίδρασης, την ανίχνευση πιθανών δυσκολιών και γενικότερα τη "μετάφραση" της χρήσης του συστήματος σε μετρήσιμους δείκτες που μπορούν να αξιοποιηθούν παιδαγωγικά.

Με βάση αυτή τη λογική, τα Moodle Logs αποτελούν τον κεντρικό μηχανισμό καταγραφής δραστηριότητας της πλατφόρμας Moodle και συνιστούν το βασικό υλικό πάνω στο οποίο μπορεί να στηριχθεί ανάλυση Learning Analytics στο συγκεκριμένο περιβάλλον.

2.2 Moodle Logs

Το Moodle αποτελεί ένα από τα πιο διαδεδομένα Συστήματα Διαχείρισης Μάθησης (Learning Management Systems - LMS) παγκοσμίως, χρησιμοποιούμενο σε εκπαιδευτικά ιδρύματα, οργανισμούς και επιχειρήσεις για τη διαχείριση και υποστήριξη της διαδικασίας της ηλεκτρονικής μάθησης. Μέσω της πλατφόρμας αυτής, οι εκπαιδευτές μπορούν να δημιουργούν μαθήματα, να ανεβάζουν εκπαιδευτικό υλικό, να οργανώνουν δραστηριότητες και να παρακολουθούν τη συμμετοχή των εκπαιδευομένων. Για να επιτευχθεί αυτή η λειτουργικότητα, το Moodle καταγράφει ένα τεράστιο πλήθος γεγονότων και ενεργειών που πραγματοποιούνται από τους χρήστες του. Αυτές οι καταγραφές αποτελούν τα Moodle Logs.

Τα Moodle Logs είναι ουσιαστικά ένα ιστορικό αλληλεπιδράσεων μεταξύ των χρηστών (εκπαιδευτών, φοιτητών, διαχειριστών) και των στοιχείων του συστήματος. Κάθε ενέργεια που πραγματοποιείται, όπως η είσοδος ενός χρήστη στην σελίδα του μαθήματος, η υποβολή μιας εργασίας, η προβολή ενός αρχείου ή η συμμετοχή σε ένα κουίζ κ.τ.λ., αποθηκεύεται με λεπτομέρειες όπως: το αναγνωριστικό του χρήστη, ο τύπος της ενέργειας, το μάθημα ή η δραστηριότητα που αφορά, καθώς και η ημερομηνία και ώρα που συνέβη.

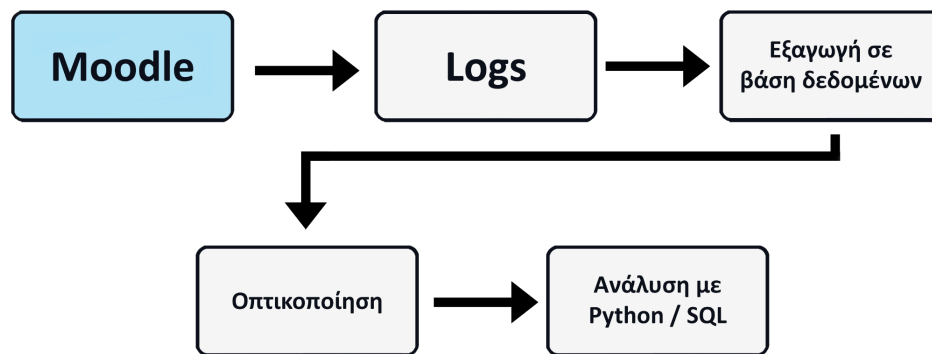
	id bigint	eventname text	component text	action text	target text	crud text	edulevel integer	userid bigint	courseid bigint	anonymous boolean	timecreated bigint	origin text	datetime timestamp with time zone	time time without time zone
1	75279739	\mod_quiz\event\attempt_viewed	mod_quiz	viewed	attempt	read	2	7642	532	false	1583959488	web	2020-03-11 20:44:48+02	20:44:48
2	75279868	\core\event\course_viewed	core	viewed	course	read	2	0	1	false	1583959609	web	2020-03-11 20:46:49+02	20:46:49
3	75279916	\core\event\course_module_compl...	core	updated	course_mo...	upd...	2	4247	532	false	1583959646	web	2020-03-11 20:47:26+02	20:47:26
4	75280323	\mod_quiz\event\attempt_started	mod_quiz	started	attempt	crea...	2	12664	2260	false	1583959936	web	2020-03-11 20:52:16+02	20:52:16
5	75280542	\mod_quiz\event\attempt_viewed	mod_quiz	viewed	attempt	read	2	5833	532	false	1583960067	web	2020-03-11 20:54:27+02	20:54:27

Εικόνα 2: Παράδειγμα εγγραφών από Moodle Logs

Η σημασία των Moodle Logs είναι ιδιαίτερα μεγάλη, καθώς προσφέρουν πολύτιμες πληροφορίες για την ανάλυση της εκπαιδευτικής συμπεριφοράς και τη βελτίωση της μαθησιακής διαδικασίας. Μέσα από την ανάλυση αυτών των δεδομένων, μπορούμε να εξάγουμε συμπεράσματα σχετικά με:

- Το επίπεδο συμμετοχής των φοιτητών σε ένα μάθημα
- Τη συχνότητα και το είδος των αλληλεπιδράσεων (π.χ. ενεργή συμμετοχή σε φόρουμ, υποβολή εργασιών, προσπάθειες κουίζ)
- Τη χρονική κατανομή δραστηριοτήτων (π.χ. πότε οι φοιτητές είναι πιο ενεργοί)
- Την αποτελεσματικότητα του εκπαιδευτικού σχεδιασμού

Τα Logs, πέρα από την καθαρά εκπαιδευτική τους αξία, αποτελούν και σημαντική πηγή δεδομένων για ερευνητικούς σκοπούς στο πλαίσιο του Educational Data Mining (EDM) και του Learning Analytics (LA) [4]. Τα δεδομένα αλληλεπίδρασης που καταγράφονται σε LMS όπως το Moodle μπορούν να αξιοποιηθούν για την ανάλυση συμμετοχής και τη διερεύνηση προτύπων συμπεριφοράς, ενώ σε ευρύτερο πλαίσιο τέτοιου τύπου log δεδομένα μπορούν να αποτελέσουν βάση για μεθόδους μοντελοποίησης/πρόβλεψης ή εντοπισμού αποκλίσεων, ανάλογα με τους στόχους και τη διαθέσιμη πληροφορία [1], [2].



Εικόνα 3: Διάγραμμα ροής δεδομένων από το Moodle προς τα Logs και στη συνέχεια προς ανάλυση/επεξεργασία

Τεχνικά, τα Moodle Logs αποθηκεύονται στη βάση δεδομένων του Moodle, συνήθως σε πίνακες, και μπορούν να εξαχθούν μέσω του γραφικού περιβάλλοντος της πλατφόρμας ή απευθείας μέσω SQL queries. Κάθε εγγραφή συνοδεύεται από πεδία που περιγράφουν το γεγονός, όπως eventname, component, action, target, userid, courseid και timecreated. Αυτή η λεπτομερής καταγραφή καθιστά τα logs ιδανική πηγή για ανάλυση συμπεριφορών, ανίχνευση προβλημάτων ή προσαρμογή του περιεχομένου της μάθησης στις ανάγκες των εκπαιδευομένων.

Επιπλέον, τα Moodle Logs συμβάλλουν καθοριστικά στην παρακολούθηση της ασφάλειας και της διαφάνειας του συστήματος. Μέσω αυτών, ο διαχειριστής μπορεί να εντοπίζει μη εξουσιοδοτημένες προσβάσεις, ύποπτες ενέργειες ή τεχνικά σφάλματα, προσφέροντας έτσι ένα επίπεδο ελέγχου που είναι κρίσιμο για την ομαλή λειτουργία ενός LMS.

Συνοψίζοντας, τα Moodle Logs αποτελούν τον πυρήνα των δεδομένων δραστηριότητας μέσα σε ένα εκπαιδευτικό σύστημα. Η σωστή κατανόηση, εξαγωγή και ανάλυσή τους επιτρέπουν τη μετάβαση από τη συλλογή δεδομένων στη μετουσίωση των δεδομένων σε γνώση, που μπορεί να οδηγήσει σε καλύτερες παιδαγωγικές αποφάσεις, πιο στοχευμένες παρεμβάσεις και ενίσχυση της μαθησιακής εμπειρίας.



Εικόνα 4: Θερμικός χάρτης δραστηριότητας από Moodle Logs ανά χρήστη και μήνα. Το χρώμα δείχνει το πλήθος events ανά ημέρα (y: ημέρα εβδομάδας, x: εβδομάδα μήνα)

2.2.1 Πώς χρησιμοποιούνται τα Moodle Logs

Τα **Moodle Logs** αποτελούν ένα από τα πιο σημαντικά εργαλεία που προσφέρει η πλατφόρμα Moodle για την παρακολούθηση, αξιολόγηση και ανάλυση της εκπαιδευτικής δραστηριότητας. Μέσα από την καταγραφή κάθε ενέργειας των χρηστών, παρέχουν μια λεπτομερή εικόνα της αλληλεπίδρασης των εκπαιδευομένων και των εκπαιδευτών με το εκπαιδευτικό περιβάλλον. Η αξιοποίηση των Logs δεν περιορίζεται μόνο στη διαχείριση ή στην επίλυση τεχνικών ζητημάτων, αλλά εκτείνεται σε ερευνητικές, διοικητικές και παιδαγωγικές εφαρμογές.

Εκπαιδευτική και Παιδαγωγική Χρήση

Μία από τις βασικότερες εφαρμογές των Moodle Logs είναι η **ανάλυση της μαθησιακής συμπεριφοράς** των φοιτητών. Μέσα από την παρακολούθηση της συχνότητας σύνδεσης, των υποβολών εργασιών, της συμμετοχής σε κουίζ και φόρουμ, μπορεί να εκτιμηθεί το επίπεδο δέσμευσης (engagement) των φοιτητών σε κάθε μάθημα. Η πληροφορία αυτή βοηθά τους εκπαιδευτές να εντοπίσουν:

- Φοιτητές που παρουσιάζουν χαμηλή συμμετοχή ή καθυστερημένες υποβολές
- Περιόδους αυξημένης δραστηριότητας που μπορεί να συνδέονται με παραδόσεις ή εξετάσεις
- Μοτίβα συμμετοχής που σχετίζονται με την επίδοση των φοιτητών

Η συστηματική ανάλυση των Logs μπορεί να οδηγήσει στη βελτίωση της παιδαγωγικής προσέγγισης: π.χ. ενίσχυση δραστηριοτήτων που προκαλούν μεγαλύτερη αλληλεπίδραση ή ανασχεδιασμό μαθημάτων με χαμηλά επίπεδα συμμετοχής.

☐ First name / Surname	Last in course	Completion Progress
☐ Student 1	Tuesday, 17 December 2019, 1:06 PM	
☐ Student 2	Tuesday, 17 March 2020, 3:33 PM	 Enroll for your EYL project Completed Expected: 20 September 2019
☐ Student 3	Sunday, 19 April 2020, 5:11 PM	
☐ Student 4	Sunday, 12 January 2020, 3:31 PM	
☐ Student 5	Friday, 24 January 2020, 8:41 AM	 Enroll for your EYL project Completed Expected: 20 September 2019

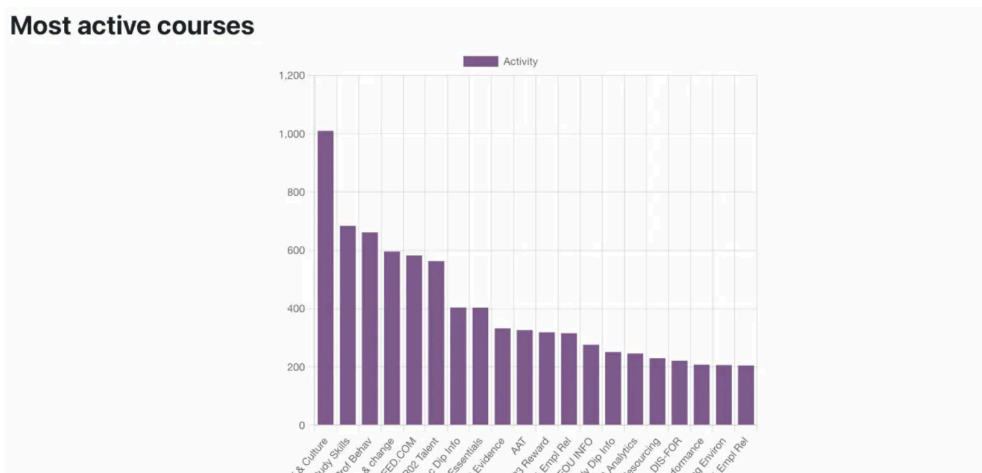
Εικόνα 5: Παράδειγμα οπτικοποίησης συμμετοχής φοιτητών ανά δραστηριότητα

Διαχειριστική και Τεχνική Χρήση

Από τη σκοπιά των διαχειριστών συστήματος (administrators), τα Moodle Logs χρησιμοποιούνται για την παρακολούθηση της λειτουργίας και της ασφάλειας της πλατφόρμας. Κάθε είσοδος, αποσύνδεση ή τεχνικό σφάλμα καταγράφεται, επιτρέποντας την ανίχνευση προβλημάτων όπως αποτυχημένες συνδέσεις, υπερφόρτωση σελίδων ή κακόβουλες προσπάθειες πρόσβασης.

Με τον τρόπο αυτό, οι διαχειριστές μπορούν να:

- Εντοπίζουν **προβλήματα απόδοσης** (π.χ. καθυστερήσεις φόρτωσης συγκεκριμένων μαθημάτων)
- Ελέγχουν **προσπάθειες παραβίασης** ή μη εξουσιοδοτημένες ενέργειες
- Παρακολουθούν τη **συνολική χρήση του συστήματος** και να σχεδιάζουν βελτιώσεις υποδομής



Εικόνα 6: Παράδειγμα πίνακα αναφοράς συστήματος με στατιστικά από Moodle Logs

Ερευνητική Χρήση

Στο πλαίσιο της εκπαιδευτικής έρευνας, τα Moodle Logs αποτελούν πολύτιμη πηγή δεδομένων για το πεδίο του *LA* και του *EDM*. Ερευνητές χρησιμοποιούν τα δεδομένα αυτά για να μελετήσουν τη σχέση μεταξύ συμπεριφοράς και επίδοσης, να αναπτύξουν μοντέλα πρόβλεψης ή να ανιχνεύσουν πρότυπα συνεργασίας και αυτορρύθμισης στη μάθηση.

Ενδεικτικές εφαρμογές περιλαμβάνουν:

- Πρόβλεψη επιτυχίας ή αποτυχίας φοιτητών βάσει της δραστηριότητάς τους
- Ταξινόμηση τύπων συμπεριφοράς (π.χ. ενεργός, παθητικός, καθυστερημένος συμμετέχων)

- Ανίχνευση ανωμαλιών στη δραστηριότητα, όπως απότομες μειώσεις συμμετοχής ή μη ρεαλιστικές συμπεριφορές (π.χ. πλήρης ολοκλήρωση κουίζ σε λίγα δευτερόλεπτα)

2.2.2 Παράμετροι Moodle Logs

Τα Moodle Logs αποτελούνται από ένα πλήθος παραμέτρων που περιγράφουν με ακρίβεια κάθε ενέργεια που πραγματοποιείται μέσα στην πλατφόρμα Moodle. Οι παράμετροι αυτοί είναι οργανωμένες με τρόπο ώστε να παρέχουν τόσο τεχνικές όσο και παιδαγωγικές πληροφορίες, καθιστώντας δυνατή την αναλυτική αποτύπωση της συμπεριφοράς των χρηστών. Η κατανόηση των πεδίων που περιέχουν τα Logs είναι καθοριστικής σημασίας για την επεξεργασία, καθαρισμό και ανάλυση των δεδομένων, καθώς επιτρέπει την επιλογή των πιο σχετικών πληροφοριών για κάθε είδος ανάλυσης.

Κύριες παράμετροι των Moodle Logs

Κάθε εγγραφή (record) στο Moodle Log αντιστοιχεί σε ένα γεγονός (event) που πραγματοποιήθηκε από έναν χρήστη σε ένα συγκεκριμένο χρονικό σημείο. Οι πιο βασικές παράμετροι που συναντώνται είναι οι εξής:

Στήλη	Περιγραφή
id	Μοναδικό αναγνωριστικό κάθε εγγραφής στο σύνολο δεδομένων. Αντιστοιχεί σε ένα μεμονωμένο γεγονός (event) που καταγράφηκε στο Moodle
eventname	Ονομασία του γεγονότος που έλαβε χώρα στο Moodle. Περιγράφει το είδος της ενέργειας, π.χ. <code>\core\event\course_viewed</code>
component	Υποσύστημα ή ενότητα του Moodle στην οποία ανήκει το γεγονός (π.χ. <code>mod_quiz</code> για κουίζ)
action	Η συγκεκριμένη ενέργεια που πραγματοποιήθηκε, όπως <i>viewed</i> (προβολή), <i>created</i> (δημιουργία) ή <i>updated</i> (ενημέρωση)
target	Το αντικείμενο στο οποίο αναφέρεται η ενέργεια (π.χ. <i>course</i> , <i>user</i> , <i>assignment</i> , <i>forum post</i>). Χρησιμοποιείται για την αναγνώριση του τύπου οντότητας που επηρεάστηκε
crud	Συνοπτική παράμετρος που περιγράφει το είδος της ενέργειας με βάση το πρότυπο CRUD (<i>Create</i> , <i>Read</i> , <i>Update</i> , <i>Delete</i>). Για παράδειγμα, "r" δηλώνει ανάγνωση (read) και "c" δημιουργία (create)
edulevel	Υποδεικνύει το εκπαιδευτικό επίπεδο του γεγονότος. Συνήθως παίρνει τιμές όπως 0 (άλλο), 1 (συμμετοχή σε μάθημα) ή 2 (συμμετοχή σε δραστηριότητα), επιτρέποντας τη διάκριση μεταξύ διοικητικών και μαθησιακών ενεργειών
contextid	Αναγνωριστικό του περιβάλλοντος (context) στο οποίο ανήκει η ενέργεια. Το Moodle ορίζει context για κάθε τύπο αντικειμένου, όπως μαθήματα, δραστηριότητες ή χρήστες

contextlevel	Το επίπεδο του context όπου συνέβη η ενέργεια, π.χ. <i>course</i> , <i>module</i> ή <i>user</i> . Είναι κρίσιμο για την κατηγοριοποίηση της δραστηριότητας στο σωστό ιεραρχικό επίπεδο
contextinstanceid	Αναγνωριστικός αριθμός της συγκεκριμένης περίπτωσης context. Συνδέει το γεγονός με το ακριβές αντικείμενο (π.χ. το ID του μαθήματος ή της δραστηριότητας)
userid	Ο μοναδικός αριθμός ταυτοποίησης του χρήστη που εκτέλεσε την ενέργεια. Είναι κεντρικό πεδίο για αναλύσεις συμμετοχής και προτύπων συμπεριφοράς
courseid	Το αναγνωριστικό του μαθήματος στο οποίο ανήκει το γεγονός. Επιτρέπει την ομαδοποίηση των ενεργειών ανά μάθημα
relateduserid	Ο χρήστης που επηρεάστηκε από την ενέργεια (αν υπάρχει). Για παράδειγμα, αν ο εκπαιδευτής βαθμολογήσει έναν φοιτητή, ο φοιτητής είναι το relateduserid
anonymous	Δείκτης ανώνυμης ενέργειας (boolean). Όταν έχει τιμή 1 ή True, η ενέργεια καταγράφεται χωρίς να εμφανίζεται το αναγνωριστικό του χρήστη, π.χ. σε ανώνυμα ερωτηματολόγια ή αξιολογήσεις
timecreated	Χρονική στιγμή δημιουργίας του γεγονότος (timestamp σε μορφή Unix epoch). Είναι θεμελιώδης για χρονολογική ανάλυση και δημιουργία χρονοσειρών
origin	Προσδιορίζει την προέλευση του γεγονότος, δηλαδή από ποιο μέσο πραγματοποιήθηκε: <i>web</i> , <i>cli</i> (command line), <i>web</i> (web service), ή <i>restore</i>

	id	eventname	component	action	target	crud	edulevel	contextid	contextlevel	contextinstanceid	userid	courseid	relateduserid	anonymous	timecreated	origin
	bigint	text	text	text	text	text	integer	bigint	integer	bigint	bigint	bigint	bigint	boolean	bigint	text
1	75279739	\mod_quiz\event\attempt_viewed	mod_quiz	viewed	attempt	read	2	114415	70	68252	7642	532	7642	false	1583959488	web
2	75279866	\core\event\course_viewed	core	viewed	course	read	2	2	50	1	0	1	[null]	false	1583959609	web
3	75279916	\core\event\course_module_completion_updated	core	updated	course_module_completi...	upd...	2	114670	70	68405	4247	532	4247	false	1583959646	web
4	75280323	\mod_quiz\event\attempt_started	mod_quiz	started	attempt	crea...	2	178817	70	106119	12664	2260	12664	false	1583959936	web
5	75280542	\mod_quiz\event\attempt_viewed	mod_quiz	viewed	attempt	read	2	114415	70	68252	5833	532	5833	false	1583960067	web
6	75280706	\mod_quiz\event\course_module_viewed	mod_quiz	viewed	course_module	read	2	62970	70	35346	6063	615	[null]	false	1583960142	web
7	75280847	\mod_assign\event\course_module_viewed	mod_assign	viewed	course_module	read	2	146105	70	90856	12553	1963	[null]	false	1583960200	web
8	75280898	\mod_url\event\course_module_viewed	mod_url	viewed	course_module	read	2	151468	70	95624	4199	2021	[null]	false	1583960238	web
9	75280899	\mod_quiz\event\attempt_viewed	mod_quiz	viewed	attempt	read	2	114415	70	68252	4417	532	4417	false	1583960238	web
10	75280908	\mod_assign\event\course_module_viewed	mod_assign	viewed	course_module	read	2	112260	70	66970	5319	532	[null]	false	1583960245	web
11	75281176	\mod_quiz\event\attempt_summary_viewed	mod_quiz	viewed	attempt_summary	read	1	114415	70	68252	4569	532	4569	false	1583960407	web
12	75281197	\mod_page\event\course_module_viewed	mod_page	viewed	course_module	read	2	147730	70	92347	4613	1942	[null]	false	1583960426	web
13	75281214	\mod_quiz\event\course_module_viewed	mod_quiz	viewed	course_module	read	2	209447	70	129639	4300	2659	[null]	false	1583960441	web
14	75281620	\assignsubmission_file\event\assessable_upload...	assignsubmission_f...	upload...	assessable	crea...	2	112242	70	66962	5082	532	[null]	false	1583960833	web
15	75281765	\mod_resource\event\course_module_viewed	mod_resource	viewed	course_module	read	2	266490	70	164103	33	2063	[null]	false	1583960924	web
16	75281781	\mod_assign\event\submission_form_viewed	mod_assign	viewed	submission_form	read	0	146105	70	90856	12534	1963	12534	false	1583960932	web
17	75282091	\core\event\course_viewed	core	viewed	course	read	2	98730	50	1263	14040	1263	[null]	false	1583961116	web
18	75282177	\mod_resource\event\course_module_viewed	mod_resource	viewed	course_module	read	2	78171	70	45030	13444	869	[null]	false	1583961164	web
19	75282786	\mod_assign\event\submission_status_viewed	mod_assign	viewed	submission_status	read	0	146105	70	90856	12057	1963	[null]	false	1583961542	web
20	75282859	\mod_assign\event\submission_status_viewed	mod_assign	viewed	submission_status	read	0	112242	70	66962	4279	532	[null]	false	1583961596	web

Εικόνα 7: Απόσπασμα του πίνακα από την βάση δεδομένων που χρησιμοποιήθηκε στην παρούσα εργασία (ύστερα από την εκκαθάριση των δεδομένων)

Παράλληλα, συχνά συναντάμε και άλλα είδη παραμέτρων σε Moodle Logs με τα οποία όμως δεν θα ασχοληθούμε σε αυτή την εργασία. Μερικά από αυτά είναι:

Στήλη	Περιγραφή
ip	Περιέχει τη διεύθυνση IP του χρήστη που εκτέλεσε την ενέργεια. Αν και χρήσιμη για αναλύσεις ασφάλειας και εντοπισμό τοποθεσίας, συχνά περιέχει πολλά κενά ή ανώνυμα δεδομένα, ειδικά όταν η πρόσβαση γίνεται μέσω VPN ή κινητών συσκευών. Για το λόγο αυτό αφαιρέθηκε
realuserid	Αντιπροσωπεύει τον αρχικό χρήστη όταν κάποιος άλλος ενεργεί εκ μέρους του (π.χ. όταν ένας διαχειριστής πραγματοποιεί ενέργεια στο σύστημα με τα δικαιώματα άλλου χρήστη). Επειδή οι περισσότερες ενέργειες στο dataset δεν περιείχαν τέτοια σενάρια, η στήλη παρέμενε σχεδόν πλήρως κενή και δεν συνέβαλε στην ανάλυση, οπότε αφαιρέθηκε
objectid	Ο μοναδικός αριθμός αναφοράς του αντικειμένου στο οποίο έγινε η ενέργεια. Παρότι θα μπορούσε να χρησιμεύσει στη συσχέτιση με άλλους πίνακες, στο dataset υπήρχαν πολλές ελλείψεις ή μηδενικές τιμές, καθιστώντας το μη αξιόπιστο
objecttable	Δηλώνει τον πίνακα της βάσης δεδομένων που σχετίζεται με το γεγονός. Η πληροφορία αυτή δεν ήταν ολοκληρωμένη και δεν χρησιμοποιήθηκε στην ανάλυση, οπότε αφαιρέθηκε
other	Περιέχει πρόσθετα, μη δομημένα δεδομένα (συνήθως σε μορφή JSON) που συνδέονται με το γεγονός. Παρότι ενίοτε περιλαμβάνει χρήσιμα μεταδεδομένα, στο παρόν dataset η στήλη περιείχε μη αναγνώσιμες ή ασύμβατες τιμές, οπότε εξαιρέθηκε από την τελική ανάλυση

	ip	realuserid	objectid	objecttable	other
0	158.129.0.50	NaN	NaN	<NA>	<NA>
1	<NA>	NaN	40861.0	event	{"repeatid":0,"timestart":"1576047540","name":...
2	<NA>	NaN	45038.0	event	{"repeatid":0,"timestart":"1570024800","name":...
3	<NA>	NaN	45039.0	event	{"repeatid":0,"timestart":"1575459000","name":...
4	<NA>	NaN	45290.0	event	{"repeatid":0,"timestart":"1576137600","name":...
5	<NA>	NaN	45291.0	event	{"repeatid":0,"timestart":"1576738800","name":...
6	<NA>	NaN	45292.0	event	{"repeatid":0,"timestart":"1576562400","name":...
7	<NA>	NaN	59919.0	event	{"repeatid":0,"timestart":"1573629960","name":...
8	<NA>	NaN	59920.0	event	{"repeatid":0,"timestart":"1573629960","name":...
9	<NA>	NaN	45392.0	event	{"repeatid":0,"timestart":"1575666000","name":...
10	<NA>	NaN	45393.0	event	{"repeatid":0,"timestart":"1576681200","name":...

Εικόνα 8: Απόσπασμα του πίνακα από την βάση δεδομένων που χρησιμοποιήθηκε στην παρούσα εργασία (κενές, ελλειπείς ή περιττές στήλες)

Οι παράμετροι αυτοί βρίσκονται αποθηκευμένοι στον πίνακα **moodle_data**, ο οποίος αποτελεί επεξεργασμένη εκδοχή του πίνακα καταγραφών του Moodle. Η πληρότητα και λεπτομέρειά τους επιτρέπει την ανακατασκευή της μαθησιακής δραστηριότητας με μεγάλη ακρίβεια.

Ομαδοποίηση Παραμέτρων

Οι παράμετροι των Moodle Logs μπορούν να ομαδοποιηθούν ανάλογα με τη λειτουργία τους σε τρεις βασικές κατηγορίες:

- Παράμετροι Ταυτοποίησης Χρήστη και Μαθήματος**
 Αυτές οι παράμετροι (όπως *userid*, *courseid*, *relateduserid*) καθορίζουν ποιος πραγματοποίησε την ενέργεια και σε ποιο μάθημα. Είναι απαραίτητες για αναλύσεις που αφορούν τη συμμετοχή ή την παρακολούθηση φοιτητών.
- Παράμετροι Περιγραφής Ενέργειας**
 Οι παράμετροι *eventname*, *action*, *target* και *component* καθορίζουν τι συνέβη και πού συνέβη. Χρησιμοποιούνται κυρίως για τον προσδιορισμό του τύπου δραστηριότητας (π.χ. φόρουμ, κουίζ, εργασία) και τη φύση της ενέργειας (προβολή, δημιουργία, διαγραφή).
- Παράμετροι Χρόνου και Τοποθεσίας**
 Οι *timecreated* και *ip* παρέχουν πληροφορίες για το πότε και από πού πραγματοποιήθηκε η ενέργεια. Αυτές οι μεταβλητές είναι εξαιρετικά χρήσιμες για ανάλυση χρονικών προτύπων ή για ανίχνευση ανωμαλιών στη συμπεριφορά χρηστών.

Χρήση Παραμέτρων στην Ανάλυση

Κατά την ανάλυση των Moodle Logs, οι παράμετροι αυτές αποτελούν τη βάση για κάθε τύπο στατιστικής ή μηχανικής επεξεργασίας.

Για παράδειγμα:

- Ο συνδυασμός των *userid*, *courseid* και *timecreated* επιτρέπει τη δημιουργία χρονοσειρών δραστηριότητας ανά φοιτητή.
- Οι παράμετροι *eventname* και *component* μπορούν να χρησιμοποιηθούν για τη δημιουργία κατηγοριών ενεργειών, π.χ. "παρακολούθηση περιεχομένου" ή "συμμετοχή σε αξιολόγηση".
- Η παράμετρος *ip* είναι χρήσιμη σε αναλύσεις ασφάλειας, για τον εντοπισμό ύποπτων προτύπων πρόσβασης (όπως πολλαπλές συνδέσεις από διαφορετικές χώρες σε μικρό χρονικό διάστημα).

Η ορθή επιλογή και κατανόηση των παραμέτρων επιτρέπει τη μετατροπή των ακατέργαστων logs σε ποιοτικά δεδομένα, έτοιμα για επεξεργασία με εργαλεία όπως το PostgreSQL, η Python (με βιβλιοθήκες όπως το *pandas*), ή το Power BI για οπτικοποίηση.

Συνοψίζοντας, οι παράμετροι των Moodle Logs αποτελούν τη «γλώσσα» μέσω της οποίας η πλατφόρμα επικοινωνεί τις ενέργειες των χρηστών. Η κατανόησή τους δεν είναι μόνο απαραίτητη για την τεχνική ανάλυση, αλλά και για την εξαγωγή παιδαγωγικών συμπερασμάτων, συμβάλλοντας στη σύνδεση της τεχνολογίας με τη βελτίωση της μαθησιακής εμπειρίας.

2.3 Καθαρισμός και Προεπεξεργασία Δεδομένων

Η διαδικασία καθαρισμού και προεπεξεργασίας δεδομένων αποτελεί θεμελιώδες στάδιο σε κάθε ανάλυση δεδομένων, καθώς εξασφαλίζει ότι το τελικό σύνολο δεδομένων είναι αξιόπιστο, ομοιογενές και αντιπροσωπευτικό των πραγματικών γεγονότων.

Η βάση περιλάμβανε 58.153.450 εγγραφές (rows), αντιστοιχώντας σε γεγονότα χρηστών και συστήματος. Μετά την ολοκλήρωση της διαδικασίας καθαρισμού, το τελικό πλήθος εγγραφών μειώθηκε σε 2.477.467, αισθητά μικρότερο μέγεθος από το αρχικό. Ένα μεγάλο μέρος της μείωσης προκλήθηκε από την αφαίρεση των σειρών όπου στην στήλη "origin" είχαν την καταχώρηση "cli", το οποίο υποδεικνύει πως αποτελούσαν διαχειριστικές ή συστημικές ενέργειες και όχι ενέργειες χρηστών.



Εικόνα 9: Συνοπτικό διάγραμμα ροής και προεπεξεργασίας Moodle Logs

2.3.1 Διαδικασίες Καθαρισμού

Η διαδικασία καθαρισμού πραγματοποιήθηκε σε πολλαπλά στάδια, με στόχο την αφαίρεση περιττών, ελλιπών ή τεχνικών δεδομένων και τη διαμόρφωση ενός συνόλου κατάλληλου για ανάλυση συμπεριφοράς χρηστών. Ακολουθήθηκαν οι παρακάτω ενέργειες:

Απομάκρυνση ενεργειών τύπου "cli"

Κατά την εξέταση της στήλης "origin", εντοπίστηκαν εγγραφές που είχαν τιμή cli (Command Line Interface). Οι συγκεκριμένες καταχωρήσεις αντιστοιχούν σε αυτόματες ή διαχειριστικές ενέργειες του συστήματος, όπως scripts συντήρησης ή ενέργειες bot και όχι σε ενέργειες πραγματικών χρηστών.

Επειδή ο στόχος της παρούσας ανάλυσης είναι η μελέτη συμπεριφοράς φοιτητών και εκπαιδευτών, οι εγγραφές αυτές αφαιρέθηκαν εξ ολοκλήρου.

Αφαίρεση διπλότυπων εγγραφών

Αρχικά, εντοπίστηκαν και αφαιρέθηκαν όλες οι διπλότυπες εγγραφές από την βάση δεδομένων, δηλαδή γραμμές που ήταν ακριβώς ίδιες σε όλα τα πεδία. Η ύπαρξη τέτοιων εγγραφών συνήθως προκύπτει από πολλαπλές εξαγωγές ή επανακαταγραφές στο Moodle. Η αφαίρεσή τους διασφαλίζει ότι κάθε γεγονός εκπροσωπείται μία και μόνο φορά, αποτρέποντας στρεβλώσεις στα στατιστικά αποτελέσματα.

Εντοπισμός και αφαίρεση στηλών με μεγάλο ποσοστό κενών τιμών

Για κάθε στήλη του πίνακα πραγματοποιήθηκε υπολογισμός του ποσοστού κενών τιμών. Οι στήλες που παρουσίασαν υψηλό ποσοστό κενών ή ασυνεπών δεδομένων κρίθηκαν μη χρήσιμες και αφαιρέθηκαν. Η απόφαση βασίστηκε τόσο στη στατιστική πληρότητα όσο και στη συνάφεια της πληροφορίας με το αντικείμενο της έρευνας.

Ο πίνακας που ακολουθεί παρουσιάζει τις στήλες που αφαιρέθηκαν και τα ποσοστά των κενών εγγραφών που παρατηρήθηκαν:

Στήλη	Ποσοστό κενών τιμών	Αιτιολόγηση
ip	3.96 %	Αν και το ποσοστό κενών ήταν σχετικά μικρό, η στήλη δεν προσέφερε ουσιαστική πληροφορία για τη μελέτη (η τοποθεσία του χρήστη δεν είναι ζητούμενο)
realuserid	99.97 %	Περιέχει δεδομένα μόνο όταν μια ενέργεια εκτελείται εκ μέρους άλλου χρήστη (π.χ. διαχειριστή). Εξαιρετικά σπάνια, επομένως αφαιρέθηκε
objectid	47.91 %	Μη πλήρης στήλη, χωρίς δυνατότητα συσχέτισης χωρίς το <i>objecttable</i>
objecttable	47.91 %	Συνδεδεμένη με το <i>objectid</i> , αλλά με ελλιπή στοιχεία. Αφαιρέθηκε για λόγους συνέπειας
other	57.44 %	Περιείχε μη δομημένες πληροφορίες (JSON) διαφορετικού τύπου ανά εγγραφή, μη αξιοποιήσιμες χωρίς εξειδικευμένη επεξεργασία

Ο πίνακας που ακολουθεί παρουσιάζει όλες τις στήλες της βάσης δεδομένων, το σύνολο των κενών εγγραφών για την κάθε μια, καθώς και τα εκάστοτε ποσοστά κενών εγγραφών:

columns	missing_count	missing_percent
id	0	0.000000
eventname	0	0.000000
component	0	0.000000
action	0	0.000000
target	0	0.000000
objecttable	27859951	47.907650
objectid	27859951	47.907650
crud	0	0.000000
edulevel	0	0.000000

columns	missing_count	missing_percent
contextlevel	0	0.000000
contextinstanceid	0	0.000000
userid	0	0.000000
courseid	0	0.000000
relateduserid	45234212	77.784228
anonymous	0	0.000000
other	33402333	57.438265
timecreated	0	0.000000
origin	0	0.000000

contextid	0	0.000000	ip	2302633	3.959581
realuserid	58134735	99.967818			

Μετατροπή χρονικών δεδομένων

Η στήλη "timecreated" στο Moodle αποθηκεύει τον χρόνο δημιουργίας κάθε γεγονότος σε μορφή Unix Epoch Timestamp (δηλαδή αριθμητική αναπαράσταση δευτερολέπτων από την 1η Ιανουαρίου 1970).

Για τη διευκόλυνση της ανάλυσης και οπτικοποίησης, το πεδίο αυτό μετατράπηκε σε διάφορες χρονικές μορφές, προσθέτοντας νέες στήλες:

Νέα Στήλη	Περιγραφή
datetime	Αναπαριστά τον χρόνο σε μορφή ημερομηνίας και ώρας (π.χ. 2020-01-30 02:13:56)
time	Περιλαμβάνει μόνο την ώρα της ημέρας (π.χ. 02:13:56), χρήσιμη για ανάλυση δραστηριότητας ανά ώρα
day	Αντιπροσωπεύει την ημέρα της εβδομάδας (π.χ. Monday), επιτρέποντας την αναζήτηση προτύπων ημερήσιας και εβδομαδιαίας δραστηριότητας
month	Ο μήνας στον οποίο έλαβε χώρα το γεγονός (π.χ. January), χρήσιμος για εποχιακές συγκρίσεις

Η αρχική στήλη "timecreated" διατηρήθηκε ώστε να είναι δυνατή η εναλλακτική χρήση της σε χρονικά ερωτήματα ή υπολογισμούς.

timecreated bigint	datetime timestamp with time zone	time time without time zone	day text	month text
1583959488	2020-03-11 20:44:48+02	20:44:48	Wednesd...	March
1583959609	2020-03-11 20:46:49+02	20:46:49	Wednesd...	March
1583959646	2020-03-11 20:47:26+02	20:47:26	Wednesd...	March
1583959936	2020-03-11 20:52:16+02	20:52:16	Wednesd...	March
1583960067	2020-03-11 20:54:27+02	20:54:27	Wednesd...	March
1583960142	2020-03-11 20:55:42+02	20:55:42	Wednesd...	March
1583960200	2020-03-11 20:56:40+02	20:56:40	Wednesd...	March
1583960238	2020-03-11 20:57:18+02	20:57:18	Wednesd...	March
1583960238	2020-03-11 20:57:18+02	20:57:18	Wednesd...	March
1583960245	2020-03-11 20:57:25+02	20:57:25	Wednesd...	March

Εικόνα 10: Μετατροπή της στήλης timecreated σε datetime, time, day, month

2.3.2 Τελική Μορφή Δεδομένων

Μετά την ολοκλήρωση της διαδικασίας καθαρισμού, το τελικό dataset αποτελείται από 2.477.467 εγγραφές και 20 στήλες, οι οποίες παρατίθενται παρακάτω:

id, eventname, component, action, target, crud, edulevel, contextid, contextlevel, contextinstanceid, userid, courseid, relateduserid, anonymous, timecreated, origin, datetime, time, day, month

Οι στήλες που θα χρησιμοποιούνται συχνότερα για την απάντηση των ερωτήσεων είναι οι εξής:

- **id:** Μοναδικό αναγνωριστικό κάθε εγγραφής στο σύνολο δεδομένων. Αντιστοιχεί σε ένα μεμονωμένο γεγονός (event) που καταγράφηκε στο Moodle. Χρησιμοποιείται για μοναδικοποίηση/έλεγχο διπλοεγγραφών και για αξιόπιστη καταμέτρηση συμβάντων.
- **eventname:** Ονομασία του γεγονότος που έλαβε χώρα στο Moodle. Περιγράφει το είδος της ενέργειας (π.χ. \core\event\course_viewed) και επιτρέπει πιο λεπτομερή κατηγοριοποίηση της δραστηριότητας ανά τύπο συμβάντος.
- **action:** Η συγκεκριμένη ενέργεια που πραγματοποιήθηκε (π.χ. viewed, created, updated). Χρησιμοποιείται για να μετρηθεί και να συγκριθεί η φύση της δραστηριότητας (προβολές vs δημιουργίες/υποβολές κ.λπ.).
- **userid:** Ο μοναδικός αριθμός ταυτοποίησης του χρήστη που εκτέλεσε την ενέργεια. Είναι κεντρικό πεδίο για αναλύσεις συμμετοχής, μοτίβων συμπεριφοράς και δημιουργία προφίλ δραστηριότητας ανά φοιτητή/χρήστη.
- **courseid:** Το αναγνωριστικό του μαθήματος στο οποίο ανήκει το γεγονός. Επιτρέπει ομαδοποίηση και σύγκριση ενεργειών ανά μάθημα, αλλά και φιλτράρισμα της δραστηριότητας σε επίπεδο συγκεκριμένου μαθήματος.
- **timecreated:** Χρονική στιγμή δημιουργίας του γεγονότος σε μορφή Unix epoch. Είναι θεμελιώδες για χρονολογική ανάλυση (σειρά γεγονότων, peaks δραστηριότητας, ρυθμός συμμετοχής) και για δημιουργία χρονοσειρών.
- **datetime:** Αναπαριστά τον χρόνο σε μορφή ημερομηνίας και ώρας (π.χ. 2020-01-30 02:13:56). Χρησιμοποιείται για καλύτερη ανάγνωση/αναφορές και για χρονικά φίλτρα/ομαδοποιήσεις χωρίς μετατροπές από epoch. (Παράγεται από μετασχηματισμό του timecreated στο πλαίσιο του καθαρισμού.)

Η σημαντική μείωση του όγκου των δεδομένων (από 58.153.450 σε 2.477.467 εγγραφές) βελτίωσε δραστικά την ταχύτητα εκτέλεσης ερωτημάτων, τη σταθερότητα της βάσης και την ποιότητα της ανάλυσης. Παράλληλα, διατηρήθηκαν όλες οι κρίσιμες πληροφορίες που σχετίζονται με τη συμπεριφορά των χρηστών και τη χρονική εξέλιξη των ενεργειών τους.

2.3.3 Συμπεράσματα Καθαρισμού

Η διαδικασία καθαρισμού και προεπεξεργασίας που ακολουθήθηκε εξασφάλισε ότι το τελικό dataset:

- Περιέχει μόνο γεγονότα που σχετίζονται με πραγματικούς χρήστες, καθώς αφαιρέθηκαν οι εγγραφές που προέκυψαν από συστημικές/αυτοματοποιημένες ενέργειες (αφαίρεση καταχωρήσεων που περιείχαν "cli" στην στήλη "origin")

- Διαθέτει συνεπή και πλήρη δομή, επειδή αφαιρέθηκαν διπλότυπες εγγραφές (ώστε κάθε γεγονός να εκπροσωπείται μία φορά και να αποφεύγονται στρεβλώσεις στις μετρήσεις), καθώς και πεδία με υψηλά ποσοστά ελλειπών τιμών ή περιορισμένη χρησιμότητα για τα ερευνητικά ερωτήματα. Διατηρήθηκαν εξαιρέσεις όπου η ύπαρξη κενών τιμών είναι αναμενόμενη λόγω της φύσης των δεδομένων (π.χ. relateduserid)
- Είναι κατάλληλο για περαιτέρω ανάλυση, τόσο ποσοτική όσο και χρονική

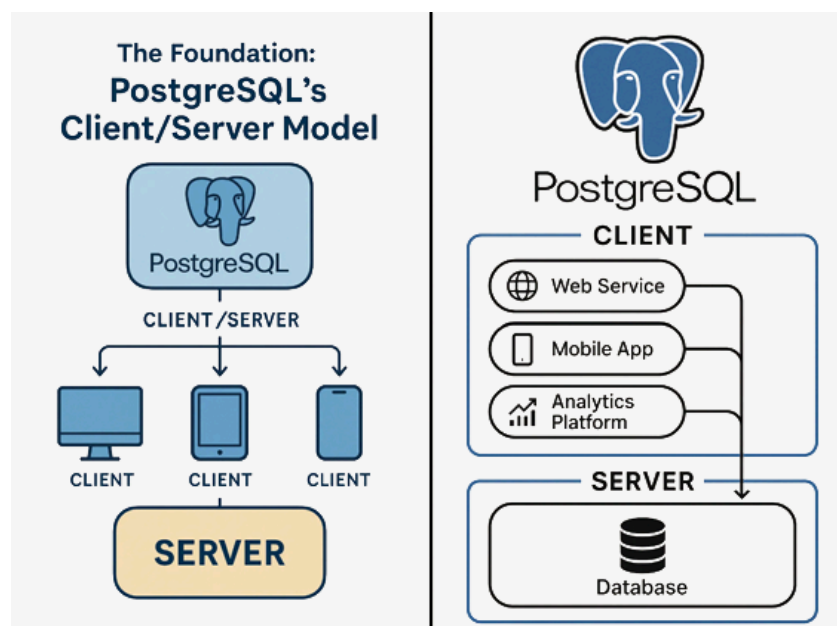
Η ποιότητα του καθαρισμένου συνόλου δεδομένων αποτελεί τη βάση για τις επόμενες ενότητες, όπου θα περιγραφεί αναλυτικά η διαδικασία επεξεργασίας και ανάλυσης μέσω PostgreSQL και Python.

2.4 PostgreSQL

Η PostgreSQL αποτελεί ένα από τα πιο ισχυρά και ευέλικτα συστήματα διαχείρισης σχεσιακών βάσεων δεδομένων (RDBMS) ανοικτού κώδικα. Διακρίνεται για τη σταθερότητα, την επεκτασιμότητα και τη συμμόρφωσή της με τα πρότυπα της γλώσσας SQL.

Χάρη στα χαρακτηριστικά αυτά, χρησιμοποιείται ευρέως σε ερευνητικά και βιομηχανικά περιβάλλοντα, όπου απαιτείται διαχείριση μεγάλου όγκου δεδομένων με υψηλή αξιοπιστία και ταχύτητα.

Στην παρούσα εργασία, η PostgreSQL χρησιμοποιήθηκε ως το κύριο περιβάλλον για την αποθήκευση, οργάνωση και προετοιμασία των δεδομένων που εξήχθησαν από το Moodle, πριν αυτά μεταφερθούν στα επόμενα στάδια ανάλυσης.



Εικόνα 11: Αρχιτεκτονική PostgreSQL. Σχέση server, databases, tables και users

2.4.1 Χαρακτηριστικά PostgreSQL

Η PostgreSQL είναι ένα σύστημα διαχείρισης βάσεων δεδομένων που υποστηρίζει την πλήρη λειτουργικότητα της γλώσσας SQL, ενώ προσφέρει επιπλέον δυνατότητες, όπως:

- Υποστήριξη πολύπλοκων τύπων δεδομένων, π.χ. JSON, XML, Arrays

- Η ευελιξία αυτή την καθιστά ιδανική για την ανάλυση δεδομένων που προέρχονται από πλατφόρμες e-learning, όπως το Moodle, καθώς επιτρέπει τη διαχείριση πολύ μεγάλων όγκων logs, την εκτέλεση σύνθετων ερωτημάτων και τη γρήγορη εξαγωγή στατιστικών στοιχείων.

Εικόνα 12: Ενδεικτικό περιβάλλον διαχείρισης δεδομένων Moodle μέσα από PostgreSQL (μέσω pgAdmin 4)

Η χρήση της PostgreSQL στην παρούσα εργασία παρείχε πολλαπλά οφέλη. Αρχικά, εξασφάλισε αξιόπιστη και ακριβή αποθήκευση του μεγάλου όγκου δεδομένων των Moodle Logs, διατηρώντας τη συνοχή και την ακεραιότητα των εγγραφών. Παράλληλα, επέτρεψε την αποδοτική εκτέλεση σύνθετων ερωτημάτων ακόμη και σε χιλιάδες (ή περισσότερες) εγγραφές, κάτι που είναι κρίσιμο για αναλύσεις που περιλαμβάνουν ομαδοποιήσεις, φίλτρα και χρονικές συσχετίσεις. Επιπλέον, η PostgreSQL ενσωματώνεται εύκολα με εργαλεία ανάλυσης και επεξεργασίας δεδομένων, όπως η Python μέσω βιβλιοθηκών (π.χ. pandas και SQLAlchemy), αλλά και με εργαλεία επιχειρησιακής ευφυΐας όπως το Power BI, διευκολύνοντας την περαιτέρω αξιοποίηση των αποτελεσμάτων. Τέλος, η δυνατότητα δημιουργίας και προσαρμογής όψεων (views) επέτρεψε την εύελκτη παραγωγή διαφορετικών τύπων αναφορών, ανάλογα με τις ανάγκες της ανάλυσης.

26

2.4.3 PostgREST API

Το PostgREST είναι εξυπηρετητής (server) που «μεταφράζει» το σχήμα μιας βάσης PostgreSQL σε REST API. Με αυτόν τον τρόπο, πίνακες, όψεις (views) και—προαιρετικά—συναρτήσεις της βάσης εκτίθενται ως endpoints και προσπελάζονται με απλά αιτήματα HTTP (π.χ. GET, POST) και παραμέτρους στο URL. Η προσέγγιση αξιοποιεί την υφιστάμενη ασφάλεια της PostgreSQL (ρόλοι, Row-Level Security / RLS), διατηρώντας τα δικαιώματα πρόσβασης «στην πηγή».

Στο πλαίσιο της εργασίας, το PostgREST χρησιμοποιείται για την άμεση και ελεγχόμενη έκθεση του πίνακα moodle_data προς ροές LLM (π.χ. LangChain για οργήστρωση, LM Studio για τοπική εκτέλεση μοντέλων).

Το PostgREST διαβάζει το *schema* (σχήμα) της βάσης, δηλαδή τη δομή των πινάκων και των πεδίων, και δημιουργεί αυτόματα REST endpoints. Κάθε πόρος (resource) της βάσης αντιστοιχίζεται σε μια «διεύθυνση» του API.

Βασικοί ορισμοί:

- *Endpoint*: η «διεύθυνση» προσπέλασης ενός πόρου. Π.χ. ο πίνακας moodle_data αντιστοιχίζεται στο /moodle_data
- *REST API*: τυποποιημένη διεπαφή πρόσβασης σε δεδομένα μέσω HTTP μεθόδων και query parameters
- *View (όψη)*: αποθηκευμένο ερώτημα που συμπεριφέρεται σαν "εικονικός πίνακας" και εκτίθεται επίσης ως endpoint
- *RPC (Remote Procedure Call)*: κλήση συνάρτησης της PostgreSQL ως endpoint (π.χ. /rpc/<function>) για σύνθετα/συγκεντρωτικά αποτελέσματα
- *RLS (Row-Level Security)*: πολιτικές που ορίζουν ποιες γραμμές δεδομένων μπορεί να δει ο κάθε ρόλος/χρήστης. Το API κληρονομεί αυτούς τους κανόνες από τη βάση

2.5 LLMs

Τα **Large Language Models (LLMs)** αποτελούν μία από τις πιο σημαντικές τεχνολογικές εξελίξεις στον τομέα της Τεχνητής Νοημοσύνης (AI). Πρόκειται για αλγορίθμους βαθιάς μάθησης (deep learning) που εκπαιδεύονται σε τεράστιους όγκους κειμενικών δεδομένων, με σκοπό να κατανοούν και να παράγουν φυσική γλώσσα με τρόπο που προσεγγίζει την ανθρώπινη επικοινωνία.

2.5.1 Πως λειτουργούν τα LLMs

Τα Large Language Models (LLMs) αποτελούν μια κατηγορία γλωσσικών μοντέλων που έχουν εκπαιδευτεί σε δεδομένα πολύ μεγάλης κλίμακα, με στόχο να μάθουν στατιστικές και σημασιολογικές συσχετίσεις της φυσικής γλώσσας. Μέσα από αυτή τη διαδικασία, αποκτούν την ικανότητα να παράγουν συνεκτικό κείμενο και να προσαρμόζονται σε διαφορετικού τύπου γλωσσικές απαιτήσεις (π.χ. απάντηση σε ερωτήσεις, περίληψη, αναδιατύπωση), ανάλογα με τις οδηγίες που λαμβάνουν. [25]

Η πλειονότητα των σύγχρονων LLMs βασίζεται στην αρχιτεκτονική **Transformer** [3]. Κεντρικό στοιχείο της αρχιτεκτονικής είναι ο μηχανισμός **self-attention**, ο οποίος επιτρέπει στο μοντέλο να λαμβάνει υπόψη του ευρύτερο πλαίσιο συμφραζομένων. Για κάθε σημείο του κειμένου, το μοντέλο «σταθμίζει» ποια άλλα σημεία είναι πιο σημαντικά για την κατανόηση και την παραγωγή της συνέχειας. Αυτό είναι ιδιαίτερα χρήσιμο, επειδή η σημασία μιας λέξης ή φράσης συχνά εξαρτάται από πληροφορία που βρίσκεται αρκετά πριν ή μετά στο ίδιο κείμενο, και όχι μόνο από το άμεσο γειτονικό περιβάλλον. [3]

Στους Transformers, η παραπάνω λειτουργία υλοποιείται σε διαδοχικά **attention layers**, τα οποία επεξεργάζονται επαναληπτικά τις ενδιάμεσες αναπαραστάσεις του κειμένου. Κάθε layer συμβάλλει στο να ενσωματώνεται σταδιακά περισσότερη πληροφορία συμφραζομένων, επιτρέποντας στο μοντέλο να «συνθέτει» πιο σύνθετες σχέσεις μέσα στο κείμενο όσο ανεβαίνει από τα αρχικά προς τα βαθύτερα επίπεδα [3].

Σε επίπεδο αναπαράστασης, το κείμενο εισόδου τεμαχίζεται σε μικρότερες μονάδες (tokens) και στη συνέχεια μετατρέπεται σε αριθμητικές αναπαραστάσεις (embeddings), ώστε να μπορεί να επεξεργαστεί από το νευρωνικό δίκτυο. Παράλληλα, ενσωματώνεται πληροφορία για τη σειρά των tokens, ώστε το μοντέλο να διατηρεί τη διάταξη του κειμένου και να μην αντιμετωπίζει την είσοδο ως σύνολο χωρίς δομή. Έτσι, ο Transformer συνδυάζει το περιεχόμενο των tokens και τη θέση τους, παράγοντας εσωτερικές αναπαραστάσεις που κωδικοποιούν τόσο τη σημασία όσο και το συμφραζόμενο [3].

Κατά την εκπαίδευση, τα LLMs μαθαίνουν να προβλέπουν τη συνέχεια του κειμένου, γεγονός που τα ωθεί να αποτυπώσουν πρότυπα της γλώσσας, συσχετίσεις εννοιών και κοινές δομές λόγου. Στη φάση χρήσης τους, η απόκρισή τους καθορίζεται σε μεγάλο βαθμό από το **prompt**, δηλαδή το κείμενο-είσοδο που περιλαμβάνει το ζητούμενο και τυχόν πρόσθετες οδηγίες ή παραδείγματα. Με αυτόν τον τρόπο, τα μοντέλα μπορούν να προσαρμόζουν τη συμπεριφορά τους στο εκάστοτε πρόβλημα μέσω της πληροφορίας που τους δίνεται «εντός συμφραζομένων» (in-context), χωρίς να απαιτείται επανεκπαίδευση των παραμέτρων τους για κάθε νέο ζητούμενο [25].

Η παραπάνω λειτουργία εξηγεί γιατί τα LLMs μπορούν να χρησιμοποιηθούν ως «ενδιάμεσος μηχανισμός» ανάμεσα στη φυσική γλώσσα και σε πιο δομημένες μορφές πληροφορίας: όταν το prompt παρέχει σαφές πλαίσιο, περιορισμούς και κατάλληλη μορφή στόχου, το μοντέλο είναι πιο πιθανό να παράγει απαντήσεις συνεπείς και αξιοποιήσιμες για επόμενα στάδια επεξεργασίας. [25]



Εικόνα 13: Εννοιολογική απεικόνιση λειτουργίας ενός Large Language Model (είσοδος → επεξεργασία → έξοδος)

2.5.2 LM Studio και LLMs που χρησιμοποιήθηκαν

Για τις ανάγκες της παρούσας εργασίας, χρησιμοποιήθηκε το **LM Studio**, ένα γραφικό περιβάλλον (GUI) που επιτρέπει την **τοπική** εκτέλεση (offline) μεγάλων γλωσσικών μοντέλων. Το LM Studio παρέχει εύχρηστο περιβάλλον για φόρτωση, παραμετροποίηση και αξιολόγηση LLMs χωρίς σύνδεση σε διαδικτυακές υπηρεσίες, επιτρέποντας πλήρη έλεγχο των δεδομένων και του υπολογιστικού φόρτου. [12]

Στο πλαίσιο της μελέτης αξιοποιήθηκαν τα εξής μοντέλα:

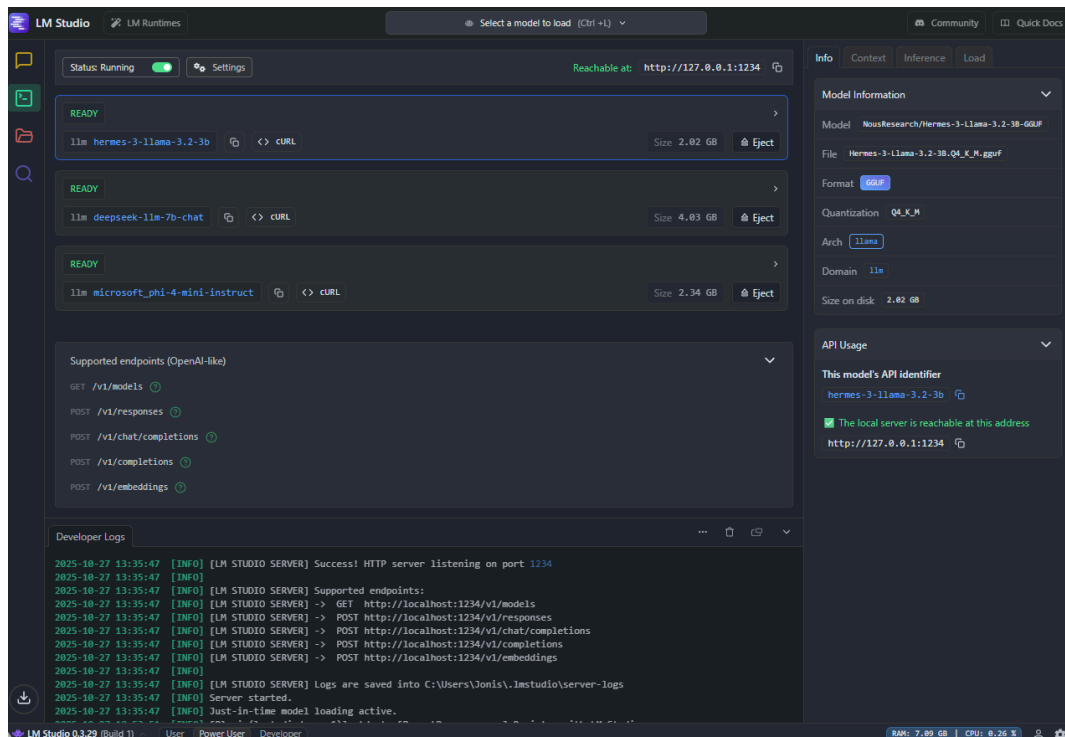
Μοντέλο	Παράμετροι	Περιγραφή / Ιδιαιτερότητα
Hermes LLama 3.2	3 δισ.	Ελαφρύ μοντέλο βασισμένο στην αρχιτεκτονική LLaMA 3, βελτιστοποιημένο για κατανόηση φυσικής γλώσσας και γρήγορη παραγωγή κειμένου
DeepSeek Chat	7 δισ.	Μεσαίου μεγέθους LLM με

αυξημένες δυνατότητες κατανόησης συμφραζομένων και δημιουργίας εκτενών, συνεκτικών απαντήσεων

Phi 4 mini instruct

3 δισ.

Μοντέλο σχεδιασμένο για λογική εξαγωγή συμπερασμάτων και "reasoning" σε σύνθετα ερωτήματα



Εικόνα 14: Περιβάλλον LM Studio – φόρτωση και εκτέλεση LLM σε τοπικό σύστημα

Τι είναι οι «παράμετροι» ενός LLM

Οι παράμετροι (parameters) αποτελούν τον πυρήνα ενός νευρωνικού δικτύου. Είναι οι αριθμητικές τιμές (βάρη και μεροληψίες) που καθορίζουν τον τρόπο με τον οποίο το μοντέλο "μαθαίνει" να επεξεργάζεται τη γλώσσα. Κάθε παράμετρος ρυθμίζεται κατά τη διαδικασία εκπαίδευσης, ώστε το μοντέλο να μπορεί να προβλέψει με ακρίβεια την επόμενη λέξη, να κατανοήσει συμφραζόμενα ή να απαντήσει σε ερωτήσεις.

Συνοπτικά, οι παράμετροι είναι η "μνήμη" του μοντέλου, αποθηκεύουν τη γνώση που αποκτά από τα δεδομένα εκπαίδευσης και καθορίζουν πόσο **ικανό** είναι το μοντέλο να κατανοεί και να παράγει κείμενο.

Ένα μοντέλο με περισσότερες παραμέτρους έχει τη δυνατότητα να κατανοεί πιο πολύπλοκα μοτίβα και λεπτομέρειες της γλώσσας, αλλά απαιτεί περισσότερους υπολογιστικούς πόρους (CPU/GPU, RAM) για να λειτουργήσει.

Μικρά μοντέλα (1–10 δισ. παράμετροι) είναι πιο γρήγορα και αποδοτικά, αλλά λιγότερο ακριβή σε πολύπλοκα ερωτήματα. Μεσαία ή μεγάλα μοντέλα (10–500 δισ. παράμετροι) έχουν ανώτερη κατανόηση και παράγουν πιο φυσικές και συνεκτικές απαντήσεις, εις βάρος της υπολογιστικής απαίτησης. Στην παρούσα εργασία επιλέχθηκαν μοντέλα με 3–7 δισεκατομμύρια παραμέτρους, ως συμβιβασμός μεταξύ απόδοσης και

αποδοτικότητας, επιτρέποντας την τοπική τους εκτέλεση μέσω LM Studio χωρίς χρήση εξωτερικού υπολογιστικού νέφους (cloud). [24], [25]

Η χρήση των LLMs μέσω LM Studio συνέβαλε στην αυτόματη νοηματική επεξεργασία των Moodle Logs, επιτρέποντας την εξαγωγή περιγραφικών και λογικών προτύπων από δεδομένα που προηγουμένως απαιτούσαν χειροκίνητη ανάλυση.

2.5.3 Prompt Engineering

Το prompt engineering είναι η διαδικασία σχεδιασμού και διαμόρφωσης της προτροπής (prompt) που παρέχεται σε ένα Large Language Model (LLM), με σκοπό να επηρεαστεί συστηματικά η συμπεριφορά του μοντέλου και να αυξηθεί η πιθανότητα να παραχθεί η επιθυμητή έξοδος. Δεδομένου ότι στην παρούσα εργασία τα LLMs χρησιμοποιούνται για παραγωγή εκτελέσιμων SQL ερωτημάτων και έπειτα για παραγωγή σύντομης απάντησης με βάση τα αποτελέσματα, ο σχεδιασμός της προτροπής αποτελεί κρίσιμο παράγοντα αξιοπιστίας της αυτοματοποιημένης διαδικασίας. [7], [8], [11]

Δεν πρόκειται απλώς για "καλύτερη διατύπωση" μιας ερώτησης, αλλά για ένα σύνολο τεχνικών που ορίζουν ρητά: (α) το πλαίσιο του προβλήματος, (β) τις επιτρεπτές ενέργειες/περιορισμούς, (γ) τη μορφή της απάντησης και (δ) τον τρόπο με τον οποίο το μοντέλο θα χειριστεί αβεβαιότητες. Η σημασία του prompt engineering είναι ιδιαίτερα έντονη σε εφαρμογές όπου το LLM καλείται να παράγει δομημένα αποτελέσματα (π.χ. SQL), επειδή μικρές ασάφειες μπορούν να οδηγήσουν σε μη εκτελέσιμα ερωτήματα, λανθασμένη χρήση πεδίων ή ανεπιθύμητες γενικεύσεις.

Στα συστήματα Text-to-SQL, ο ρόλος του prompt engineering είναι διπλός. Πρώτον, λειτουργεί ως μηχανισμός ευθυγράμμισης (alignment) μεταξύ της φυσικής γλώσσας του χρήστη και του σχήματος (schema) της βάσης, βοηθώντας το μοντέλο να αντιστοιχίσει σωστά έννοιες/όρους στις αντίστοιχες στήλες και να αποφύγει "επινοήσεις" μη υπαρκτών πεδίων. Δεύτερον, λειτουργεί ως μηχανισμός ελέγχου ποιότητας της παραγόμενης SQL, επιβάλλοντας κανόνες που περιορίζουν την πιθανότητα λαθών (π.χ. περιορισμός στο διαθέσιμο schema, απαίτηση για μία και μόνο SQL πρόταση, σταθερή μορφή εξόδου). Η ύπαρξη σαφών οδηγιών και σταθερής δομής στην προτροπή βελτιώνει την αναπαραγωγικότητα των αποτελεσμάτων και τη σταθερότητα της συνολικής διαδικασίας. [11]

Γενικότερα, οι τεχνικές prompt engineering μπορούν να οργανωθούν γύρω από τρεις βασικούς στόχους:

- **Αποσαφήνιση του ζητούμενου (task specification):**
Η προτροπή ορίζει με ακρίβεια τι θεωρείται σωστή απάντηση, ποια δεδομένα επιτρέπεται/πρέπει να χρησιμοποιηθούν και ποια μορφή εξόδου είναι επιθυμητή. Αυτό μειώνει τα μη σχετικά αποτελέσματα και περιορίζει την πιθανότητα το μοντέλο να "συμπληρώνει" πληροφορίες που δεν προκύπτουν από τα δεδομένα. [7], [8]
- **Καθοδήγηση του τρόπου επίλυσης (reasoning guidance):**
Σε ερωτήματα που απαιτούν πολλαπλά βήματα (π.χ. φιλτράρισμα → μετασχηματισμός → ομαδοποίηση → ταξινόμηση), η προτροπή μπορεί να κατευθύνει το μοντέλο σε βηματικό συλλογισμό ("step-by-step"), αυξάνοντας την πιθανότητα ορθής αλληλουχίας ενεργειών. Η πρακτική αυτή έχει μελετηθεί ως τρόπος ενίσχυσης της συνοχής του συλλογισμού σε συγκεκριμένα είδη προβλημάτων. [26]
- **Μείωση σφαλμάτων και αύξηση συνέπειας (robustness & consistency):**
Η προτροπή μπορεί να λειτουργήσει ως "φραγμός" σε συνηθισμένα λάθη μέσω κανόνων/περιορισμών και να επιβάλει σταθερή δομή εξόδου ώστε να είναι καλύτερος ο έλεγχος και η εκτέλεση. Σε

περιβάλλοντα όπου η έξοδος είναι εκτελέσιμη (όπως η SQL), αυτό μειώνει αποτυχίες εκτέλεσης και βελτιώνει τη συνολική αξιοπιστία του συστήματος. [7], [8], [11]

Ενδεικτικά, μερικές τεχνικές prompt engineering, οι οποίες αποτελούν αυτές που χρησιμοποιήθηκαν στην παρούσα εργασία, είναι οι εξής:

- **Instructional prompting (εντολές/περιορισμοί):**

Η προτροπή διατυπώνεται ως σαφής οδηγία που προσδιορίζει το ζητούμενο και συχνά περιλαμβάνει περιορισμούς (π.χ. ποια πεδία να χρησιμοποιηθούν, ποιο format εξόδου είναι αποδεκτό, τι να αποφευχθεί). Στόχος είναι να μειωθεί η ασάφεια και να αυξηθεί η συνέπεια της εξόδου, ιδιαίτερα όταν απαιτείται δομημένο αποτέλεσμα. [7], [8]

- **Schema grounding (ενσωμάτωση σχήματος/πληροφορίας πλαισίου):**

Σε tasks που αλληλεπιδρούν με βάσεις δεδομένων, η προτροπή μπορεί να περιλαμβάνει ρητά το schema (πίνακες, στήλες, σχέσεις) ή άλλο σχετικό "πλαίσιο", ώστε το μοντέλο να παράγει απαντήσεις που είναι συμβατές με τους διαθέσιμους πόρους και να μειώνεται η πιθανότητα "hallucinations" (αναφορών σε μη υπάρχουσες οντότητες). Η τεχνική αυτή είναι ιδιαίτερα σημαντική σε Text-to-SQL, καθώς συνδέεται άμεσα με την εκτελεσιμότητα του παραγόμενου ερωτήματος [11].

- **Step-by-step / Chain-of-Thought prompting:**

Η προτροπή καθοδηγεί το μοντέλο να οργανώσει τη λύση σε διακριτά βήματα, κάτι που μπορεί να βελτιώσει την απόδοση σε προβλήματα που απαιτούν σύνθετη συλλογιστική ή πολλαπλούς μετασχηματισμούς πριν την τελική απάντηση. Η τεχνική μπορεί να εφαρμοστεί είτε με παραδείγματα συλλογιστικής είτε ακόμη και με απλή προτροπή που ζητά από το μοντέλο να "σκεφτεί βήμα-βήμα" [26].

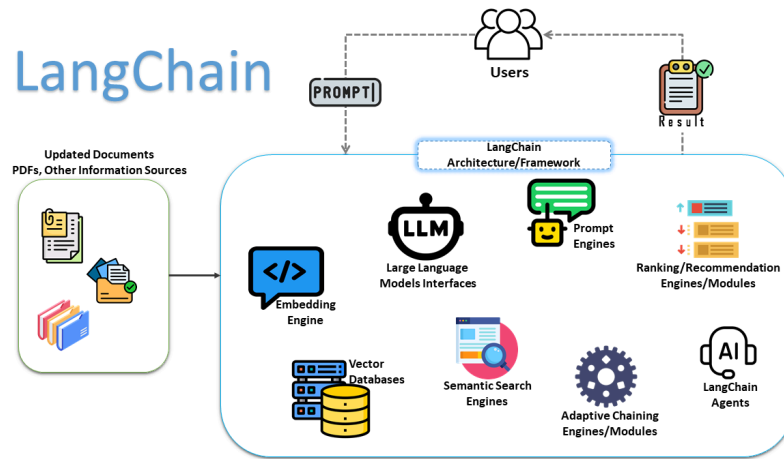
- **Feedback/error-aware prompting (ανατροφοδότηση βάσει σφάλματος):**

Σε επαναληπτικές διαδικασίες, η προτροπή μπορεί να εμπλουτίζεται με πληροφορία ανατροφοδότησης (π.χ. μήνυμα σφάλματος από εκτέλεση SQL ή έλεγχο εγκυρότητας), ώστε το μοντέλο να αναθεωρήσει και να διορθώσει την έξοδό του σε επόμενη προσπάθεια. Αυτή η τεχνική χρησιμοποιείται για να αυξηθεί η ανθεκτικότητα της διαδικασίας όταν η αρχική έξοδος αποτύχει ή παραβιάζει περιορισμούς [7], [8], [11].

2.6 LangChain

Η LangChain αποτελεί ένα σύγχρονο και ευέλικτο πλαίσιο ανάπτυξης εφαρμογών που βασίζονται σε Large Language Models (LLMs). Ο βασικός της στόχος είναι να επιτρέψει στους ερευνητές και προγραμματιστές να συνδυάζουν LLMs με εξωτερικές πηγές δεδομένων, εργαλεία και βάσεις, δημιουργώντας πιο «έξυπνες» και διαδραστικές εφαρμογές τεχνητής νοημοσύνης.

Με τη LangChain, τα LLMs δεν περιορίζονται στο να απαντούν γενικά σε ερωτήσεις, αλλά μπορούν να αναζητούν πληροφορίες, αναλύουν δεδομένα, εκτελούν υπολογισμούς, και συνθέτουν περιεχόμενο με βάση προσαρμοσμένα ή δυναμικά δεδομένα.



Εικόνα 15: Διάγραμμα συνοπτικής απεικόνισης αρχιτεκτονικής και χρήσεων της LangChain

2.6.1 Τι είναι η LangChain, μια συντομη επισκοπή

Η LangChain αναπτύχθηκε το 2022 και έχει εξελιχθεί σε ένα από τα πιο δημοφιλή εργαλεία για τη διασύνδεση LLMs με εφαρμογές πραγματικού κόσμου. Αποτελεί βιβλιοθήκη Python και JavaScript, με στόχο να κάνει τα LLMs "πράκτορες" (agents) που μπορούν να εργάζονται με εργαλεία, βάσεις και APIs.

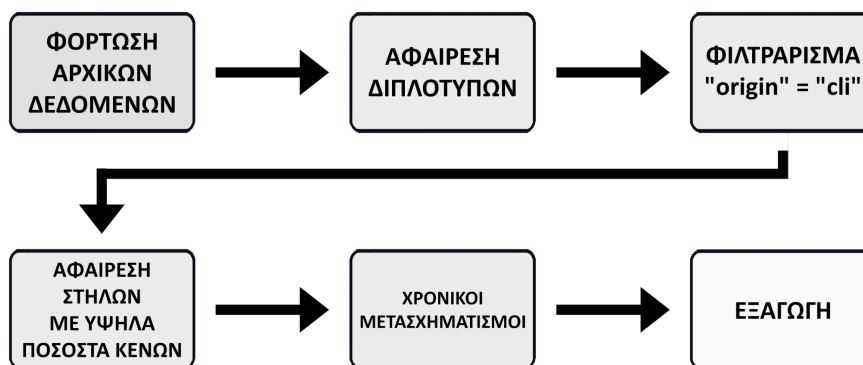
Η αρχιτεκτονική της LangChain βασίζεται σε μερικές κύριες έννοιες:

- **LLMs:** Τα ίδια τα γλωσσικά μοντέλα που εκτελούν τις λειτουργίες κατανόησης και δημιουργίας κειμένου
- **Prompts:** Οι οδηγίες που δίνουμε στο μοντέλο (π.χ. "ανέλυσε αυτά τα δεδομένα και δώσε περίληψη")
- **Chains:** Αλληλουχίες βημάτων, όπου η έξοδος ενός LLM χρησιμοποιείται ως είσοδος για το επόμενο
- **Agents:** Πιο σύνθετες οντότητες που αποφασίζουν ποιες ενέργειες να εκτελέσουν (π.χ. αναζήτηση σε βάση ή ανάγνωση αρχείου) για να απαντήσουν σε ένα ερώτημα
- **Retrieval (Ανάκτηση):** Ο μηχανισμός μέσω του οποίου το μοντέλο αναζητά και ανακτά σχετικές πληροφορίες από εξωτερικές πηγές ή βάσεις δεδομένων, όπως embeddings, vector stores ή έγγραφα
- **Memory:** Μηχανισμός προσωρινής μνήμης που επιτρέπει στα LLMs να θυμούνται το ιστορικό συνομιλιών ή ενεργειών

Χάρη σε αυτά τα στοιχεία, η LangChain προσφέρει μια σπονδυλωτή (modular) προσέγγιση, όπου κάθε εφαρμογή μπορεί να προσαρμόζεται εύκολα στις ανάγκες του εκάστοτε έργου.

Κεφάλαιο 3ο: Επεξεργασία και Καθαρισμός Δεδομένων

Η ενότητα περιγράφει τη ροή προετοιμασίας των Moodle Logs από την αρχική φόρτωση έως το τελικό, καθαρισμένο dataset που αξιοποιείται στα επόμενα κεφάλαια. Η επεξεργασία πραγματοποιήθηκε σε Jupyter Notebook με Dask-pandas, ώστε να υποστηριχθεί ο μεγάλος αρχικός όγκος και να διατηρηθούν αποδοτικοί χρόνοι εκτέλεσης. Στόχος ήταν ένα συνεπές, καθαρό και αναπαραγώγιμο σύνολο δεδομένων, με σαφή χρονική αναπαράσταση και πεδία υψηλής χρησιμότητας για τα ερευνητικά ερωτήματα



Εικόνα 16: Διάγραμμα ροής καθαρισμού

3.1 Αρχική Φόρτωση και Βασικός Χαρακτηρισμός

Η αρχική φόρτωση έγινε από αρχείο CSV (η αρχική βάση δεδομένων που είχαμε στην διάθεσή μας: moodle_logs_ktu_1.csv) στους Dask dataframes, με έλεγχο τύπων και βασικών μεγεθών. Πριν τον καθαρισμό, ο πίνακας περιείχε 58.153.450 εγγραφές.

```
file_path = "moodle_logs_ktu_1.csv"

df_original = dd.read_csv(file_path)

total_rows = df_original.shape[0].compute()
print("Total number of rows:", total_rows)

Total number of rows: 58153450
```

Εικόνα 17: Ενδεικτικό απόσπασμα κώδικα: φόρτωση δεδομένων και καταμέτρηση καταχωρήσεων (rows) σε περιβάλλον Jupyter Notebook

3.2 Κανόνες Καθαρισμού και Μετασχηματισμοί

Αφαίρεση διπλότυπων καταχωρήσεων

Απομακρύνθηκαν εγγραφές απολύτως ταυτόσημες, ώστε κάθε γεγονός να εμφανίζεται μία φορά (μείωση θορύβου και αποφυγή τεχνητών μεγεθών).

```
df_drop = df_original.map_partitions(lambda part: part.drop_duplicates())
```

Ενδεικτικό απόσπασμα αφαίρεσης διπλότυπων καταχωρήσεων

Αποκοπή συστημικών/μη ανθρώπινων ενεργειών

Οι εγγραφές "cli" της στήλης "origin" αφορούν εργασίες bot/διαχειριστή και όχι συμπεριφορά τελικών χρηστών οπότε και αφαιρέθηκαν. Η αφαίρεση αυτή οδήγησε στη μεγαλύτερη μείωση του όγκου.

```
df = df_drop[df_drop["origin"].str.contains("cli") == False]
```

Μετά τις δύο τελευταία ενέργειες, ο συνολικός όγκος μειώθηκε δραστικά. Το τελικό πλήθος εγγραφών μετά τον καθαρισμό είναι 2.477.467.

Αφαίρεση στηλών με υψηλή έλλειψη ή περιορισμένη χρησιμότητα

Στο σημείο αυτό πραγματοποιήθηκε επισκόπηση πληρότητας (κενές τιμές ανά στήλη) και εντοπισμός συστημικών καταγραφών που δεν αντιστοιχούν σε ανθρώπινη δραστηριότητα.

```
missing_val = df_original.isnull().sum().compute()

print(f"Total number of missing values per column:\n {missing_val} \n")

for col in df_original.columns:
    perc = missing_val.loc[col] / total_rows
    if perc > 0:
        print(f"Percentage of missing info of column {col}: {perc:.2%}")
```

Ενδεικτικό απόσπασμα κώδικα για την καταμέτρηση κενών τιμών ανά στήλη και την ποσοστιαία απεικόνιση αυτού

```

Total number of missing values per column:
id                0
eventname         0
component         0
action           0
target           0
objecttable      27859951
objectid         27859951
crud             0
edulevel         0
contextid        0
contextlevel     0
contextinstanceid 0
userid          0
courseid        0
relateduserid    45234212
anonymous        0
other            33402333
timecreated      0
origin           0
ip              2302633
realuserid       58134735
dtype: int64

Percentage of missing info of column objecttable: 47.91%
Percentage of missing info of column objectid: 47.91%
Percentage of missing info of column relateduserid: 77.78%
Percentage of missing info of column other: 57.44%
Percentage of missing info of column ip: 3.96%
Percentage of missing info of column realuserid: 99.97%

```

Εικόνα 18: Ποσοστά κενών τιμών ανά στήλη σε περιβάλλον Jupyter Notebook

Οι στήλες με μεγάλα ποσοστά κενών τιμών αφαιρέθηκαν. Εξαίρεση αποτελεί η στήλη "relateduserid", καθώς στην συγκεκριμένη στήλη αναμένεται να υπάρχουν πολλές καταχωρήσεις λόγω της φύσης των δεδομένων και όχι λόγω ανεπιτυχής συλλογής τους. Επιπροσθέτως αφαιρέθηκε η στήλη "ip" καθώς ενώ τα ποσοστά κενών καταχωρήσεων είναι μικρά, τα δεδομένα που παρέχει δεν προσφέρουν οποιαδήποτε αξία στην παρούσα εργασία.

```
df = df_drop.drop(columns=['ip', 'realuserid', 'objectid', 'other', 'objecttable'])
```

Ενδεικτικό απόσπασμα κώδικα για την αφαίρεση στηλών με ελλιπή/περιττά δεδομένα

Χρονικοί μετασχηματισμοί

Το πεδίο timecreated (Unix epoch) μετατράπηκε σε datetime (UTC) και παράχθηκαν βοηθητικά πεδία:

- datetime (ημερομηνία και ώρα)
- time (ώρα)
- day (ημέρα εβδομάδας)
- month (μήνας).

Η ύπαρξη ποικίλων επιλογών για τη διατήρηση του αρχικού timecreated επιτρέπει ευελιξία σε ερωτήματα που απαιτούν epoch.

3.3 Τελικό Schema και Εξαγωγή

Μετά τον καθαρισμό/μετασχηματισμούς, το τελικό dataset περιλαμβάνει τις στήλες:

- id
- eventname
- component
- action
- target
- crud
- edulevel
- contextid
- contextlevel
- contextinstanceid
- userid
- courseid
- relateduserid
- anonymous
- timecreated
- origin
- datetime
- time
- day
- month

Η εξαγωγή υλοποιήθηκε σε μορφή CSV (πολλαπλά αρχεία—partitions για out-of-core ή ενιαίο αρχείο όταν το επέτρεπαν οι πόροι).

	id	eventname	component	action	target	crud	edulevel	contextid	contextlevel	contextinstanceid	userid	courseid	relateduserid	anonymous	timecreated	origin	datetime	time	day	month
0	70142113	\mod_assign\event\course_module_viewed	mod_assign	viewed	course_module	r	2	260612	70	159211	9213	2844	NaN	0	1580352285	web	2020-01-30 02:44:45	02:44:45	Thursday	January
1	70146780	\core\event\course_viewed	core	viewed	course	r	2	2	50	1	0	1	NaN	0	1580354156	web	2020-01-30 03:15:56	03:15:56	Thursday	January
2	70149239	\core\event\course_viewed	core	viewed	course	r	2	2	50	1	0	1	NaN	0	1580355117	web	2020-01-30 03:31:57	03:31:57	Thursday	January
3	70151208	\assignsubmission_file\event\submission_created	assignsubmission_file	created	submission	c	2	187863	70	113362	14345	2440	14345.0	0	1580355935	web	2020-01-30 03:45:35	03:45:35	Thursday	January
4	70152257	\core\event\course_viewed	core	viewed	course	r	2	2	50	1	0	1	NaN	0	1580356381	web	2020-01-30 03:53:01	03:53:01	Thursday	January
5	70155501	\core\event\course_viewed	core	viewed	course	r	2	2	50	1	0	1	NaN	0	1580357637	web	2020-01-30 04:13:57	04:13:57	Thursday	January

Εικόνα 19: Τελική μορφή δεδομένων σε πίνακα σε περιβάλλον Jupyter Notebook

3.4 Ενδεικτικοί Έλεγχοι Εγκυρότητας

Μετά την εξαγωγή, πραγματοποιήθηκαν συνοπτικοί έλεγχοι για να διαπιστωθεί ότι τα δεδομένα είναι συνεπή και έτοιμα για ανάλυση.

Παραδείγματα:

- Κατανομή ανά ημέρα:

```
df.groupby("day").size().compute().sort_values(ascending=False)
```

- Συχνότερα action:

```
df.groupby("action").size().compute().sort_values(ascending=False).head(10)
```

- Χρονικό εύρος:

```
(df["datetime"].min().compute(), df["datetime"].max().compute())
```

3.5 Συμπεράσματα

Μετά τον καθαρισμό, ο όγκος του dataset μειώθηκε από 58.153.450 σε 2.477.467 εγγραφές, κυρίως λόγω της αφαίρεσης των συστημικών καταγραφών με `origin="cli"` και της αποδιπλοποίησης. Απομακρύνθηκαν τα πεδία `ip` (3,96%), `realuserid` (99,97%), `objectid` (47,91%), `objecttable` (47,91%) και `other` (57,44%), είτε λόγω υψηλής έλλειψης είτε λόγω περιορισμένης συνεισφοράς στα ερευνητικά ερωτήματα. Ενισχύθηκε η χρονική αναπαράσταση με τη δημιουργία των πεδίων `datetime`, `time`, `day` και `month`, διατηρώντας παράλληλα το `timecreated` για ευελιξία σε ερωτήματα που απαιτούν Unix epoch. Τέλος, πραγματοποιήθηκαν έλεγχοι συνέπειας στο `crud` και κανονικοποιήσεις στο `action`, με αποτέλεσμα ένα συμπαγές και ομοιογενές σύνολο δεδομένων, κατάλληλο για περιγραφική/διερευνητική ανάλυση και για τις επόμενες ροές LLM (π.χ. παραγωγή SQL με καθοδηγούμενα prompts).

Κεφάλαιο 4ο: Υλοποίηση Εφαρμογής

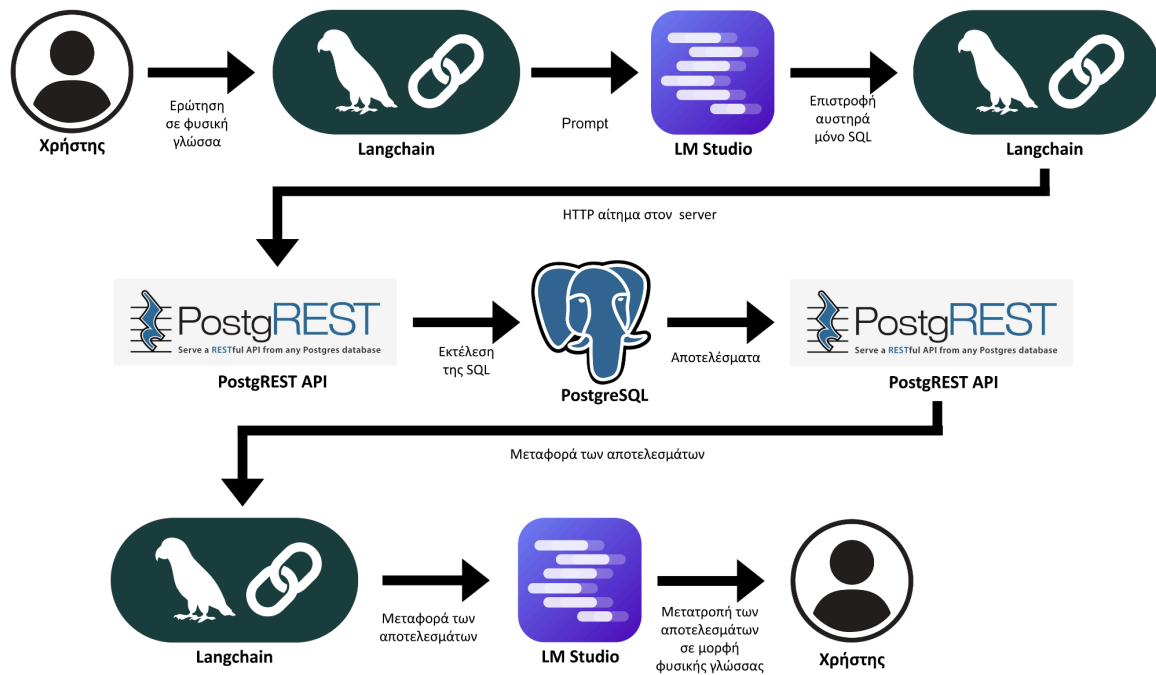
Το κεφάλαιο παρουσιάζει τη μεθοδολογία προετοιμασίας, εκτέλεσης και αξιολόγησης του συστήματος Φυσική γλώσσα → SQL → Εκτέλεση → Απάντηση πάνω στο καθαρισμένο σύνολο Moodle Logs, καθώς και τα αποτελέσματα ανά Large Language Model (LLM). Η αξιολόγηση πραγματοποιήθηκε με τρία διαφορετικά LLM εκτελούμενα μέσω LM Studio (OpenAI-compatible API), με δύο *θερμοκρασίες* ανά μοντέλο (0.1 και 0.5) και τέσσερα διακριτά prompts. Για κάθε συνδυασμό υπολογίστηκαν: (α) χρόνος ολοκλήρωσης ανά prompt και (β) ποσοστό επιτυχίας (παραγωγή συντακτικά εκτελέσιμου και σημασιολογικά ορθού SQL που επιστρέφει αποτέλεσμα χωρίς σφάλμα).

Η **Θερμοκρασία (temperature)** είναι παράμετρος που ελέγχει το βαθμό στοχαστικότητας στην παραγωγή κειμένου από ένα LLM. Το εύρος τιμών που μπορεί να λάβει αυτή η παράμετρος εξαρτάται από την εκάστοτε υλοποίηση/διεπαφή. Σε API περιβάλλοντα συναντάται συχνά ως τιμή μεταξύ 0 και 2, ενώ στην πράξη χρησιμοποιείται συχνά και σε υπο-εύρος (π.χ. 0–1) για πιο "συντηρητική" ρύθμιση [22].

Τεχνικά, η θερμοκρασία επηρεάζει το πόσο "ανοιχτό" είναι το μοντέλο στο να επιλέγει λιγότερο πιθανές επόμενες λέξεις/τοκεν. Χαμηλές τιμές (π.χ. 0.0–0.2) «αιχμηροποιούν» την επιλογή προς τις πιο πιθανές συνέχειες και οδηγούν σε πιο ντετερμινιστικές, σταθερές απαντήσεις, ενώ υψηλότερες τιμές (π.χ. 0.5–0.9) αυξάνουν την ποικιλία/δημιουργικότητα αλλά και τον κίνδυνο ασυνέπειας ή σφαλμάτων. Με άλλα λόγια, όσο πιο υψηλά θέτουμε την τιμή της θερμοκρασίας, τόσο περισσότερο το μοντέλο τείνει να "εξερευνά" εναλλακτικές διατυπώσεις, ενώ όσο πιο χαμηλά την θέτουμε τόσο πιο "συντηρητικά" παραμένει [22], [27].

Η επίδραση αυτή είναι ιδιαίτερα σημαντική όταν το ζητούμενο είναι δομημένη έξοδος με αυστηρούς κανόνες, όπως η SQL. Στην περίπτωση των παρουσών μετρήσεων, χαμηλή θερμοκρασία τείνει να βελτιώνει τη συντακτική και σημασιολογική ορθότητα του παραγόμενου SQL, επειδή μειώνει τις απρόβλεπτες αποκλίσεις (π.χ. ασυνήθιστες επιλογές σύνταξης ή "δημιουργικές" αλλά λανθασμένες υποθέσεις). Αντίθετα, υψηλότερη θερμοκρασία μπορεί να βοηθήσει σε πιο «ευρηματικές» λύσεις ή εναλλακτικές διατυπώσεις, με ενδεχόμενη όμως μείωση της αξιοπιστίας/σταθερότητας εκτέλεσης [22], [27], [28].

Τέλος, για να είναι τα πειράματα συγκρίσιμα, είναι σημαντικό να διατηρούνται σταθερές οι ρυθμίσεις δειγματοληψίας ανά εκτέλεση. Ειδικά όταν χρησιμοποιούνται και άλλες παράμετροι δειγματοληψίας (όπως το `top_p`), συνήθως προτείνεται να μεταβάλλεται είτε η θερμοκρασία είτε το `top_p`, ώστε να είναι πιο καθαρή/ερμηνεύσιμη η επίδραση κάθε ρύθμισης [22], [27].



Εικόνα 20: Διάγραμμα ροής της λειτουργίας του προγράμματος

4.1 Περιβάλλον Υλοποίησης

Περιβάλλον εκτέλεσης: Τα δεδομένα έχουν φορτωθεί σε PostgreSQL. Η πρόσβαση γίνεται είτε απευθείας (SQLAlchemy) είτε μέσω PostgREST για HTTP endpoints. Τα LLMs τρέχουν τοπικά με LM Studio, ώστε να διασφαλίζεται σταθερός χρόνος απόκρισης και επαναληψιμότητα των μετρήσεων.

Ροή αξιολόγησης: Για κάθε LLM και για κάθε θερμοκρασία (0.1, 0.5) εκτελέστηκαν κυκλικά τα τέσσερα prompts. Για κάθε εκτέλεση καταγράφηκε ο συνολικός χρόνος από την δημιουργία SQL έως την επιστροφή απάντησης και επισημάνθηκε η επιτυχία ή αποτυχία (εκτελέσιμο SQL χωρίς σφάλμα, με ορθή αναφορά στη στήλη/πίνακα). Η διαδικασία επαναλήφθηκε, ώστε να ληφθεί αντιπροσωπευτικό δείγμα για τους μέσους χρόνους και τα ποσοστά επιτυχίας.

Μετρικές:

- **Χρόνος ολοκλήρωσης (sec):** Μετράει τον συνολικό χρόνο ανά prompt (δημιουργία SQL → εκτέλεση → επιστροφή απάντησης)
- **Ποσοστό επιτυχίας (%):** Λόγος επιτυχημένων εκτελέσεων προς το σύνολο εκτελέσεων ανά prompt/θερμοκρασία/LLM

Κατά την παρούσα εργασία χρησιμοποιήθηκαν σχετικά μικρού μεγέθους παραμέτρων μοντέλα, καθώς η υπολογιστική υποδομή που ήταν διαθέσιμη ήταν περιορισμένη. Συγκεκριμένα, για την εκτέλεση των μετρήσεων χρησιμοποιήθηκε σταθερός υπολογιστής με τις παρακάτω προδιαγραφές:

- CPU: AMD Ryzen 5 2600 στα 3.4 GHz με 6 πυρήνες
- GPU: Gigabyte GeForce RTX 3060 Ti με 8 GB VRAM
- RAM: 16 GB στα 3200 MHz

Λόγω της φύσης των μετρήσεων, ένα υπολογιστικό σύστημα υψηλότερων προδιαγραφών θα μπορούσε να υποστηρίξει μοντέλα μεγαλύτερου αριθμού παραμέτρων, τα οποία με την σειρά τους θα επέστρεφαν ποιοτικότερα αποτελέσματα. Για αυτόν τον λόγο υπήρξε μια σχετική «επιείκεια» στον τρόπο όπου καταγράφηκαν οι επιτυχημένες μετρήσεις. Συγκεκριμένα:

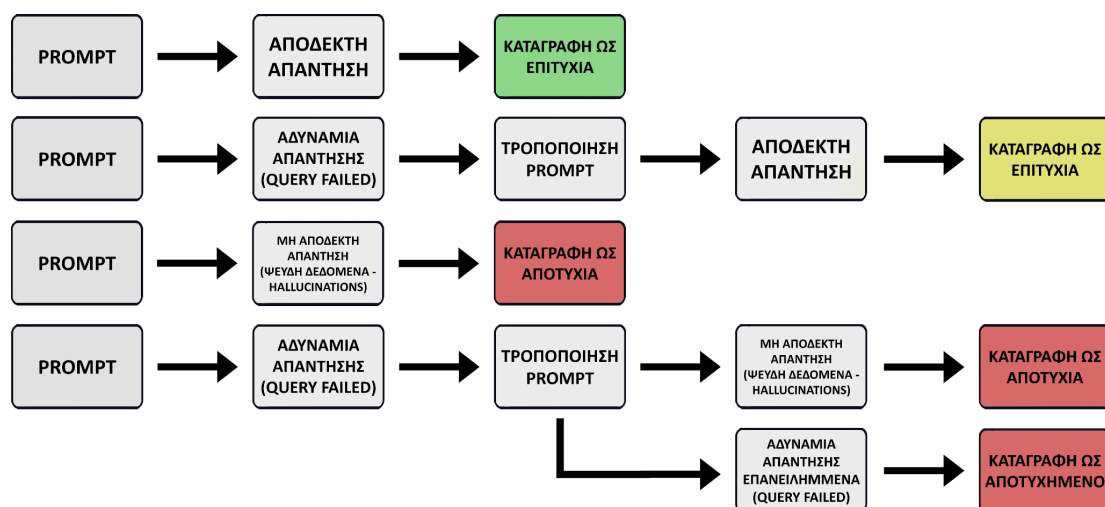
- Ως **επιτυχημένες** καταγράφηκαν οι μετρήσεις που παράχθηκε απόλυτα ορθολογικό και εκτελέσιμο SQL και επιστράφηκε η αντίστοιχη σωστή απάντηση στο prompt σε φυσική γλώσσα
- Ως **επιτυχημένες** καταγράφηκαν επίσης οι μετρήσεις όπου είχαμε αντίστοιχα αποδεκτά αποτελέσματα ακόμα κι αν χρειάστηκαν μικρές **διαφοροποιήσεις** στην διατύπωση του prompt (οι οποίες δεν το αλλάζουν εννοιολογικά). Αυτό έγινε καθώς τα LLMs μικρού αριθμού παραμέτρων δεν έχουν την ίδια ευελιξία στην διατύπωση των prompts και διαχειρίζονται πιο δύσκολα σύνθετα προβλήματα. Αυτό συνεπάγεται ότι το ίδιο LLM με περισσότερες παραμέτρους θα μπορούσε να ανταποκριθεί στο αρχικό prompt που του δόθηκε χωρίς διαφοροποιήσεις και προσαρμογές στην διατύπωσή του, οπότε θα ήταν «άδικο» προς το εκάστοτε LLM να του προσάψουμε αποτυχημένες μετρήσεις λόγω έλλειψης υπολογιστικής υποδομής. Φυσικά στις μετρήσεις όπου χρειάστηκε τέτοια τροποποίηση στην διατύπωση σημειώθηκε αντίστοιχα.

Παράδειγμα τροποποίησης:

Αρχικό prompt: For month='January', which actions have the most total events? (Use 'month' and 'action'; return action and event_count.). Return top 5 by that count.

Τροποποιημένο prompt: For month='January', which actions have the most total events? (Use 'month' and 'action'; return action and event_count.). Return the top 5.

- Ως **αποτυχημένες** καταγράφηκαν οι μετρήσεις όπου μετά από πολλαπλές τέτοιες τροποποιήσεις δεν ήταν εφικτό να επιστραφεί οποιουδήποτε είδους απάντηση. Όταν δηλαδή επιστρέφεται "SQL Query failed". Αυτή η περίπτωση βέβαια ήταν εξαιρετικά σπάνια.
- Ως επίσης **αποτυχημένες** καταγράφηκαν οι μετρήσεις στις οποίες είτε παράχθηκε SQL εκτελέσιμο το οποίο δεν επέστρεφε τις επιθυμητές απαντήσεις, είτε το LLM έπραττε αυθαίρετα και δημιουργούσε δικές του «ανύπαρκτες» απαντήσεις δίχως να παράγει SQL (φαινόμενο LLM hallucination). Σε κάθε περίπτωση, όταν το LLM επέστρεφε τελική απάντηση με «αυτοπεποίθηση» δίχως αυτή να είναι σωστή και ορθολογική καταγραφόταν ως αποτυχία. Το φαινόμενο αυτό παρατηρήθηκε πιο έντονα στις μετρήσεις του Deepseek (ανεξαρτήτως θερμοκρασίας).



Εικόνα 21: Διάγραμμα ροής της καταγραφής των μετρήσεων. Με πράσινο καταγράφονται οι επιτυχημένες μετρήσεις που δεν χρειάστηκαν τροποποίηση στο prompt, με κίτρινο αυτές που χρειάστηκαν τροποποίηση για να καταλήξουν σε αποδεκτή απάντηση και με κόκκινο όλες οι αποτυχημένες μετρήσεις

4.2 Υλοποίηση

4.2.1 Επεξεργασία και προετοιμασία δεδομένων στην PostgreSQL

Μετά τον καθαρισμό των αρχικών δεδομένων, ο πίνακας **moodle_data** εισήχθη στη βάση δεδομένων, η οποία δημιουργήθηκε για τους σκοπούς της παρούσας εργασίας.

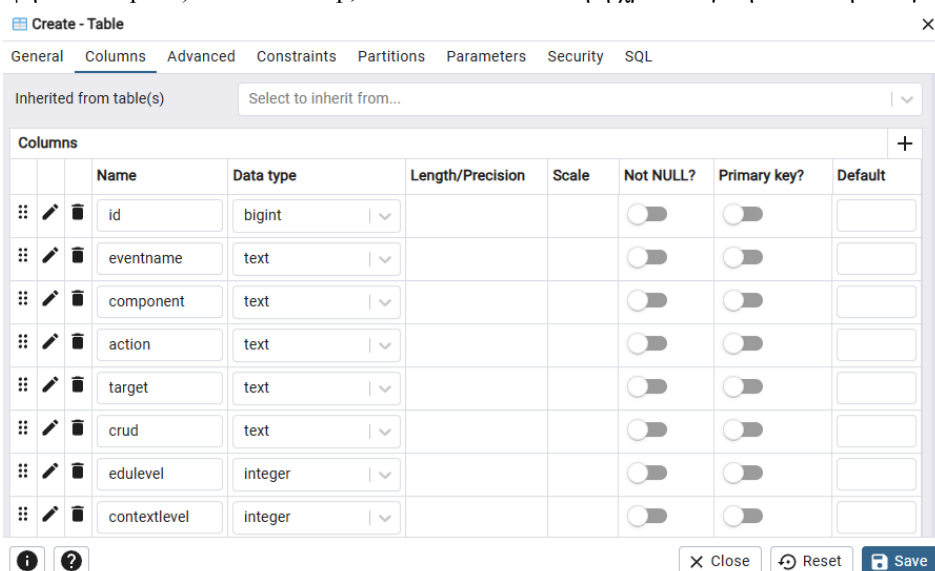
Η εισαγωγή πραγματοποιήθηκε μέσω του **pgAdmin 4**, το γραφικό περιβάλλον διαχείρισης της PostgreSQL, με στόχο τη δημιουργία μιας οργανωμένης και αποδοτικής βάσης δεδομένων που θα υποστηρίζει τις επόμενες φάσεις της ανάλυσης.

Στο στάδιο αυτό εκτελέστηκαν βασικές λειτουργίες προετοιμασίας, όπως ο ορισμός κατάλληλων τύπων δεδομένων για κάθε στήλη, η εισαγωγή του αρχείου CSV, η δημιουργία ευρετηρίων (indexes) για τη βελτίωση της απόδοσης, καθώς και έλεγχοι εγκυρότητας και συνέπειας των εγγραφών.

α) Δημιουργία βάσης και πίνακα

Αρχικά δημιουργήθηκε νέα βάση δεδομένων μέσω του περιβάλλοντος του pgAdmin 4. Στη συνέχεια, δημιουργήθηκε ο πίνακας **moodle_data**, του οποίου η δομή καθορίστηκε με βάση τις στήλες του αρχείου CSV (**cleaned_data.csv**). Οι τύποι δεδομένων επιλέχθηκαν προσεκτικά ώστε να αντανakλούν τη φύση κάθε πεδίου:

- Αριθμητικά αναγνωριστικά (όπως **id**, **userid**, **courseid**, **contextid**) ορίστηκαν ως **bigint**, ώστε να υποστηρίζουν μεγάλο εύρος τιμών
- Τα λεκτικά πεδία (όπως **eventname**, **component**, **action**, **target**, **origin**) δηλώθηκαν ως **text**
- Η στήλη **anonymous** δηλώθηκε ως **boolean**, με μετατροπή των τιμών 0/1 του αρχείου σε **false/true**
- Οι στήλες **timecreated** και **datetime** δηλώθηκαν αντίστοιχα ως **bigint** (αποθηκεύοντας τον χρόνο σε μορφή UNIX epoch) και **timestamp**, ώστε να είναι δυνατή η χρονολογική ανάλυση των γεγονότων



Εικόνα 22: Δημιουργία του πίνακα **moodle_data** και ορισμός τύπων δεδομένων μέσω του περιβάλλοντος **pgAdmin 4**

β) Εισαγωγή δεδομένων

Η εισαγωγή του καθαρισμένου αρχείου CSV πραγματοποιήθηκε μέσω της επιλογής *Import/Export Data* στο pgAdmin 4.

Κατά την εισαγωγή επιλέχθηκαν οι κατάλληλες ρυθμίσεις:

- **Format: CSV**

Επιλέχθηκε επειδή το αρχείο εξαγωγής βρίσκεται σε μορφή CSV, η οποία είναι δομημένη και αναγνώσιμη από το pgAdmin/PostgreSQL χωρίς επιπλέον μετατροπές. Έτσι διατηρείται η απλή αντιστοίχιση "στήλες αρχείου → πεδία πίνακα".

- **Header: Yes**

Το αρχείο περιλαμβάνει στην πρώτη γραμμή τα ονόματα των στηλών. Η ενεργοποίηση του header εξασφαλίζει ότι το σύστημα δεν θα εισάγει την πρώτη γραμμή ως δεδομένα, αλλά θα τη χρησιμοποιήσει ως οδηγό για σωστή ευθυγράμμιση/αντιστοίχιση των στηλών κατά την εισαγωγή.

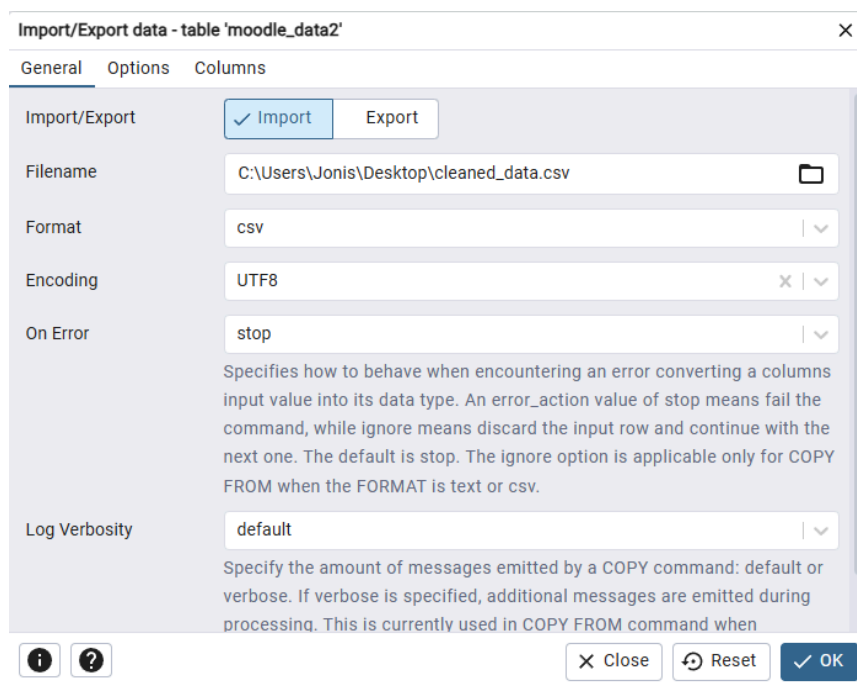
- **Delimiter: ,**

Το κόμμα είναι ο χαρακτήρας διαχωρισμού που χρησιμοποιείται στο συγκεκριμένο CSV. Με τη ρύθμιση αυτή διασφαλίζεται ότι κάθε γραμμή θα "σπάει" στα σωστά πεδία και ότι οι τιμές θα τοποθετούνται στις αντίστοιχες στήλες του πίνακα χωρίς μετατοπίσεις ή σφάλματα εισαγωγής.

- **Encoding: UTF8**

Επιλέχθηκε ώστε να υποστηρίζονται με ασφάλεια ειδικοί χαρακτήρες (π.χ. σύμβολα) και να αποφεύγονται αλλοιώσεις κειμένου ή αποτυχίες εισαγωγής λόγω ασυμβατότητας κωδικοποίησης.

Με αυτόν τον τρόπο, τα δεδομένα φορτώθηκαν απευθείας στον πίνακα `moodle_data`. Η συγκεκριμένη μέθοδος επιλέχθηκε για τη σταθερότητα και την ευχρηστία της, καθώς επιτρέπει την άμεση εποπτεία της διαδικασίας και τη διασφάλιση της ορθής αντιστοίχισης των στηλών.



Εικόνα 23: Εισαγωγή του αρχείου CSV μέσω του περιβάλλοντος pgAdmin 4

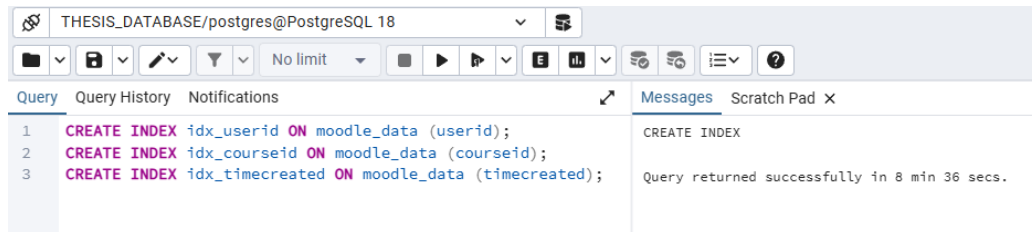
γ) Δημιουργία ευρετηρίων (Indexes)

Για την επιτάχυνση των ερωτημάτων που εκτελούνται συχνότερα, δημιουργήθηκαν ευρετήρια (indexes) σε πεδία που χρησιμοποιούνται για φιλτράρισμα ή ταξινόμηση, όπως *userid*, *courseid* και *timecreated*.

Η δημιουργία των ευρετηρίων πραγματοποιήθηκε με τις ακόλουθες εντολές SQL:

```
CREATE INDEX idx_userid ON moodle_data (userid);
CREATE INDEX idx_courseid ON moodle_data (courseid);
CREATE INDEX idx_timecreated ON moodle_data (timecreated);
```

Η ύπαρξη ευρετηρίων οδήγησε σε αισθητή βελτίωση της ταχύτητας απόκρισης των ερωτημάτων κατά την ανάλυση των δεδομένων, χωρίς να επηρεάζεται αρνητικά η συνολική απόδοση της βάσης.

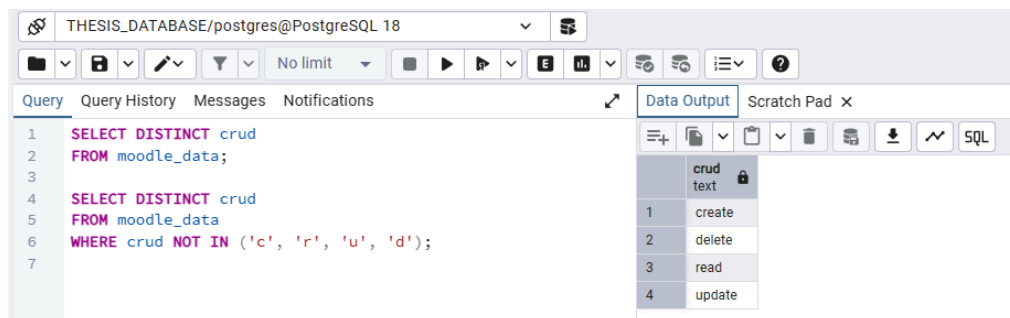


Εικόνα 24: Δημιουργία ευρετηρίων (indexes) στις κύριες στήλες του πίνακα moodle_data

δ) Έλεγχοι και μετασχηματισμοί δεδομένων

Μετά την ολοκλήρωση της εισαγωγής, πραγματοποιήθηκαν βασικοί έλεγχοι και μετασχηματισμοί για τη διασφάλιση της εγκυρότητας και συνέπειας των εγγραφών:

- Επαληθεύτηκε η μοναδικότητα της στήλης *id*, ώστε κάθε γεγονός να αντιστοιχεί σε μία μόνο εγγραφή
- Ελέγχθηκαν οι στήλες *userid* και *courseid* για τυχόν κενές ή μη έγκυρες τιμές
- Οι τιμές *NaN* στη στήλη *relateduserid* αντικαταστάθηκαν με *NULL*, ώστε να διατηρηθεί η ακεραιότητα του τύπου δεδομένων
- Επιβεβαιώθηκε ότι τα πεδία *crud* και *edulevel* περιείχαν αποκλειστικά τις επιτρεπόμενες κατηγορίες τιμών
- Ελέγχθηκε η χρονική συνέπεια μεταξύ των πεδίων *timecreated* (epoch) και *datetime* (timestamp)



Εικόνα 25: Ενδεικτικός έλεγχος εγκυρότητας δεδομένων στη PostgreSQL μέσω του περιβάλλοντος pgAdmin 4

Σύνοψη

Η χρήση της PostgreSQL σε συνδυασμό με το pgAdmin 4 διευκόλυνε την οργάνωση, επεξεργασία και επαλήθευση των δεδομένων Moodle. Η προσεκτική επιλογή τύπων δεδομένων, η εφαρμογή ευρετηρίων και οι έλεγχοι εγκυρότητας εξασφάλισαν ότι η βάση THESIS_DATABASE είναι αξιόπιστη, συνεπής και κατάλληλα προετοιμασμένη για τα επόμενα στάδια της ανάλυσης.

4.2.2 PostgREST API - Μεθοδολογία και βήματα υλοποίησης

Τα βήματα που ακολουθήθηκαν για την εγκατάσταση και τη ρύθμιση του PostgREST (σε λειτουργικό σύστημα Windows) περιγράφονται συνοπτικά παρακάτω και είναι εναρμονισμένα με τις απαιτήσεις της παρούσας εργασίας, δηλαδή την έκθεση του moodle_data ως REST API με ελεγχόμενη πρόσβαση.

Βήμα 1: Προϋποθέσεις

- Εγκατεστημένο PostgreSQL (credentials και όνομα βάσης διαθέσιμα)
- Ο πίνακας που θα εκτεθεί (π.χ. moodle_data)

Βήμα 2: Λήψη και βασική εγκατάσταση

1. Λήψη του εκτελέσιμου από: *GitHub* → *PostgREST* → *Releases* (έκδοση για Windows x64)
2. Αποσυμπίεση σε φάκελο, π.χ. C:\postgrest (πρέπει να υπάρχει το postgrest.exe)
3. Αν ζητηθεί libpq.dll, αντιγραφή από C:\Program Files\PostgreSQL\<ver>\bin\libpq.dll στον φάκελο C:\postgrest

Βήμα 3: Αρχείο ρυθμίσεων (postgrest.conf)

Δημιουργία αρχείου ρυθμίσεων με ελάχιστες παραμέτρους:

```
db-uri = "postgres://postgres:*****@localhost:5432/THESIS_DATABASE"
db-schema = "public"
db-anon-role = "anon"
server-port = 3000
```

Επεξήγηση:

db-uri: Στοιχεία σύνδεσης στη βάση

db-schema: Ποια σχήματα εκτίθενται

db-anon-role: Ο ρόλος που θα χρησιμοποιείται για ανώνυμες κλήσεις (μόνο τα ελάχιστα δικαιώματα)

server-port: Θύρα ακρόασης του API

Βήμα 4: Ρόλοι και δικαιώματα στη βάση (ασφάλεια/RLS)

Ελάχιστα, ενδεικτικά βήματα (προσαρμόζονται στις πολιτικές του οργανισμού):

```
CREATE ROLE anon NOLOGIN;
GRANT USAGE ON SCHEMA public TO anon;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO anon;
GRANT SELECT ON ALL VIEWS IN SCHEMA public TO anon;
GRANT SELECT ON ALL SEQUENCES IN SCHEMA public TO anon;

ALTER TABLE public.moodle_data ENABLE ROW LEVEL SECURITY;

CREATE POLICY api_read ON public.moodle_data
FOR SELECT USING (true);
```

Στο πρώτο "μπλοκ" εντολών δημιουργείται ο ρόλος *anon* (χωρίς δυνατότητα login) και απονέμονται τα ελάχιστα δικαιώματα χρήσης του schema καθώς και ανάγνωσης σε όλους τους πίνακες, τις όψεις (views) και, όπου απαιτείται, τις ακολουθίες (sequences) του schema public.

Στο δεύτερο "μπλοκ" εντολών ενεργοποιείται το RLS στον πίνακα *moodle_data*.

Στο τρίτο "μπλοκ" εντολών ορίζεται βασική πολιτική ανάγνωσης (SELECT).

Σχόλιο: Το RLS επιτρέπει λεπτό έλεγχο ορατότητας σε επίπεδο γραμμής. Για παραγωγικά περιβάλλοντα, συνίσταται ρητή διατύπωση συνθηκών (π.χ. φιλτράρισμα ανά ρόλο/μάθημα).

Βήμα 5: Εκκίνηση υπηρεσίας

Σε Command Prompt:

```
cd C:\postgrest
postgrest.exe postgrest.conf
```

Αναμενόμενο μήνυμα: *Listening on port 3000*.

Βήμα 6: Έλεγχος λειτουργίας (ενδεικτικά αιτήματα)

- Λίστα εγγραφών (με όριο):
`http://localhost:3000/moodle_data?limit=100`
- Φίλτρα και επιλογή πεδίων:
`http://localhost:3000/moodle_data?select=userid,action,datetime&action=eq.loggedin&order=datetime.desc&limit=50`
- Χρονικό διάστημα:
`http://localhost:3000/moodle_data?datetime=gte.2024-01-01&datetime=lt.2024-02-01&limit=100`

Βήμα 7: Ενσωμάτωση με ροές LLM

Η ενσωμάτωση με ροές LLM διευκολύνεται από το PostgREST, το οποίο παρέχει σταθερά HTTP endpoints για την αξιόπιστη ανάκτηση δεδομένων απευθείας από τη βάση. Με αυτόν τον τρόπο, τα δεδομένα καθίστανται άμεσα προσβάσιμα από συστήματα τεχνητής νοημοσύνης, επιτρέποντας την τυποποιημένη κατανάλωσή τους μέσω απλών κλήσεων API και χωρίς ενδιάμεσο εφαρμογικό στρώμα. Η προσέγγιση αυτή υποστηρίζει τόσο την ορχήστρωση ροών με **LangChain** (π.χ. καθορισμός prompts, chains, retrieval) όσο και την τοπική εκτέλεση μοντέλων μέσω LM Studio (OpenAI-compatible API), εξασφαλίζοντας συνεπή, ελεγχόμενη και αναπαραγώγιμη πρόσβαση στα ίδια σύνολα δεδομένων κατά την ανάλυση.

4.2.3 LangChain - Μεθοδολογία και βήματα υλοποίησης

Στόχος ήταν να δοθεί φυσική ερώτηση στα LLMs και να παραχθεί έγκυρο SQL query για το PostgreSQL schema, να εκτελεστεί και να επιστραφεί συνοπτική, χρήσιμη απάντηση. Η πορεία υλοποίησης έγινε σε δύο στάδια (prototype → παραγωγικό).

Βήμα 1: Εγκατάσταση πακέτων (pip install)

Εγκαταστάθηκαν τα απαραίτητα πακέτα με εκτέλεση της παρακάτω εντολής μέσω του Command Prompt:

```
pip install langchain langchain-community langchain-openai sqlalchemy psycpg2-binary
```

(LangChain & langchain-community / langchain-openai για chains και LLM client, SQLAlchemy/psycpg2 για PostgreSQL.)

Βήμα 2: Πρωτότυπο με LangChain SQLDatabaseChain

Δημιουργήθηκε ένα ελάχιστο λειτουργικό με το SQLDatabaseChain, που «μεταφράζει» φυσικές ερωτήσεις σε SQL και εκτελεί το ερώτημα:

```
from langchain_community.utilities import SQLDatabase
from langchain_openai import ChatOpenAI
from langchain_experimental.sql import SQLDatabaseChain

db = SQLDatabase.from_uri("postgresql+psycpg2://username:password@localhost:5432/dbname")

llm = ChatOpenAI(
    base_url="http://localhost:1234/v1",
    api_key="lm-studio",
    model="any-model",
    temperature=0.0,
)

db_chain = SQLDatabaseChain.from_llm(llm, db, verbose=True)
answer = db_chain.run("Your question here")
print(answer)
```

Το prototype επιβεβαίωσε ότι η ροή: 'Φυσική γλώσσα → SQL → Απάντηση' λειτουργεί με LM Studio.

Βήμα 3: Ενίσχυση της ροής με σαφές prompt και ελεγχόμενη εκτέλεση

Στο στάδιο αυτό διαμορφώνεται ένα σαφές πρότυπο οδηγιών (PromptTemplate) προς το LLM, βασισμένο στο πραγματικό schema της βάσης, και ακολουθεί ελεγχόμενη εκτέλεση του παραγόμενου SQL μέσω PostgreSQL. Σε περίπτωση σφάλματος, εφαρμόζεται κυκλική ανατροφοδότηση για αυτόματη διόρθωση.

```
from sqlalchemy import create_engine
from sqlalchemy.engine import URL

DB_URL = URL.create(
    "postgresql+psycpg2",
    username="postgres", password="*****",
    host="127.0.0.1", port=5432, database="THESIS_DATABASE",
)
engine = create_engine(DB_URL, pool_pre_ping=True)
```

Ο παρουσιαζόμενος κώδικας παρατίθεται ενδεικτικά, με σκοπό την περιγραφή της βασικής ροής, και δεν αποτελεί την πλήρη υλοποίηση

Βήμα 3.1: Απόκτηση schema βάσης

Το schema είναι η περιγραφή των πινάκων και των στηλών τους. Με το αντικείμενο "SQLDatabase" της LangChain πραγματοποιείται ανάκτηση του schema (π.χ. διαθέσιμες στήλες στον πίνακα moodle_data). Η πληροφορία αυτή παρέχεται στο LLM, ώστε να μην γίνεται χρήση ανύπαρκτων πεδίων.

```
from langchain_community.utilities import SQLDatabase

db = SQLDatabase(engine=engine)
table_info = db.get_table_info()
```

Ο παρουσιαζόμενος κώδικας παρατίθεται ενδεικτικά, με σκοπό την περιγραφή της βασικής ροής, και δεν αποτελεί την πλήρη υλοποίηση

Βήμα 3.2: Καθορισμός PromptTemplate (σαφές πρότυπο οδηγιών)

Το PromptTemplate είναι δομημένο κείμενο που καθοδηγεί το μοντέλο. Περιλαμβάνει:

- Τη λίστα των στηλών του moodle_data και βασικές σημασίες (π.χ. τιμές από την στήλη "action" όπως *loggedin*, *loggedout*, *viewed*). Παρόλο που έχουν ληφθεί νωρίτερα οι ονομασίες των στηλών του πίνακα, είναι χρήσιμο να αναφερθούν ρητώς στο PromptTemplate ώστε να μην επινοεί το LLM αυθαίρετα "νέες" μη-υπαρκτές στήλες (LLM Hallucination)
- Κανόνες χρόνου (χρήση της στήλης "time" όταν είναι διαθέσιμη, εναλλακτικά to_timestamp(timecreated))
- Κανόνες σύνταξης (ένα και μόνο πλήρες SQL ερώτημα που καταλήγει σε ";", δυνατότητα χρήσης CTE με ορθή «εμβέλεια» στηλών)

Η διατύπωση αυτή περιορίζει την αοριστία και αυξάνει την πιθανότητα παραγωγής έγκυρου SQL.

```
from langchain.prompts import PromptTemplate

sql_prompt = PromptTemplate(
    input_variables=["input", "table_info"],
    template="""
You are a PostgreSQL expert. Output ONE executable SQL statement for this dataset.

Database schema:
{table_info}

Use table:
- moodle_data(id, eventname, component, action, target, crud, edulevel,
    contextid, contextlevel, contextinstanceid, userid, courseid,
    relateduserid, anonymous, timecreated, origin, datetime, "time", day, month)

Rules:
```

- Prefer "time":time when present; else use to_timestamp(timecreated).
- For logins use action IN ('loggedin','loggedinas'); logout: 'loggedout'.
- ONE complete SQL ending with ';' then <END>. No prose.

Task:

{input}

SQL:

```
""""
)
```

Ο παρουσιαζόμενος κώδικας παρατίθεται ενδεικτικά, με σκοπό την περιγραφή της βασικής ροής, και δεν αποτελεί την πλήρη υλοποίηση

Βήμα 3.3: Κλήση LLM μέσω LM Studio (τοπικό API)

Το LM Studio λειτουργεί ως τοπικός εξυπηρετητής (API) για την εκτέλεση LLMs. Ο κώδικας αποστέλλει το PromptTemplate και το schema βάσης στο endpoint του LM Studio (OpenAI-compatible API) και λαμβάνει ως απόκριση το προτεινόμενο SQL ερώτημα.

```
import re
from openai import OpenAI

client = OpenAI(base_url="http://localhost:1234/v1", api_key="lm-studio")
MODEL_NAME = "hermes-3-llama-3.2-3b"

def extract_sql(text: str) -> str:
    text = text.strip()
    if "<END>" in text:
        text = text.split("<END>", 1)[0]
    lower = text.lower()
    if "with" in lower:
        start = lower.find("with")
        end = text.rfind(";")
        if start != -1 and end != -1 and end > start:
            return text[start:end+1].strip()
    m = re.search(r"(?is)\bselect\b[\s\S]*?;", text)
    if m:
        return m.group(0).strip()
    m2 = re.search(r"(?is)\bselect\b[\s\S]*$", text)
    if m2:
        sql = m2.group(0).strip()
        if not sql.endswith(";"):
            sql += ";"
        return sql
    raise ValueError("No SQL query found in LLM output")

messages = [
    {"role": "system", "content": "You output exactly one valid SQL statement; nothing else."},
    {"role": "user", "content": sql_prompt.format(
        input="Show 10 sample rows from moodle_data ordered by id",
        table_info=table_info
    )},
]
```

```

resp = client.chat.completions.create(
    model=MODEL_NAME,
    messages=messages,
    temperature=0.0,
    max_tokens=350,
    stop=["<END>"],
)
llm_text = resp.choices[0].message.content.strip()

sql = extract_sql(llm_text)
print("[LLM → SQL]\n", sql)

```

Ο παρουσιαζόμενος κώδικας παρατίθεται ενδεικτικά, με σκοπό την περιγραφή της βασικής ροής, και δεν αποτελεί την πλήρη υλοποίηση

Βήμα 3.4: Εκτέλεση SQL και έλεγχος αποτελεσμάτων (SQLAlchemy)

Η SQLAlchemy χρησιμοποιείται για την ασφαλή εκτέλεση του SQL στην PostgreSQL και την ανάκτηση των αποτελεσμάτων. Σε περίπτωση σφάλματος (π.χ. "column does not exist", συντακτικό λάθος), δημιουργείται στοχευμένο μήνυμα ανατροφοδότησης προς το LLM και επαναλαμβάνεται ο κύκλος παραγωγής-εκτέλεσης έως ότου παραχθεί έγκυρο ερώτημα ή εξαντληθούν οι προσπάθειες.

```

sql = extract_sql(llm_text)

with engine.connect() as conn:
    result = conn.exec_driver_sql(sql)
    rows = result.fetchall()
    cols = result.keys()

```

Ο παρουσιαζόμενος κώδικας παρατίθεται ενδεικτικά, με σκοπό την περιγραφή της βασικής ροής, και δεν αποτελεί την πλήρη υλοποίηση

4.2.4 Υλοποίηση κώδικα

Παρακάτω παρατίθενται ενδεικτικά τμήματα του κώδικα, ομαδοποιημένα στα πέντε υπο-βήματα που συγκροτούν τη ροή Φυσική γλώσσα → SQL → Εκτέλεση → Απάντηση.

α) Έλεγχος σύνδεσης και δημιουργία SQLAlchemy engine

Στόχος του βήματος αυτού είναι να διασφαλιστεί ότι η σύνδεση προς PostgreSQL λειτουργεί (smoke test με psycopg2), και να δημιουργηθεί ένα σταθερό *engine* για τις εκτελέσεις SQL.

```

def _smoke_test():
    try:
        import psycopg2
        conn = psycopg2.connect(
            dbname=PG_DB, user=PG_USER, password=PG_PASS, host=PG_HOST,
            port=PG_PORT
        )
        cur = conn.cursor()
        cur.execute("SELECT 1;")
        cur.fetchone()
    
```

```

        cur.close()
        conn.close()
        print("[Smoke Test] psycopg2 connect OK")
    except Exception as e:
        raise SystemExit(f"[Smoke Test] FAILED: {e}")

_smoke_test()

from sqlalchemy.engine import URL
from sqlalchemy import create_engine

DB_URL = URL.create(
    "postgresql+psycopg2",
    username=PG_USER,
    password=PG_PASS,
    host=PG_HOST,
    port=PG_PORT,
    database=PG_DB,
)
engine = create_engine(DB_URL, pool_pre_ping=True)

with engine.connect() as conn:
    v = conn.exec_driver_sql(
        "select inet_server_addr(), inet_server_port(), current_database();"
    ).fetchone()
    print("[SQLAlchemy Test] OK ->", v)

```

β) Ανάκτηση schema και διαμόρφωση PromptTemplate

Στόχος εδώ είναι να «δέσει» το μοντέλο πάνω στο πραγματικό schema (στήλες/πίνακες) και σε σαφείς κανόνες εξόδου (ένα και μόνο SQL που καταλήγει σε ; + <END>), μειώνοντας αστοχίες/«εφευρέσεις» στηλών.

```

from langchain_community.utilities import SQLDatabase
from langchain.prompts import PromptTemplate

db = SQLDatabase(engine=engine)
schema = db.get_table_info()

sql_prompt = PromptTemplate(
    input_variables=["input", "table_info"],
    template="""
You are a PostgreSQL expert who outputs exactly ONE executable SQL statement for
this specific dataset.

Database schema:
{table_info}

Table & columns (do NOT invent):
- moodle_data(

```

```

id, eventname, component, action, target, crud, edulevel,
contextid, contextlevel, contextinstanceid, userid, courseid,
relateduserid, anonymous, timecreated, origin, datetime, time, day, month
)

... (κανόνες ενεργειών/χρόνου/μορφοποίησης) ...

Task:
{input}

SQL:
"""
)

```

γ) Κλήση LLM (LM Studio API) και εξαγωγή του SQL

Σε αυτό το βήμα στόχος είναι να σταλεί το prompt στο LLM (μέσω LM Studio, OpenAI-compatible), και να εξαχθεί μοναδικό SQL query (υποστήριξη και για CTEs).

```

from openai import OpenAI
import re

client = OpenAI(base_url="http://localhost:1234/v1", api_key="lm-studio")
MODEL_NAME = "hermes-3-llama-3.2-3b"

def extract_sql(text: str) -> str:
    if "CANNOT_ANSWER" in text:
        raise ValueError("Model refused: CANNOT_ANSWER (missing columns)")
    text = text.split("<END>", 1)[0]
    lower = text.lower()
    if "with" in lower:
        start = lower.find("with"); end = text.rfind(";")
        if start != -1 and end != -1 and end > start: return
    text[start:end+1].strip()
    m = re.search(r"(?is)\bselect\b[\s\S]*?;", text)
    if m: return m.group(0).strip()
    m2 = re.search(r"(?is)\bselect\b[\s\S]*$", text)
    if m2:
        sql = m2.group(0).strip()
        if not sql.endswith(";"): sql += ";"
        return sql
    raise ValueError("No SQL query found in LLM output")

messages = [
    {"role": "system", "content": "You are a precise PostgreSQL SQL generator. Output only SQL."},
    {"role": "user", "content": sql_prompt.format(input="Show 10 sample rows", table_info=schema)},

```

```

]

resp = client.chat.completions.create(
    model=MODEL_NAME,
    messages=messages,
    temperature=0.0,
    top_p=0.9,
    max_tokens=350,
    seed=42,
    stop=["<END>"],
)

llm_output = resp.choices[0].message.content.strip()
sql = extract_sql(llm_output)
print("[LLM → SQL]\n", sql)

```

Η τιμή της **θερμοκρασίας** (temperature) στο παρόν απόσπασμα που έχει οριστεί ως 0.0, αφορά την παραγωγή SQL και όχι την επεξεργασία των δεδομένων σε δεύτερο χρόνο από το LLM και την παραγωγή φυσικής γλώσσας. Αυτό γίνεται επειδή στην παραγωγή SQL είναι σημαντικό να υπάρχει όσο το δυνατόν περισσότερο ντετερμινιστικό και σταθερό αποτέλεσμα και να μην υπάρχουν καθόλου «αυθαιρεσίες».

δ) Εκτέλεση SQL, ανάγνωση αποτελεσμάτων και σύντομη περίληψη

Στόχος εδώ είναι να εκτελεστεί με ασφάλεια το SQL, να ανακτηθούν οι τιμές, και όταν το αποτέλεσμα είναι σε μέγεθος που το επιτρέπει, να παραχθεί ένα συνοπτικό κείμενο που επεξηγεί/αναφέρει τα δεδομένα που επιστράφηκαν.

```

import json, time
from sqlalchemy.exc import SQLAlchemyError

start = time.time()
with engine.connect() as conn:
    result = conn.exec_driver_sql(sql)
    rows = result.fetchall()
    columns = result.keys()

print("\n[Query Results]\n", rows)

def summarize_result(question: str, sql: str, columns, rows):
    payload = {"question": question, "sql": sql, "columns": list(columns),
               "total_rows": len(rows), "sample_rows": rows[:2]}
    messages = [
        {"role": "system", "content":
            "You are a precise analyst. Based ONLY on the provided SQL result JSON, "
            "answer the user's question in ONE concise sentence."},
        {"role": "user", "content": json.dumps(payload, ensure_ascii=False)},
    ]
    resp = client.chat.completions.create(
        model=MODEL_NAME,

```

```

        messages=messages,
        temperature=0.1,
        top_p=0.9,
        max_tokens=100,
        seed=42
    )
    return resp.choices[0].message.content.strip()

if len(rows) > 20:
    print("\n[Answer]\nReturned", len(rows), "rows. (Summary skipped for large result.)")
else:
    human = summarize_result("Show 10 sample rows", sql, columns, rows)
    print("\n[Answer]\n", human)

elapsed = time.time() - start
print(f"\nSuccess in {elapsed:.2f} seconds")

```

Η τιμή της θερμοκρασίας στο παρόν απόσπασμα είναι 0.1 (θα μπορούσε να έχει οποιαδήποτε τιμή μεταξύ 0 και 1, σε άλλες μετρήσεις ήταν 0.5) και αφορά την περίληψη και την επεξεργασία σε δεύτερο χρόνο των αποτελεσμάτων (αν χρειάζεται). Εδώ είναι θεμιτή έως ένα βαθμό η ελευθερία στην διατύπωση και η «αυθαιρεσία» από το LLM καθώς η υπολογιστική ισχύς που χρειάζεται από πλευράς του είναι μικρή οπότε πολύ δύσκολα θα δημιουργηθούν αναξιόπιστα αποτελέσματα. Αντιθέτως, μπορεί η ελευθερία που του δίνεται να οδηγήσει σε πιο «εφευρετική» χρήση των δεδομένων που του παρατίθενται.

ε) Στοχευμένος χειρισμός σφαλμάτων και retry loop

Σε αυτό το βήμα επιδιώκεται σε περίπτωση αποτυχίας (π.χ. «column does not exist», «syntax error»), να επιστρέφεται στοχευμένη ανατροφοδότηση στο LLM για διόρθωση του SQL και επανάληψη μέχρι ορισμένο όριο.

```

from typing import Optional

def summarize_db_error(err: Exception) -> Optional[str]:
    msg = str(err)
    m = re.search(r'column "[^"]+" does not exist', msg, flags=re.IGNORECASE)
    if m:
        col = m.group(1)
        return (
            f'You referenced column "{col}" that is not in scope. '
            f'If you used a CTE, include it in that CTE\'s SELECT list.'
        )
    if "relation" in msg.lower() and "does not exist" in msg.lower():
        return "You referenced a table or alias that does not exist. Use only the table moodle_data."
    if "syntax error" in msg.lower():
        return ("Your SQL has a syntax error. Ensure each CTE/SELECT is properly closed "
            "and the final statement ends with a semicolon.")
    if "timestamp" in msg.lower() or "time zone" in msg.lower():
        return ("Use to_timestamp(timecreated) for epoch or use datetime directly; "
            "avoid string concatenation with timestamps.")
    return None

```

```

def ask_question(question: str, retries: int = 4):
    schema = db.get_table_info()
    messages = [
        {"role": "system", "content": "You are a precise PostgreSQL SQL generator. Output only SQL."},
        {"role": "user", "content": sql_prompt.format(input=question, table_info=schema)},
    ]
    for attempt in range(1, retries + 1):
        response = client.chat.completions.create(
            model=MODEL_NAME, messages=messages, temperature=0.0, top_p=0.9,
            max_tokens=350, seed=42, stop=["<END>"],
        )
        llm_output = response.choices[0].message.content.strip()
        try:
            sql = extract_sql(llm_output)
            with engine.connect() as conn:
                result = conn.exec_driver_sql(sql)
                rows = result.fetchall()
            print(f"[Attempt {attempt}] OK → rows={len(rows)}"); return
        except (SQLAlchemyError, Exception) as e:
            guidance = summarize_db_error(e) or ""
            if attempt >= retries:
                print("Query failed after retries."); return
            messages.extend([
                {"role": "assistant", "content": llm_output},
                {"role": "user", "content":
                    (f"The SQL above failed with error:\n{e}\n\n"
                     "Correct the SQL. Use only columns from moodle_data; one complete statement ending with ';'."
                     f"{guidance}\n"
                     f"Database schema for reference:\n{schema}\n\n"
                     "Return only the corrected SQL statement ending with a semicolon, then <END>."
                    )}
            ])

```

Σε όλα τα αποσπάσματα του κώδικα, αλλά και στον πλήρη κώδικα, οι μόνες παράμετροι που αλλάζουν μεταξύ των μετρήσεων είναι το όνομα του εκάστοτε LLM, η θερμοκρασία καθώς και το εκάστοτε prompt το οποίο δίνεται στο πρόγραμμα από τον χρήστη. Όλες οι υπόλοιπες παράμετροι παραμένουν σταθερές ανεξαρτήτως της φύσης του task που δίνεται στο πρόγραμμα.

4.2.5 Υλοποίηση prompt engineering

Στην παρούσα ενότητα, πέρα από την περιγραφή των επιμέρους βημάτων του κώδικα, είναι σημαντικό να αποτυπωθεί πώς οι τεχνικές prompt engineering που παρουσιάστηκαν στο Κεφάλαιο 2 μεταφράζονται σε συγκεκριμένες επιλογές στην υλοποίηση. Ο πυρήνας της προσέγγισης είναι ότι το prompt δεν λειτουργεί ως απλή «ερώτηση προς το μοντέλο», αλλά ως μηχανισμός ελέγχου που καθορίζει τι ζητείται, ποιοι πόροι/δεδομένα επιτρέπεται να χρησιμοποιηθούν και ποια μορφή εξόδου είναι αποδεκτή, ώστε η έξοδος να είναι εκτελέσιμη και αναπαραγώγιμη.

Πρώτον, εφαρμόζεται **instructional prompting** μέσω ρητών κανόνων που επιβάλλουν συγκεκριμένη μορφή αποτελέσματος (ένα και μόνο εκτελέσιμο ερώτημα, χωρίς επεξηγηματικό κείμενο), περιορίζοντας την ασάφεια και αυξάνοντας τη συνέπεια της παραγόμενης εξόδου. Αυτό αποτυπώνεται άμεσα στο prompt template, όπου

ορίζονται αυστηροί κανόνες μορφοποίησης/εξόδου [7], [8]. Παρακάτω παρουσιάζεται ένα ενδεικτικό σημείο του κώδικα στο οποίο πραγματοποιείται η συγκεκριμένη υλοποίηση.

```
template = ""
You are a PostgreSQL expert who outputs exactly ONE executable SQL statement...

Formatting rules:
- Return exactly ONE complete SQL statement ending with a semicolon, then the
token <END>.
- No comments, no markdown, no prose.
...
SQL:
""
```

Δεύτερον, υλοποιείται **schema grounding**, καθώς στο prompt εισάγεται το πραγματικό σχήμα της βάσης. Στον κώδικα αυτό γίνεται με ανάκτηση των πληροφοριών του σχήματος και εισαγωγή τους στο template μέσω της μεταβλητής "table_info" (ώστε το μοντέλο να "βλέπει" τους διαθέσιμους πίνακες/στήλες και να μην εφευρίσκει πεδία) [11]. Παρακάτω παρουσιάζεται ένα ενδεικτικό σημείο του κώδικα στο οποίο πραγματοποιείται η συγκεκριμένη υλοποίηση.

```
schema = db.get_table_info()

messages = [
    {"role": "system", "content": "You are a precise PostgreSQL SQL generator.
Output only SQL."},
    {"role": "user", "content": sql_prompt.format(input=question,
table_info=schema)},
]
```

Στο ίδιο πνεύμα, το template δηλώνει ρητά τον πίνακα και τις στήλες που επιτρέπεται να χρησιμοποιηθούν ("Table & columns (do NOT invent) ... moodle_data(...)", αλλά και περιορισμούς όπως "Do not invent columns" και "Use only standard PostgreSQL functions", που λειτουργούν ως επιπλέον "άγκυρες" προς το πραγματικό περιβάλλον εκτέλεσης [11].

Τρίτον, σε ερωτήματα που απαιτούν διαδοχικά βήματα επίλυσης χρησιμοποιείται **step-by-step prompting** (Chain-of-Thought ως "καθοδήγηση διαδικασίας"), μέσω ρητής εντολής στο prompt του χρήστη (π.χ. "Think step by step."). Έτσι το μοντέλο ωθείται να οργανώσει τη λύση με λογική ακολουθία ενεργειών πριν καταλήξει στο τελικό ερώτημα [26]. Παρακάτω παρουσιάζεται ενδεικτικά απόσπασμα ενός prompt που χρησιμοποιήθηκε στην υλοποίηση της παρούσας εργασίας.

```
... Tell me when the 15-minute window starts and when it ends. Think step by step.
```

Τέλος, η ανθεκτικότητα της ροής ενισχύεται με **feedback / error-aware prompting**, μέσω επαναληπτικού κύκλου διόρθωσης. Όταν η εκτέλεση αποτύχει, ο κώδικας "επιστρέφει" στο μοντέλο το μήνυμα σφάλματος της βάσης μαζί με στοχευμένες οδηγίες διόρθωσης, αυξάνοντας την πιθανότητα επιτυχούς επανεκτέλεσης χωρίς χειροκίνητη παρέμβαση [7], [8], [11].

```
except (SQLAlchemyError, Exception) as e:
    guidance = summarize_db_error(e)
    correction = (
        "Correct the SQL.\n"
```

```

    "- Only one complete SQL statement ending with a semicolon.\n"
    "- If you used a CTE, only reference columns selected by that CTE...\n"
    "- Use only columns that exist in moodle_data...\n"
)
if guidance:
    correction += f"\nSpecific error to fix: {guidance}\n"

messages.extend([
    {"role": "assistant", "content": llm_output},
    {"role": "user", "content": f"The SQL above failed with\nerror:\n{e}\n\n{correction}\n..."},
])

```

Με τον παραπάνω σχεδιασμό, οι τεχνικές που παρουσιάζονται θεωρητικά στο Κεφάλαιο 2 αποκτούν συγκεκριμένη μορφή στην υλοποίηση: αποτυπώνονται αφενός στο PromptTemplate (κανόνες, schema, παραδείγματα) και αφετέρου στον κύκλο εκτέλεσης (έλεγχος, εκτέλεση, ανατροφοδότηση σφάλματος και retries), ώστε η παραγωγή να παραμένει ελεγχόμενη και όσο το δυνατόν πιο αναπαραγώγιμη. [7], [8], [11], [25] [26].

Κεφάλαιο 5ο: Αποτελέσματα

Στο παρόν κεφάλαιο παρουσιάζεται η πειραματική αξιολόγηση του συστήματος ερωτήσεων σε φυσική γλώσσα πάνω στα καθαρισμένα Moodle Logs, με στόχο να εξεταστεί κατά πόσο διαφορετικά LLMs μπορούν να υποστηρίξουν αξιόπιστη ανάκτηση πληροφορίας από σχεσιακή βάση δεδομένων. Για να διασφαλιστεί συγκρισιμότητα, τα πειράματα οργανώθηκαν ως ένα κοινό σύνολο cases που αντιστοιχούν σε ρεαλιστικές αναλυτικές ανάγκες ενός εκπαιδευτικού. Κάθε σενάριο εκφράστηκε ως prompt σε φυσική γλώσσα, έτσι ώστε να ελέγχεται όχι μόνο η ικανότητα του μοντέλου να "καταλάβει" το ερώτημα, αλλά και να παραγάγει ένα εκτελέσιμο και σημασιολογικά ορθό SQL query προς τη βάση, με σταθερή μορφή και χωρίς αυθαίρετες υποθέσεις. Στη συνέχεια, τα ίδια prompts εκτελέστηκαν επαναληπτικά για κάθε μοντέλο και για τις επιλεγμένες ρυθμίσεις δειγματοληψίας, ώστε να καταγραφούν χρόνοι απόκρισης και ποσοστά επιτυχίας ανά περίπτωση.

Οι μετρήσεις έγιναν με δύο θερμοκρασίες (0.1 και 0.5) για τέσσερα prompts. Σύμφωνα με την πειραματική παρατήρηση:

- Το **Hermes LLaMA 3.2 (3B)** παρουσίασε σαφώς το υψηλότερο ποσοστό επιτυχίας
- Το **Phi-4 Mini Reasoning (3B)** είχε επίσης καλά ποσοστά, λίγο χαμηλότερα από το LLaMA
- Το **DeepSeek Chat (7B)** εμφάνισε σημαντικά χαμηλότερο ποσοστό επιτυχίας
- Οι χρόνοι ολοκλήρωσης ήταν παρεμφερείς για όλα τα μοντέλα (χωρίς στατιστικά σημαντικές διαφορές)

Τα prompts που χρησιμοποιήθηκαν είναι τα εξής:

PROMPT 1: According to the dataset provided to you, what is the most popular 15-minute window of the day that log-ins happen the most? First, use the "action" column and use the "loggedin" actions. Then use the "time" column to determine what time these actions happen. Tell me when the 15-minute window starts and when it ends. Think step by step.

Ενδεικτική απάντηση: The most popular log-in window is from 5:45 AM to 6:00 AM.

Επιθυμητό Query Result: [(1, 12492498), (2021, 651346), (39, 416576), (3, 414361), (2050, 390374), (1963, 332076), (1824, 330549), (26, 289180), (2681, 272535), (0, 268586), (532, 253656), (28, 251830), (275, 248896), (298, 246934), (338, 238656), (228, 220655), (21, 208180), (2982, 204738), (396, 179671), (3094, 167330)]

Σκοπός του πρώτου prompt είναι να ελεγχθεί αν το μοντέλο μπορεί να χειριστεί χρονική ανάλυση και να απαντήσει σε ένα ερώτημα που απαιτεί μετασχηματισμό χρόνου, φίλτρο πάνω σε συγκεκριμένο τύπο συμβάντος (logins) και τελική επιλογή του πιο συχνού χρονικού διαστήματος. Είναι ένα "δύσκολο" σενάριο γιατί συνδυάζει χρονικά πεδία, ομαδοποίηση και εύρεση μέγιστου.

PROMPT 2: For month='January', which actions have the most total events? (Use 'month' and 'action'; return action and event_count.). Return top 5 by that count.

Ενδεικτική απάντηση: The top 5 actions with the most total events in January are: viewed (3,588,061), loggedin (237,164), updated (184,620), graded (156,858), and called (131,830).

Επιθυμητό Query Result: [('viewed', 3588061), ('loggedin', 237164), ('updated', 184620), ('graded', 156858), ('called', 131830)]

Σκοπός εδώ είναι να αξιολογηθεί η ικανότητα του μοντέλου να εκτελεί σωστά φιλτράρισμα σε μηνιαία βάση και στη συνέχεια να ομαδοποιεί και να κατηγοριοποιεί τα δεδομένα ανάλογα με το ζητούμενο (στην προκειμένη περίπτωση ζητείται να επιστρέψει τα 5 actions με τις περισσότερες εμφανίσεις σε έναν συγκεκριμένο μήνα). Αποτελεί επίσης "δύσκολο" σενάριο καθώς συνδυάζει διάφορες στήλες της βάσης δεδομένων και επεξεργασία των δεδομένων.

PROMPT 3: For userid=10014, which courseid did they interact with the most? Count all actions (from 'action' column) for that userid grouped by courseid; return top 5.

Ενδεικτική απάντηση: The user with userid 10014 interacted most with courseids 3242 and 5855, each having a total count of 320.

Επιθυμητό Query Result: [(1, Decimal('406')), (3242, Decimal('320')), (5855, Decimal('320')), (2210, Decimal('319')), (3053, Decimal('229'))]

Σκοπός του τρίτου prompt είναι να ελεγχθεί ένα σενάριο "ανά φοιτητή", δηλαδή αν το μοντέλο μπορεί να εκτελέσει εξατομικευμένη ανάλυση: φίλτρο σε συγκεκριμένο userid, ομαδοποίηση ανά courseid και ανάδειξη των μαθημάτων με τη μεγαλύτερη αλληλεπίδραση. Αυτό αντιστοιχεί σε πρακτική ανάγκη εκπαιδευτικού (π.χ. σε ποια μαθήματα/χώρους είναι ενεργός ένας φοιτητής) και ελέγχει αν το μοντέλο διαχειρίζεται σωστά αριθμητικά φίλτρα και ομαδοποιήσεις.

PROMPT 4: Top 20 courses by number of 'viewed' actions (column courseid).

Ενδεικτική απάντηση: The top 20 courses by number of 'viewed' actions are: 1 with 12,492,498 views, followed by course 2021 with 651,346 views.

Επιθυμητό Query Result: [(1, 12492498), (2021, 651346), (39, 416576), (3, 414361), (2050, 390374), (1963, 332076), (1824, 330549), (26, 289180), (2681, 272535), (0, 268586), (532, 253656), (28, 251830), (275, 248896), (298, 246934), (338, 238656), (228, 220655), (21, 208180), (2982, 204738), (396, 179671), (3094, 167330)]

Σκοπός του τέταρτου prompt είναι να εξεταστεί ένα σενάριο "ανά μάθημα", όπου ζητείται κατάταξη (ranking) με βάση μια συγκεκριμένη συμπεριφορά (προβολές/viewed). Είναι πιο "καθαρό" από το Prompt 1 γιατί δεν έχει μετασχηματισμό χρόνου, αλλά ελέγχει αν το μοντέλο εκτελεί σωστά φίλτρο σε κατηγορική τιμή (action='viewed'), καταμέτρηση, ταξινόμηση και επιστροφή μεγαλύτερου πλήθους αποτελεσμάτων (top 20).

5.1 Hermes Llama 3.2 (3B)

PROMPT 1 RESULTS (LLAMA)

ΘΕΡΜΟΚΡΑΣΙΑ = 0.1		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	134.06 sec	ΕΠΙΤΥΧΙΑ
2	135.44 sec	ΕΠΙΤΥΧΙΑ
3	134.56 sec	ΕΠΙΤΥΧΙΑ
4	135.99 sec	ΕΠΙΤΥΧΙΑ
5	136.66 sec	ΕΠΙΤΥΧΙΑ
6	136.79 sec	ΕΠΙΤΥΧΙΑ
7	132.56 sec	ΕΠΙΤΥΧΙΑ
8	134.19 sec	ΕΠΙΤΥΧΙΑ
9	133.62 sec	ΕΠΙΤΥΧΙΑ
10	130.82 sec	ΕΠΙΤΥΧΙΑ
ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ		134.17 sec
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ		100%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ		100%

ΘΕΡΜΟΚΡΑΣΙΑ = 0.5		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	137.12 sec	ΕΠΙΤΥΧΙΑ
2	147.57 sec	ΕΠΙΤΥΧΙΑ
3	136.78 sec	ΕΠΙΤΥΧΙΑ
4	134.83 sec	ΕΠΙΤΥΧΙΑ
5	133.82 sec	ΕΠΙΤΥΧΙΑ
6	134.92 sec	ΕΠΙΤΥΧΙΑ
7	133.76 sec	ΕΠΙΤΥΧΙΑ
8	133.79 sec	ΕΠΙΤΥΧΙΑ
9	139.95 sec	ΕΠΙΤΥΧΙΑ
10	140.49 sec	ΕΠΙΤΥΧΙΑ
ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ		137.30 sec
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ		100%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ		100%

PROMPT 2 RESULTS (LLAMA)

ΘΕΡΜΟΚΡΑΣΙΑ = 0.1		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	127.43 sec	ΕΠΙΤΥΧΙΑ
2	126.02 sec	ΕΠΙΤΥΧΙΑ
3	125.13 sec	ΕΠΙΤΥΧΙΑ
4	126.28 sec	ΕΠΙΤΥΧΙΑ
5	126.14 sec	ΕΠΙΤΥΧΙΑ
6	126.80 sec	ΕΠΙΤΥΧΙΑ
7	126.26 sec	ΕΠΙΤΥΧΙΑ
8	125.70 sec	ΕΠΙΤΥΧΙΑ

9	132.12 sec	ΕΠΙΤΥΧΙΑ
10	129.06 sec	ΕΠΙΤΥΧΙΑ

ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ	128.04 sec
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ	100%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ	100%

ΘΕΡΜΟΚΡΑΣΙΑ = 0.5		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	127.48 sec	ΕΠΙΤΥΧΙΑ
2	126.87 sec	ΕΠΙΤΥΧΙΑ
3	128.21 sec	ΕΠΙΤΥΧΙΑ
4	126.92 sec	ΕΠΙΤΥΧΙΑ
5	125.48 sec	ΕΠΙΤΥΧΙΑ
6	127.62 sec	ΕΠΙΤΥΧΙΑ
7	125.79 sec	ΕΠΙΤΥΧΙΑ
8	131.84 sec	ΕΠΙΤΥΧΙΑ
9	129.56 sec	ΕΠΙΤΥΧΙΑ
10	125.99 sec	ΕΠΙΤΥΧΙΑ

ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ	127.58 sec
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ	100%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ	100%

PROMPT 3 RESULTS (LLAMA)

ΘΕΡΜΟΚΡΑΣΙΑ = 0.1

ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	4.26 sec	ΕΠΙΤΥΧΙΑ
2	4.20 sec	ΕΠΙΤΥΧΙΑ
3	4.20 sec	ΕΠΙΤΥΧΙΑ
4	4.20 sec	ΕΠΙΤΥΧΙΑ
5	4.12 sec	ΕΠΙΤΥΧΙΑ
6	4.15 sec	ΕΠΙΤΥΧΙΑ
7	4.21 sec	ΕΠΙΤΥΧΙΑ
8	4.16 sec	ΕΠΙΤΥΧΙΑ
9	4.22 sec	ΕΠΙΤΥΧΙΑ
10	4.22 sec	ΕΠΙΤΥΧΙΑ

ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ	4.20 sec
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ	100%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ	100%

ΘΕΡΜΟΚΡΑΣΙΑ = 0.5		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	4.02 sec	ΕΠΙΤΥΧΙΑ
2	4.20 sec	ΕΠΙΤΥΧΙΑ
3	4.17 sec	ΕΠΙΤΥΧΙΑ
4	4.09 sec	ΕΠΙΤΥΧΙΑ
5	4.11 sec	ΕΠΙΤΥΧΙΑ
6	4.21 sec	ΕΠΙΤΥΧΙΑ
7	4.18 sec	ΕΠΙΤΥΧΙΑ
8	4.20 sec	ΕΠΙΤΥΧΙΑ
9	4.26 sec	ΕΠΙΤΥΧΙΑ
10	4.22 sec	ΕΠΙΤΥΧΙΑ

ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ	4.17 sec
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ	100%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ	100%

PROMPT 4 RESULTS (LLAMA)

ΘΕΡΜΟΚΡΑΣΙΑ = 0.1		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	126.32 sec	ΕΠΙΤΥΧΙΑ
2	125.00 sec	ΕΠΙΤΥΧΙΑ
3	125.18 sec	ΕΠΙΤΥΧΙΑ
4	125.07 sec	ΕΠΙΤΥΧΙΑ
5	127.28 sec	ΕΠΙΤΥΧΙΑ
6	124.04 sec	ΕΠΙΤΥΧΙΑ
7	123.40 sec	ΕΠΙΤΥΧΙΑ
8	124.51 sec	ΕΠΙΤΥΧΙΑ
9	126.64 sec	ΕΠΙΤΥΧΙΑ
10	127.25 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ

ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ	125.47 sec
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ	100%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ	90%

ΘΕΡΜΟΚΡΑΣΙΑ = 0.5		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	130.90 sec	ΕΠΙΤΥΧΙΑ
2	126.78 sec	ΕΠΙΤΥΧΙΑ

3	130.54 sec	ΕΠΙΤΥΧΙΑ
4	128.90 sec	ΕΠΙΤΥΧΙΑ
5	136.11 sec	ΕΠΙΤΥΧΙΑ
6	128.63 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
7	127.23 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
8	127.59 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
9	124.47 sec	ΕΠΙΤΥΧΙΑ
10	128.90 sec	ΕΠΙΤΥΧΙΑ

ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ	129.01 sec
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ	100%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ	70%

5.2 Phi-4 mini instruct (3B)

PROMPT 1 RESULTS (PHI-4)

ΘΕΡΜΟΚΡΑΣΙΑ = 0.1		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	133.90 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
2	133.07 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
3	133.52 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
4	-	ΑΠΟΤΥΧΙΑ
5	-	ΑΠΟΤΥΧΙΑ
6	132.78 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
7	137.07 sec	ΜΕΡΙΚΗ

		ΕΠΙΤΥΧΙΑ
8	-	ΑΠΟΤΥΧΙΑ
9	132.81 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
10	132.12 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ

ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ	133.61 sec
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ	70%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ	0%

ΘΕΡΜΟΚΡΑΣΙΑ = 0.5		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	130.68 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
2	130.33 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
3	130.42 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
4	134.85 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
5	132.21 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
6	-	ΑΠΟΤΥΧΙΑ
7	142.21 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
8	130.55 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
9	-	ΑΠΟΤΥΧΙΑ
10	129.40 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ		132.58 sec

ΜΕΤΡΗΣΕΩΝ	
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ	80%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ	0%

PROMPT 2 RESULTS (PHI-4)

ΘΕΡΜΟΚΡΑΣΙΑ = 0.1		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	124.84 sec	ΕΠΙΤΥΧΙΑ
2	125.00 sec	ΕΠΙΤΥΧΙΑ
3	125.77 sec	ΕΠΙΤΥΧΙΑ
4	123.94 sec	ΕΠΙΤΥΧΙΑ
5	123.90 sec	ΕΠΙΤΥΧΙΑ
6	127.28 sec	ΕΠΙΤΥΧΙΑ
7	134.50 sec	ΕΠΙΤΥΧΙΑ
8	129.41 sec	ΕΠΙΤΥΧΙΑ
9	125.35 sec	ΕΠΙΤΥΧΙΑ
10	130.01 sec	ΕΠΙΤΥΧΙΑ

ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ	127.03 sec
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ	100%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ	100%

ΘΕΡΜΟΚΡΑΣΙΑ = 0.5		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	126.58 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ

2	125.85 sec	ΕΠΙΤΥΧΙΑ
3	128.59 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
4	133.44 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
5	125.73 sec	ΕΠΙΤΥΧΙΑ
6	129.55 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
7	125.58 sec	ΕΠΙΤΥΧΙΑ
8	129.47 sec	ΕΠΙΤΥΧΙΑ
9	125.80 sec	ΕΠΙΤΥΧΙΑ
10	124.63 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ		127.52 sec
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ		100%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ		50%

PROMPT 3 RESULTS (PHI-4)

ΘΕΡΜΟΚΡΑΣΙΑ = 0.1		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	3.80 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
2	3.83 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
3	3.87 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
4	4.00 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
5	3.99 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
6	3.92 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ

7	3.85 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
8	3.94 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
9	3.82 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
10	3.76 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ

ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ	3.88 sec
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ	100%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ	0%

ΘΕΡΜΟΚΡΑΣΙΑ = 0.5		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	3.73 sec	ΕΠΙΤΥΧΙΑ
2	3.70 sec	ΕΠΙΤΥΧΙΑ
3	4.00 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
4	3.70 sec	ΕΠΙΤΥΧΙΑ
5	3.58 sec	ΕΠΙΤΥΧΙΑ
6	3.87 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
7	3.94 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
8	3.96 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
9	3.62 sec	ΕΠΙΤΥΧΙΑ
10	3.95 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ		3.81 sec

ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ	100%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ	50%

PROMPT 4 RESULTS (PHI-4)

ΘΕΡΜΟΚΡΑΣΙΑ = 0.1		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	125.65 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
2	126.68 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
3	126.29 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
4	125.26 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
5	127.52 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
6	127.11 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
7	125.42 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
8	125.83 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
9	126.71 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
10	124.93 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ		126.14 sec
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ		100%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ		0%

ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ		
ΘΕΡΜΟΚΡΑΣΙΑ = 0.5		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	127.58 sec	ΕΠΙΤΥΧΙΑ
2	126.46 sec	ΕΠΙΤΥΧΙΑ
3	126.24 sec	ΕΠΙΤΥΧΙΑ
4	124.49 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
5	127.67 sec	ΕΠΙΤΥΧΙΑ
6	125.20 sec	ΕΠΙΤΥΧΙΑ
7	124.88 sec	ΕΠΙΤΥΧΙΑ
8	125.44 sec	ΕΠΙΤΥΧΙΑ
9	125.67 sec	ΕΠΙΤΥΧΙΑ
10	125.68 sec	ΕΠΙΤΥΧΙΑ
ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ		125.93 sec
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ		100%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ		90%

5.3 DeepSeek chat (7B)

PROMPT 1 RESULTS (DEEPSEEK)

ΘΕΡΜΟΚΡΑΣΙΑ = 0.1		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	-	ΑΠΟΤΥΧΙΑ
2	251.27 sec	ΑΠΟΤΥΧΙΑ
3	136.28 sec	ΑΠΟΤΥΧΙΑ
4	143.45 sec	ΑΠΟΤΥΧΙΑ

5	139.52 sec	ΑΠΟΤΥΧΙΑ
6	130.87 sec	ΑΠΟΤΥΧΙΑ
7	126.93 sec	ΑΠΟΤΥΧΙΑ
8	125.95 sec	ΑΠΟΤΥΧΙΑ
9	132.47 sec	ΑΠΟΤΥΧΙΑ
10	136.72 sec	ΑΠΟΤΥΧΙΑ

ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ	-
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ	0%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ	0%

ΘΕΡΜΟΚΡΑΣΙΑ = 0.5		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	129.08 sec	ΑΠΟΤΥΧΙΑ
2	135.69 sec	ΑΠΟΤΥΧΙΑ
3	-	ΑΠΟΤΥΧΙΑ
4	-	ΑΠΟΤΥΧΙΑ
5	253.58 sec	ΑΠΟΤΥΧΙΑ
6	135.42 sec	ΑΠΟΤΥΧΙΑ
7	-	ΑΠΟΤΥΧΙΑ
8	134.14 sec	ΑΠΟΤΥΧΙΑ
9	-	ΑΠΟΤΥΧΙΑ
10	135.09 sec	ΑΠΟΤΥΧΙΑ

ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ	-
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ	0%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ	0%

PROMPT 2 RESULTS (DEEPSEEK)

ΘΕΡΜΟΚΡΑΣΙΑ = 0.1		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	8.25 sec	ΑΠΟΤΥΧΙΑ
2	128.21 sec	ΑΠΟΤΥΧΙΑ
3	127.19 sec	ΑΠΟΤΥΧΙΑ
4	35.14 sec	ΑΠΟΤΥΧΙΑ
5	6.38 sec	ΑΠΟΤΥΧΙΑ
6	138.94 sec	ΑΠΟΤΥΧΙΑ
7	134.55 sec	ΑΠΟΤΥΧΙΑ
8	134.92 sec	ΑΠΟΤΥΧΙΑ
9	129.58 sec	ΑΠΟΤΥΧΙΑ
10	136.39 sec	ΑΠΟΤΥΧΙΑ

ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ	-
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ	0%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ	0%

ΘΕΡΜΟΚΡΑΣΙΑ = 0.5		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	132.15 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
2	135.02 sec	ΑΠΟΤΥΧΙΑ
3	142.13 sec	ΑΠΟΤΥΧΙΑ
4	132.69 sec	ΑΠΟΤΥΧΙΑ
5	140.22 sec	ΑΠΟΤΥΧΙΑ
6	135.85 sec	ΑΠΟΤΥΧΙΑ
7	129.62 sec	ΑΠΟΤΥΧΙΑ

8	10.31 sec	ΑΠΟΤΥΧΙΑ
9	134.61 sec	ΑΠΟΤΥΧΙΑ
10	138.45 sec	ΑΠΟΤΥΧΙΑ
ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ		132.15 sec
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ		10%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ		0%

PROMPT 3 RESULTS (DEEPSEEK)

ΘΕΡΜΟΚΡΑΣΙΑ = 0.1		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	4.79 sec	ΑΠΟΤΥΧΙΑ
2	4.69 sec	ΑΠΟΤΥΧΙΑ
3	4.24 sec	ΑΠΟΤΥΧΙΑ
4	4.77 sec	ΑΠΟΤΥΧΙΑ
5	4.51 sec	ΑΠΟΤΥΧΙΑ
6	5.41 sec	ΑΠΟΤΥΧΙΑ
7	5.26 sec	ΑΠΟΤΥΧΙΑ
8	5.28 sec	ΑΠΟΤΥΧΙΑ
9	5.26 sec	ΑΠΟΤΥΧΙΑ
10	5.18 sec	ΑΠΟΤΥΧΙΑ
ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ		-
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ		0%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ		0%

ΘΕΡΜΟΚΡΑΣΙΑ = 0.5		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	5.01 sec	ΑΠΟΤΥΧΙΑ
2	4.79 sec	ΑΠΟΤΥΧΙΑ
3	4.81 sec	ΑΠΟΤΥΧΙΑ
4	4.86 sec	ΑΠΟΤΥΧΙΑ
5	4.89 sec	ΑΠΟΤΥΧΙΑ
6	6.15 sec	ΑΠΟΤΥΧΙΑ
7	4.95 sec	ΑΠΟΤΥΧΙΑ
8	4.82 sec	ΑΠΟΤΥΧΙΑ
9	4.93 sec	ΑΠΟΤΥΧΙΑ
10	4.93 sec	ΑΠΟΤΥΧΙΑ
ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ		-
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ		0%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ		0%

PROMPT 4 RESULTS (DEEPSEEK)

ΘΕΡΜΟΚΡΑΣΙΑ = 0.1		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	127.86 sec	ΕΠΙΤΥΧΙΑ
2	127.34 sec	ΕΠΙΤΥΧΙΑ
3	129.92 sec	ΕΠΙΤΥΧΙΑ
4	126.57 sec	ΕΠΙΤΥΧΙΑ
5	128.18 sec	ΕΠΙΤΥΧΙΑ
6	129.05 sec	ΕΠΙΤΥΧΙΑ
7	125.86 sec	ΕΠΙΤΥΧΙΑ
8	124.84 sec	ΕΠΙΤΥΧΙΑ

9	131.99 sec	ΕΠΙΤΥΧΙΑ
10	128.37 sec	ΕΠΙΤΥΧΙΑ

ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ	128.00 sec
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ	100%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ	100%

ΘΕΡΜΟΚΡΑΣΙΑ = 0.5		
ΜΕΤΡΗΣΗ	ΧΡΟΝΟΣ	ΑΠΟΤΕΛΕΣΜΑ
1	129.71 sec	ΕΠΙΤΥΧΙΑ
2	128.79 sec	ΕΠΙΤΥΧΙΑ
3	130.39 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
4	131.58 sec	ΕΠΙΤΥΧΙΑ
5	130.05 sec	ΕΠΙΤΥΧΙΑ
6	143.00 sec	ΕΠΙΤΥΧΙΑ
7	125.89 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
8	126.49 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ
9	124.52 sec	ΑΠΟΤΥΧΙΑ
10	126.01 sec	ΜΕΡΙΚΗ ΕΠΙΤΥΧΙΑ

ΜΕΣΟΣ ΧΡΟΝΟΣ ΕΠΙΤΥΧΗΜΕΝΩΝ ΜΕΤΡΗΣΕΩΝ	130.21 sec
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ	90%
ΠΟΣΟΣΤΟ ΕΠΙΤΥΧΙΑΣ ΧΩΡΙΣ ΤΡΟΠΟΠΟΙΗΣΗ	50%

5.4 Συγκεντρωτικά Αποτελέσματα

ΘΕΡΜΟΚΡΑΣΙΑ = 0.1				
		Hermes Llama 3.2 (3B)	Phi-4 mini instruct (3B)	DeepSeek chat (7B)
Prompt 1	Χρόνος (sec)	134.17	133.61	-
	Ποσοστό επιτυχίας	100%	70%	0%
	Ποσοστό επιτυχίας χωρίς τροποποίηση	100%	0%	0%
Prompt 2	Χρόνος (sec)	128.04	127.03	-
	Ποσοστό επιτυχίας	100%	100%	0%
	Ποσοστό επιτυχίας χωρίς τροποποίηση	100%	100%	0%
Prompt 3	Χρόνος (sec)	4.20	3.88	-
	Ποσοστό επιτυχίας	100%	100%	0%
	Ποσοστό επιτυχίας χωρίς τροποποίηση	100%	0%	0%
Prompt 4	Χρόνος (sec)	125.47	126.14	128.00
	Ποσοστό επιτυχίας	100%	100%	100%
	Ποσοστό επιτυχίας χωρίς τροποποίηση	90%	0%	100%

ΘΕΡΜΟΚΡΑΣΙΑ = 0.5				
		Hermes Llama 3.2 (3B)	Phi-4 mini instruct (3B)	DeepSeek chat (7B)
Prompt 1	Χρόνος (sec)	137.30	132.58	-
	Ποσοστό επιτυχίας	100%	80%	0%
	Ποσοστό επιτυχίας χωρίς τροποποίηση	100%	0%	0%
Prompt 2	Χρόνος (sec)	127.58	127.52	132.15
	Ποσοστό επιτυχίας	100%	100%	10%
	Ποσοστό επιτυχίας χωρίς τροποποίηση	100%	50%	0%
Prompt 3	Χρόνος (sec)	4.17	3.81	-
	Ποσοστό επιτυχίας	100%	100%	0%
	Ποσοστό επιτυχίας χωρίς τροποποίηση	100%	50%	0%
Prompt 4	Χρόνος (sec)	129.01	125.93	130.21
	Ποσοστό επιτυχίας	100%	100%	90%
	Ποσοστό επιτυχίας χωρίς τροποποίηση	70%	90%	50%

Συνολικά παρατηρήθηκαν τα παρακάτω.

Στην θερμοκρασία **0.1**, συμψηφίζοντας τα αποτελέσματα όλων των prompts:

- Το **Hermes Llama 3.2 (3B)** επέστρεψε αποδεκτή και ορθολογική απάντηση στο **100%** των επαναλήψεων (40/40), ενώ επέστρεψε αποδεκτή και ορθολογική απάντηση, δίχως να χρειαστεί τροποποίηση του prompt, στο **97.50%** των επαναλήψεων (39/40).
- Το **Phi-4 mini instruct (3B)** επέστρεψε αποδεκτή και ορθολογική απάντηση στο **92.50%** των επαναλήψεων (37/40), ενώ επέστρεψε αποδεκτή και ορθολογική απάντηση, δίχως να χρειαστεί τροποποίηση του prompt, στο **25%** (10/40) των επαναλήψεων.
- Το **DeepSeek chat (7B)** επέστρεψε αποδεκτή και ορθολογική απάντηση στο **25%** των επαναλήψεων (10/40), ενώ επέστρεψε αποδεκτή και ορθολογική απάντηση, δίχως να χρειαστεί τροποποίηση του prompt, στο **22.50%** των επαναλήψεων (9/40).

Στην θερμοκρασία **0.5**, συμψηφίζοντας τα αποτελέσματα όλων των prompts:

- Το **Hermes Llama 3.2 (3B)** επέστρεψε αποδεκτή και ορθολογική απάντηση στο **100%** των επαναλήψεων (40/40), ενώ επέστρεψε αποδεκτή και ορθολογική απάντηση, δίχως να χρειαστεί τροποποίηση του prompt, στο **92.50%** των επαναλήψεων (37/40).

- Το **Phi-4 mini instruct (3B)** επέστρεψε αποδεκτή και ορθολογική απάντηση στο **95%** των επαναλήψεων (38/40), ενώ επέστρεψε αποδεκτή και ορθολογική απάντηση, *δίχως να χρειαστεί τροποποίηση του prompt*, στο **47.50%** των επαναλήψεων (19/40).
- Το **DeepSeek chat (7B)** επέστρεψε αποδεκτή και ορθολογική απάντηση στο **25%** των επαναλήψεων (10/40), ενώ επέστρεψε αποδεκτή και ορθολογική απάντηση, *δίχως να χρειαστεί τροποποίηση του prompt*, στο **12.50%** των επαναλήψεων (5/40).

Στον παρακάτω πίνακα καταγράφονται τα αποτελέσματα για κάθε LLM σε 80 επαναλήψεις των ερωτήσεων (συμψηφίζοντας τα αποτελέσματα των δύο θερμοκρασιών):

	Ποσοστό αποδεκτών απαντήσεων	Αριθμός αποδεκτών απαντήσεων	Ποσοστό αποδεκτών απαντήσεων χωρίς τροποποίηση	Αριθμός αποδεκτών απαντήσεων χωρίς τροποποίηση
Hermes Llama 3.2 (3B)	100%	80 / 80	95.00%	76 / 80
Phi-4 mini instruct (3B)	93.75%	75 / 80	36.25%	29 / 80
DeepSeek chat (7B)	25%	20 / 80	18.75%	15 / 80

5.5 Συζήτηση Αποτελεσμάτων

Τα πειραματικά ευρήματα δείχνουν καθαρή ιεράρχηση μεταξύ των μοντέλων. Το Hermes Llama 3.2 (3B) πέτυχε τα υψηλότερα ποσοστά επιτυχίας στη ροή $NL \rightarrow SQL \rightarrow \text{Εκτέλεση} \rightarrow \text{Απάντηση}$, το Phi-4 mini (3B) ακολούθησε με ελαφρώς χαμηλότερη ακρίβεια, ενώ το DeepSeek Chat (7B) εμφάνισε τη χαμηλότερη αξιοπιστία. Οι χρόνοι ολοκλήρωσης ήταν συγκρίσιμοι μεταξύ των τριών μοντέλων, στοιχείο που υποδηλώνει ότι οι διαφορές αποδίδονται πρωτίστως σε ικανότητα παραγωγής έγκυρου SQL και όχι σε υστέρηση ταχύτητας. Η αξιολόγηση εκτελέστηκε σε δύο θερμοκρασίες (0.1, 0.5) και με τέσσερα διακριτά prompts, ακολουθώντας την αρχιτεκτονική που περιγράφεται στο παρόν κεφάλαιο.

Επίδραση θερμοκρασίας

Η επίδραση της θερμοκρασίας διαφοροποιείται ανά μοντέλο. Για το Hermes LLaMA 3.2 (3B), η χαμηλή θερμοκρασία ενόησε ελαφρώς τη σταθερότητα και την «απόλυτη» ορθότητα του παραγόμενου SQL, χωρίς ουσιαστική μεταβολή στη συνολική αποτελεσματικότητα (οι μικρές αποκλίσεις κρίνονται αμελητέες). Στο Phi-4 mini instruct (3B), αντίθετα, η υψηλότερη θερμοκρασία οδήγησε σε αισθητά μικρότερη ανάγκη για διαφοροποιήσεις στα prompts, διατηρώντας παρόμοια συνολική αποτελεσματικότητα. Το μοντέλο φαίνεται να επωφελείται από ελαφρώς αυξημένη στοχαστικότητα και ελευθερία στην διατύπωση. Για το DeepSeek chat (7B), η μεγαλύτερη ρύθμιση θερμοκρασίας βελτίωσε την ικανότητα του LLM να επιστρέφει αποδεκτά αποτελέσματα χωρίς διαφοροποιήσεις στο prompt, χωρίς όμως να μεταβάλλει το χαμηλό επίπεδο συνολικής αποτελεσματικότητας. Συνολικά, η χαμηλή θερμοκρασία προκρίνεται για μοντέλα που ήδη επιδεικνύουν υψηλή συνέπεια (LLaMA), ενώ μια μέτρια αύξηση μπορεί να ωφελήσει μοντέλα που χρειάζονται περισσότερη «ευελιξία» στη σύνθεση (Phi-4), με την προϋπόθεση ότι δεν υπονομεύεται η συντακτική και σημασιολογική ορθότητα.

Ευελιξία στα prompts

Παρατηρήθηκε ευαισθησία διατυπώσεων: prompts με σαφείς «κανόνες» (π.χ. ρητή αναφορά διαθέσιμων στηλών, οδηγίες χρόνου, χρήση "time") περιόρισαν τις «εφευρέσεις» πεδίων και βελτίωσαν την επιτυχία. Το Hermes Llama επωφελήθηκε περισσότερο από τους περιοριστικούς κανόνες του prompt, στοιχείο συμβατό με τη γενικότερη επίδοσή του.

Αξίζει να σημειωθεί βέβαια πως αυτό το φαινόμενο οφείλεται, όπως αναφέρθηκε και νωρίτερα, στην υπολογιστική ισχύ του συστήματος και στο πλήθος των παραμέτρων στο κάθε μοντέλο. Σε μοντέλα αισθητά μεγαλύτερου αριθμού παραμέτρων (π.χ. 100 δισεκατομμυρίων) αυτό το φαινόμενο θα ήταν πολύ πιο σπάνιο, ακόμα και σε εκδόσεις με περισσότερες παραμέτρους των ίδιων μοντέλων με αυτών που χρησιμοποιήσαμε.

Τυπολογία σφαλμάτων και ανάκαμψη

Τα συχνότερα σφάλματα ήταν: α) αναφορές σε μη διαθέσιμες στήλες/logs, β) συντακτικά λάθη σε υποερωτήματα που ξεκινούν με "WITH" (προσωρινά ενδιάμεσα αποτελέσματα διαθέσιμα μόνο στο ίδιο ερώτημα) και στο κλείσιμο του τελικού ερωτήματος, και γ) λανθασμένος χειρισμός χρονικών πεδίων. Ο κυκλικός μηχανισμός ανατροφοδότησης (feedback) προς το LLM (με ρητή επισήμανση του σφάλματος και υπενθύμιση των κανόνων) οδήγησε συχνά σε επιτυχή διόρθωση στο επόμενο βήμα, ιδίως για Hermes Llama και, σε μικρότερο βαθμό, για Phi-4.

Χρόνος απόκρισης και αποδοτικότητα

Παρότι οι χρόνοι ήταν παρόμοιοι, η διαφορά στην «πρώτη σωστή απάντηση» καθόρισε την πρακτική αποδοτικότητα. Μοντέλα με υψηλότερη ακρίβεια παρήγαγαν λιγότερα retries, άρα μικρότερο πραγματικό χρόνο για να ληφθεί αξιοποιήσιμο αποτέλεσμα.

Αδυναμία ανταπόκρισης Deepseek

Το DeepSeek chat εμφάνισε ιδιαίτερη αδυναμία να ανταποκριθεί στις απαιτήσεις και τα ζητούμενα των prompts. Αυτό οφείλεται στον τρόπο λειτουργίας του μοντέλου, καθώς ήταν εμφανώς πιο "φλύαρο". Παρά τις αυστηρές οδηγίες να παράγει αποκλειστικά SQL queries, εκείνο επέμενε συχνά "πεισματικά" στην παραγωγή κειμένου παράλληλα με το query. Παράλληλα ήταν πολύ πιο επιρρεπές σε hallucinations, καθώς παρήγαγε ανύπαρκτα δεδομένα και στήλες για την βάση δεδομένων, ενώ παρήγαγε ακόμα και τελείως πλασματικές/"φανταστικές" απαντήσεις δίχως καθόλου δεδομένα (υπαρκτά ή μη). Συνολικά εμφάνισε μεγάλη αναξιопιστία, δίνοντας πολύ μεγαλύτερη έμφαση στην παραγωγή μιας οποιαδήποτε απάντησης παρά στην παραγωγή της σωστής απάντησης.

Αξίζει να σημειωθεί ότι μέρος της παραπάνω συμπεριφοράς ενδέχεται να οφείλεται στην ασυμβατότητα της χρησιμοποιούμενης τεχνικής prompting με την αρχιτεκτονική του μοντέλου. Το DeepSeek ενσωματώνει μηχανισμό Chain-of-Thought (CoT) συλλογισμού, ο οποίος απαιτεί διαφορετική προσέγγιση στη διατύπωση των prompts σε σχέση με μοντέλα που δεν ακολουθούν αντίστοιχη στρατηγική εσωτερικού συλλογισμού, όπως τα LLaMA και το Phi-4. Η χρήση ενιαίου prompt και για τα δύο μοντέλα, αν και επιβεβλημένη για λόγους συγκρισιμότητας, ενδέχεται να μην επέτρεψε στο DeepSeek να αξιοποιήσει πλήρως τις δυνατότητές του. Ωστόσο, η εν λόγω παρατήρηση δεν αναιρεί τα καταγεγραμμένα αποτελέσματα, αλλά αποτελεί έναν πιθανό παράγοντα που θα μπορούσε να διερευνηθεί περαιτέρω σε μελλοντική επέκταση.

Επιπτώσεις για παραγωγική χρήση

Για ροές τύπου $NL \rightarrow SQL$ σε Moodle Logs (ή αντίστοιχα σχήματα), συνιστάται:

- Χαμηλή θερμοκρασία στην παραγωγή SQL και αυστηρό prompt με ρητές λίστες πεδίων/κανόνες χρόνου.
- Στοχευμένος έλεγχος σφαλμάτων και αυτόματη ανατροφοδότηση στο LLM.
- Ενιαίο σημείο πρόσβασης δεδομένων (π.χ. μέσω PostgREST) και σταθερό schema, ώστε να μειώνονται οι μεταβλητές του πειράματος και να ενισχύεται η αναπαραγωγιμότητα.

Συνολική εκτίμηση

Με βάση τα αποτελέσματα, το **Hermes Llama 3.2 (3B)** αποτελεί την καταλληλότερη επιλογή για την παρούσα ροή, επιτυγχάνοντας ανώτερη αξιοπιστία χωρίς υστέρηση χρόνου. Το Phi-4 mini (3B) προσφέρει μια ικανοποιητική εναλλακτική με μικρή πτώση στο success rate, ενώ το DeepSeek chat (7B) απαιτεί ισχυρότερη καθοδήγηση (πιο περιοριστικά prompts/κανόνες) ή επιπλέον μηχανισμούς επαλήθευσης πριν χρησιμοποιηθεί σε σενάρια παραγωγής.

5.6 Περιορισμοί

Η αξιολόγηση της ροής Φυσική Γλώσσα → SQL → Εκτέλεση → Απάντηση πάνω στα καθαρισμένα Moodle Logs ανέδειξε συγκεκριμένα όρια και δυνητικούς κινδύνους, τα οποία σχετίζονται τόσο με τα ίδια τα LLMs όσο και με το περιβάλλον υλοποίησης (PostgreSQL/PostgREST, LM Studio, LangChain). Η αποτύπωσή τους είναι κρίσιμη για την ορθή ερμηνεία των αποτελεσμάτων 4 και για την ασφαλή αξιοποίηση της λύσης.

Κύριοι περιορισμοί

- **Υπολογιστική ισχύς & τοπικό περιβάλλον εκτέλεσης**

Η εκτέλεση μέσω LM Studio σε τοπικό hardware εξασφαλίζει έλεγχο και ιδιωτικότητα, αλλά περιορίζεται από CPU/GPU και μνήμη. Μεγάλα μοντέλα ή συνεχή φορτία μπορούν να υποβαθμίσουν τη σταθερότητα/καθυστέρηση, επηρεάζοντας χρόνους απόκρισης και επαναληψιμότητα.

- **Σχεδιασμός prompts**

Η ποιότητα και η σαφήνεια των prompts επηρεάζουν άμεσα τη συντακτική και σημασιολογική ορθότητα του παραγόμενου SQL. Ασαφείς οδηγίες ή ελλιπής οριοθέτηση των ζητούμενων αυξάνουν σφάλματα και αποκλίσεις.

- **Κίνδυνος «ορθότητας εκ πρώτης όψεως»**

Ιδίως σε ασθενέστερα μοντέλα, ενδέχεται να παραχθούν απαντήσεις που φαίνονται πειστικές αλλά είναι συντακτικά/σημασιολογικά λανθασμένες (*false positives*). Απαιτούνται μηχανισμοί ελέγχου πριν την εκτέλεση.

- **Σύζευξη με schema**

Η ροή βασίζεται σε συγκεκριμένο πίνακα/στήλες (`moodle_data`). Μεταβολές στο schema (προσθαφαιρέσεις πεδίων, μετονομασίες) μπορούν να προκαλέσουν αποτυχίες ή σιωπηρές ασυμβατότητες, αν δεν ενημερωθεί αντίστοιχα το prompt και οι κανόνες.

- **Ασφάλεια & ιδιωτικότητα**

Η έκθεση δεδομένων μέσω PostgREST, ακόμη και με «ελάχιστα» δικαιώματα, απαιτεί προσεκτικές πολιτικές (RLS, περιορισμοί ανά ρόλο, ανωνυμοποίηση) ώστε να αποτραπεί διαρροή ή πρόσβαση σε μη εξουσιοδοτημένα δεδομένα.

Διάχυση κινδύνων

- **Προδιαγραφή & έλεγχος prompts**

Χρήση σαφών prompt templates με ρητή απαρίθμηση πεδίων του schema, κανόνες χρόνου (π.χ. προτίμηση "time"), και έναν μόνο ολοκληρωμένο SQL τερματισμένο σε «;». Ενσωμάτωση παραδειγμάτων-υποδειγμάτων και απαγόρευση μη υπαρκτών πεδίων μειώνουν τις παρεκκλίσεις.

- **Σταθερές ρυθμίσεις παραγωγής SQL**

Σχετικά χαμηλή θερμοκρασία στη φάση παραγωγής SQL για συνέπεια, σταθερό seed, και ξεχωριστό βήμα περίληψης αποτελεσμάτων για να μην επηρεάζεται το query από παραμετρική στοχαστικότητα.

- **Έλεγχοι πριν την εκτέλεση**

Schema-aware validators, αυτόματος εντοπισμός επικίνδυνων μοτίβων και δοκιμαστική εκτέλεση σε ασφαλές περιβάλλον μειώνουν το ρίσκο εσφαλμένων/επικίνδυνων queries.

- **Κυκλική ανατροφοδότηση με στοχευμένα μηνύματα**

Σε σφάλμα, αυτόματη επαναπροσπάθεια με επιστροφή του ακριβούς μηνύματος («η στήλη X δεν υπάρχει», «συντακτικό λάθος»), ώστε το LLM να διορθώνει συγκεκριμένα σημεία αντί για «τυφλές» επαναλήψεις. (Η πρακτική αυτή ήδη ενσωματώνεται στη ροή του κώδικα.)

- **Διακυβέρνηση PostgREST.**

Πρόσβαση μόνο όσο χρειάζεται (όχι περισσότερα δικαιώματα από τα απολύτως απαραίτητα), με κανόνες ορατότητας ανά γραμμή όπου χρειάζεται (ώστε κάθε χρήστης να βλέπει μόνο ό,τι του επιτρέπεται). Όρια στα αποτελέσματα (π.χ. limit/offset για μικρά «πακέτα» δεδομένων), όρια στον ρυθμό κλήσεων (για να μην επιβαρύνεται ο server) και καταγραφή ενεργειών (log) για έλεγχο και ανιχνευσιμότητα. Τέλος, όπου απαιτείται, αφαίρεση ή αντικατάσταση ταυτοποιητικών στοιχείων (ανωνυμοποίηση/ψευδωνυμοποίηση) ώστε να προστατεύεται η ιδιωτικότητα.

- **Διαχείριση πόρων στο τοπικό περιβάλλον**

Καταγραφή εκδόσεων βιβλιοθηκών/μοντέλων, έλεγχος χρήσης μνήμης/CPU/GPU, και πρόβλεψη fallback (π.χ. ελαφρύτερο μοντέλο) σε συνθήκες πίεσης.

- **Αναθεώρηση σε αλλαγές schema**

Αυτοματοποιημένος έλεγχος διαφορών (schema diff) και ανανέωση των prompts/κανόνων κάθε φορά που τροποποιείται ο πίνακας moodle_data ή οι τύποι πεδίων.

Συνολικά, οι παραπάνω ενέργειες καθιστούν τη λύση πιο ανθεκτική, μειώνουν την πιθανότητα λανθασμένων ή παραπλανητικών απαντήσεων και διασφαλίζουν ότι τα ευρήματα του κεφαλαίου παραμένουν έγκυρα υπό τεκμηριωμένους όρους λειτουργίας και ασφάλειας.

Κεφάλαιο 6ο: Επίλογος

6.1 Σύνοψη

Στα πλαίσια της διπλωματικής εργασίας πραγματοποιήθηκε η προετοιμασία και αξιοποίηση ενός συνόλου **Moodle Logs** με στόχο τη μελέτη και την αξιολόγηση μιας αυτοματοποιημένης ροής Φυσική Γλώσσα (NL) → SQL → Εκτέλεση → Απάντηση. Το πρώτο μέρος επικεντρώθηκε στη διαμόρφωση κατάλληλου dataset και στη ρύθμιση του υποστηρικτικού περιβάλλοντος. Η εκκαθάριση δεδομένων εφαρμόστηκε σε πολλαπλά στάδια, ώστε να αφαιρεθούν συστηματικές ή μη αντιπροσωπευτικές εγγραφές και να περιοριστεί ο θόρυβος. Απομακρύνθηκαν καταχωρήσεις με origin="cli" (οι οποίες αντιστοιχούν σε αυτοματοποιημένες/διαχειριστικές ενέργειες), αφαιρέθηκαν διπλότυπες εγγραφές και απορρίφθηκαν πεδία με υψηλά ποσοστά ελλিপών τιμών ή περιορισμένη χρησιμότητα για τα ερευνητικά ερωτήματα. Παράλληλα, ενισχύθηκε η χρονική αναπαράσταση μέσω παραγωγής βοηθητικών πεδίων (datetime, time, day, month) με διατήρηση του αρχικού timecreated, ώστε να υποστηρίζονται διαφορετικοί τύποι χρονικών ερωτημάτων. Το αποτέλεσμα ήταν ένα συμπαγές και ομοιογενές σύνολο δεδομένων (με μεγάλη μείωση όγκου σε σχέση με το αρχικό), το οποίο φορτώθηκε σε **PostgreSQL** και οργανώθηκε σε πίνακα κατάλληλο για συστηματικά ερωτήματα και πειραματισμό.

Το δεύτερο μέρος της διπλωματικής αφορούσε την υλοποίηση του απαιτούμενου κώδικα, την εκτέλεση επαναλαμβανόμενων δοκιμών και την εξαγωγή συμπερασμάτων από τα τελικά αποτελέσματα. Η εφαρμογή στηρίχθηκε σε αυστηρά διατυπωμένα prompt templates που ενσωματώνουν το σχήμα της βάσης και κανόνες παραγωγής SQL (π.χ. χρήση υπαρκτών στηλών/πινάκων, μορφοποίηση, χρονικοί κανόνες), καθώς και σε μηχανισμό ελέγχου/ανατροφοδότησης σφαλμάτων ώστε, όταν προκύπτουν αποτυχίες εκτέλεσης, να παρέχεται στο μοντέλο στοχευμένη καθοδήγηση για διόρθωση. Η αξιολόγηση πραγματοποιήθηκε με τρία διαφορετικά **LLMs** που εκτελέστηκαν τοπικά μέσω **LM Studio** (OpenAI-compatible API), σε δύο ρυθμίσεις θερμοκρασίας (0.1 και 0.5) και με τέσσερα διακριτά prompts, ενώ καταγράφηκαν τόσο ο χρόνος ολοκλήρωσης όσο και η επιτυχία (παραγωγή εκτελέσιμου και σημασιολογικά ορθού SQL που επιστρέφει αποτέλεσμα χωρίς σφάλμα). Από τα συγκεντρωτικά αποτελέσματα προέκυψε σαφής διαφοροποίηση μεταξύ των μοντέλων: το **Hermes Llama 3.2 (3B)** εμφάνισε τη μεγαλύτερη σταθερότητα και το υψηλότερο επίπεδο αξιοπιστίας, το **Phi-4 mini instruct (3B)** ακολούθησε με καλή αλλά λιγότερο συνεπή απόδοση, ενώ το **DeepSeek chat (7B)** παρουσίασε σημαντικά χαμηλότερα ποσοστά επιτυχίας και μεγαλύτερη ευαισθησία στη διατύπωση/τροποποίηση των prompts. Η επίδραση της θερμοκρασίας δεν ήταν ενιαία για όλα τα μοντέλα. Σε μοντέλα που ήδη παράγουν «πειθαρχημένες» εξόδους η χαμηλότερη θερμοκρασία ενίσχυσε τη συνέπεια, ενώ σε άλλα μοντέλα η μέτρια αύξησή της βελτίωσε την πιθανότητα παραγωγής απολύτως ορθής λύσης, χωρίς να αλλάζει ουσιαστικά ο χρόνος εκτέλεσης.

Τα οφέλη από την υλοποίηση του προγράμματος είναι πολλά. Ένας καθηγητής μπορεί να αξιοποιήσει τις δυνατότητές του με σκοπό την βέλτιστη προσέγγιση στο εκάστοτε μάθημα που διδάσκει, αλλά και τον εντοπισμό και την στοχευμένη αντιμετώπιση των αδυναμιών των φοιτητών. Αυτό μπορεί να γίνει παραδείγματος χάριν ρωτώντας το πρόγραμμα σε ποιο μάθημα φαίνεται να υπάρχει η μεγαλύτερη συμμετοχή, ποιές ενέργειες πραγματοποιούνται από τους φοιτητές συχνότερα στο εκάστοτε μάθημα, ποιές χρονικές περιόδους εμφανίζουν οι φοιτητές τα μεγαλύτερα ποσοστά ενεργειών κ.ο.κ.

Σε κάθε περίπτωση, αποτελεί ένα χρήσιμο εργαλείο που μπορεί να αξιοποιηθεί δίχως κάποια ιδιαίτερη τεχνογνωσία από την πλευρά του χρήστη και αναλαμβάνει τον φόρτο της ανάλυσης των δεδομένων για αυτόν.

6.2 Προοπτικές Μελλοντικών Επεκτάσεων

Το συγκεκριμένο σύνολο δεδομένων, σε συνδυασμό με τη μεθοδολογία που εφαρμόστηκε για τη ροή Φυσική Γλώσσα → SQL → Εκτέλεση → Απάντηση, ενδείκνυται για περαιτέρω διερεύνηση τόσο σε επίπεδο αξιοπιστίας των LLMs όσο και σε επίπεδο ουσιαστικής αξιοποίησης των Moodle Logs. Μία κατεύθυνση που προκύπτει φυσικά από την πορεία της εργασίας είναι η ενδεδειγμένη κατανόηση των αιτιών αποτυχίας κατά την παραγωγή SQL (π.χ. αναφορές σε μη υπαρκτά πεδία, σφάλματα στη δομή του ερωτήματος, εσφαλμένες χρονικές μετατροπές). Η συστηματική κατηγοριοποίηση σφαλμάτων και η ανάπτυξη «κανόνων διόρθωσης» (π.χ. αυτόματα checks πριν την εκτέλεση) θα

μπορούσε να οδηγήσει σε πιο σταθερή λειτουργία, ιδιαίτερα σε μοντέλα που εμφάνισαν χαμηλότερη συνέπεια. Παράλληλα, ενδιαφέρον θα είχε η μελέτη τεχνικών που ενισχύουν την επαναληψιμότητα, όπως πιο αυστηρές προδιαγραφές prompts, σταθεροποίηση παραμέτρων παραγωγής, καθώς και επανασχεδιασμός της ανάδρασης (feedback) ώστε να «κατευθύνει» αποτελεσματικότερα το μοντέλο προς ορθές λύσεις.

Μία δεύτερη προσέγγιση αφορά την ενίσχυση του συστήματος με μηχανισμούς τεκμηρίωσης και ελέγχου των απαντήσεων, δεδομένου ότι ορισμένα LLMs μπορούν να επιστρέψουν πειστικές αλλά λανθασμένες λύσεις. Προς αυτή την κατεύθυνση, θα μπορούσαν να αξιοποιηθούν διαδικασίες επαλήθευσης, όπως έλεγχοι συνέπειας στο αποτέλεσμα (π.χ. αναμενόμενα εύρη τιμών, έλεγχος κενών/ακραίων τιμών), καθώς και συγκρίσεις με εναλλακτικές εκτελέσεις ή «διπλή παραγωγή» SQL από διαφορετικό prompt.

Επιπλέον, ενδιαφέρον θα είχε η εφαρμογή της ίδιας μεθοδολογίας σε διαφορετικά σύνολα Moodle Logs (π.χ. άλλοι οργανισμοί, διαφορετικά μαθήματα ή χρονικές περίοδοι) ή και η σύγκριση με άλλα εκπαιδευτικά συστήματα καταγραφής δραστηριότητας, ώστε να διερευνηθεί η γενικευσιμότητα των συμπερασμάτων. Στο ίδιο πλαίσιο, η επέκταση του σχήματος της βάσης (π.χ. εμπλουτισμός με συμπληρωματικούς πίνακες Moodle) θα μπορούσε να υποστηρίξει πιο σύνθετα ερωτήματα και αναλύσεις, μειώνοντας ταυτόχρονα τον κίνδυνο ασυμβατότητας μεταξύ φυσικής γλώσσας και διαθέσιμου schema. Μια πολύ ενδιαφέρουσα τέτοια επέκταση σχήματος θα μπορούσε να αφορά δεδομένα που αναφέρουν την *αποσύνδεση* του εκάστοτε χρήστη από το σύστημα. Η ύπαρξη τέτοιων δεδομένων θα ωφελούσε ιδιαίτερα την συνολική εικόνα της δραστηριότητας του χρήστη και θα επέκτεινε σημαντικά το πεδίο ερωτημάτων το οποίο αφορά την εκπαιδευτική ανάλυση.

Επιπρόσθετα, η υλοποίηση της δομής της βάσης δεδομένων με τρόπο ώστε να ενημερώνεται ανά τακτά χρονικά διαστήματα με νέα δεδομένα θα ενίσχυε περαιτέρω τη σταθερότητα και την αξιοπιστία της, λόγω του μεγαλύτερου όγκου και της διαρκούς επικαιροποίησης των δεδομένων, τα οποία θα είναι πιο πρόσφατα και συνεπώς πιο αντιπροσωπευτικά σε σχέση με την τρέχουσα πραγματικότητα.

Τέλος, μία πρακτικά χρήσιμη προοπτική αφορά την επιχειρησιακή ωρίμανση της λύσης. Καλύτερη διαχείριση πόρων σε τοπικό περιβάλλον ή καλύτερα μετάβαση σε πιο ισχυρή υποδομή (ή ακόμα καλύτερα και τα δύο), βελτιστοποίηση των ορίων πρόσβασης και των πολιτικών ασφαλείας στη βάση ώστε να προστατεύονται δεδομένα και ιδιωτικότητα, καθώς και ανάπτυξη ενός πιο «φιλικού» περιβάλλοντος χρήσης. Μια ακόμα δυνατότητα, απόρροια των παραπάνω, θα ήταν η παραγωγή προγνώσεων των ενεργειών των φοιτητών με βάση τα ήδη υπάρχοντα δεδομένα, με σκοπό την βελτιστοποίηση, την προετοιμασία και όχι μόνο. Οι επεκτάσεις αυτές μπορούν να μετατρέψουν το παρόν πρωτότυπο σε ένα πιο αξιόπιστο και επαναχρησιμοποιήσιμο εργαλείο, με δυνατότητα αξιοποίησης σε πραγματικά σενάρια εκπαιδευτικής ανάλυσης και υποστήριξης αποφάσεων.

Παραρτήματα

Στο παρακάτω παράρτημα παρατίθεται ο κώδικας που χρησιμοποιήθηκε για τον καθαρισμό και την προεπεξεργασία των δεδομένων Moodle Logs, καθώς και για την εκτέλεση ορισμένων αρχικών, ενδεικτικών ελέγχων/αναλύσεων. Η υλοποίηση πραγματοποιήθηκε σε περιβάλλον Jupyter Notebook, αξιοποιώντας Dask DataFrame (dask-pandas) για τη διαχείριση του μεγάλου όγκου δεδομένων. Ως αποτέλεσμα της διαδικασίας

παρήχθη και εξήχθη αρχείο CSV που περιέχει το τελικό καθαρισμένο σύνολο δεδομένων, το οποίο χρησιμοποιήθηκε στα επόμενα στάδια της εργασίας.

```
import dask
import dask.dataframe as dd
from dask.delayed import delayed
from dask.distributed import Client

import gc
from pandas.api.types import infer_dtype

file_path = "dataset/moodle_logs_ktu_1.csv"

df_original = dd.read_csv(file_path)

print(df_original.dtypes, "\n")
print(df_original.info())

df_original.head(10)

total_rows = df_original.shape[0].compute()

print(f"Total number of rows: {total_rows}")

df_drop = df_original.map_partitions(lambda part: part.drop_duplicates())
df_drop.head()

total_afterdrop_rows = df_drop.shape[0].compute()
print(f"Number of deleted duplicates: {total_rows - total_afterdrop_rows}")

df_drop_test = df_original.drop_duplicates(subset=['action'])
df_drop_test.head()

total_afterdrop_rows_test = df_drop_test.shape[0].compute()
print(f"Number of rows after drop_duplicates(): {total_afterdrop_rows_test}")

missing_val = df_original.isnull().sum().compute()

print(f"Total number of missing values per column:\n {missing_val} \n")

for col in df_original.columns:
    perc = missing_val.loc[col] / total_rows
    if perc > 0:
        print(f"Percentage of missing info of column {col}: {perc:.2%}")

df = df_drop.drop(columns=['ip', 'realuserid', 'objectid', 'other',
'objecttable'])
print(df.head())
df.info()

df = df[df["origin"].str.contains("cli") == False]
```

```

df.head()

total_afterdropcli_rows = df.shape[0].compute()
print(f"Number of deleted rows that contain cli: {total_afterdropcli_rows}\n")
print(f"Number of total rows after deletion: {total_rows -
total_afterdropcli_rows}")

df["datetime"] = dd.to_datetime(df["timecreated"], unit="s")

df.head()

df["time"] = df["datetime"].dt.time

df["day"] = df["datetime"].dt.day_name()

df["month"] = df["datetime"].dt.month_name()

df["userid"].unique().compute()

grouped_userids = df.groupby("userid")

grouped_userids.get_group(2).compute()

grouped_userid_action = df.groupby(["userid", "action"]).size().compute()
grouped_userid_action

actions_gathered = df.groupby(["action"]).size()

actions_gathered_comp = actions_gathered.compute()
actions_gathered_comp

actions_gathered_comp.idxmax()

actions_gathered_comp.idxmin()

grouped_userid_action.loc[grouped_userid_action.groupby("userid").idxmax()]

grouped_userid_action.loc[grouped_userid_action.groupby("userid").idxmin()]

```

Στο παρακάτω παράρτημα παρουσιάζεται ο κώδικας που χρησιμοποιήθηκε για την υλοποίηση και αξιολόγηση της ροής Φυσική Γλώσσα (NL) → SQL → Εκτέλεση → Απάντηση πάνω στον πίνακα moodle_data της βάσης PostgreSQL. Συγκεκριμένα, παρουσιάζεται ο κώδικας που χρησιμοποιήθηκε για την γλώσσα Hermes Llama 3.2 (3B) και θερμοκρασία 0.1. Για την χρήση άλλου LLM και θερμοκρασίας αλλάζουν μόνο δύο γραμμές, οι οποίες έχουν επισημανθεί με σχόλιο με "#" μέσα στον κώδικα. Ο κώδικας περιλαμβάνει: (α) έλεγχο συνδεσιμότητας με τη βάση (smoke test) και δημιουργία του απαραίτητου SQLAlchemy engine, (β) διασύνδεση με το LLM μέσω LM Studio

(OpenAI-compatible API), (γ) ορισμό αυστηρού PromptTemplate που «δένεται» με το schema και τους κανόνες του dataset, (δ) εξαγωγή και καθαρισμό του παραγόμενου SQL από την έξοδο του μοντέλου, (ε) εκτέλεση του SQL στη βάση και ανάκτηση αποτελεσμάτων, καθώς και (στ) μηχανισμό χειρισμού σφαλμάτων με επαναληπτικές προσπάθειες (retries) και στοχευμένη ανατροφοδότηση προς το μοντέλο όταν προκύπτουν αποτυχίες. Τέλος, όταν το αποτέλεσμα είναι μικρό, πραγματοποιείται και σύντομη περίληψη της απάντησης αποκλειστικά βάσει των επιστρεφόμενων γραμμών.

```
import re
import time
import json
from typing import Optional

from langchain.prompts import PromptTemplate
from langchain_community.utilities import SQLDatabase
from openai import OpenAI
from sqlalchemy import create_engine, text as sql_text
from sqlalchemy.exc import SQLAlchemyError
from sqlalchemy.engine import URL

PG_USER = "postgres"
PG_PASS = "1234"
PG_HOST = "127.0.0.1"
PG_PORT = 5432
PG_DB = "THESIS_DATABASE"
MODEL_NAME = "hermes-3-llama-3.2-3b" #Επιλογή LLM

def _smoke_test():
    try:
        import psycopg2
        conn = psycopg2.connect(
            dbname=PG_DB, user=PG_USER, password=PG_PASS, host=PG_HOST,
            port=PG_PORT
        )
        cur = conn.cursor()
        cur.execute("SELECT 1;")
        cur.fetchone()
        cur.close()
        conn.close()
        print("[Smoke Test] psycopg2 connect OK")
    except Exception as e:
        raise SystemExit(f"[Smoke Test] FAILED: {e}")

_smoke_test()

DB_URL = URL.create(
    "postgresql+psycopg2",
    username=PG_USER,
    password=PG_PASS,
    host=PG_HOST,
    port=PG_PORT,
```

```

        database=PG_DB,
    )
    print("Engine URL:", DB_URL)

    engine = create_engine(DB_URL, pool_pre_ping=True)

    with engine.connect() as conn:
        v = conn.exec_driver_sql(
            "select inet_server_addr(), inet_server_port(), current_database();"
        ).fetchone()
        print("[SQLAlchemy Test] OK ->", v)

    db = SQLiteDatabase(engine=engine)

    client = OpenAI(base_url="http://localhost:1234/v1", api_key="lm-studio")

    sql_prompt = PromptTemplate(
        input_variables=["input", "table_info"],
        template="""
You are a PostgreSQL expert who outputs exactly ONE executable SQL statement for
this specific dataset.

Database schema:
{table_info}

Table & columns (do NOT invent):
- moodle_data(
    id, eventname, component, action, target, crud, edulevel,
    contextid, contextlevel, contextinstanceid, userid, courseid,
    relateduserid, anonymous, timecreated, origin, datetime, time, day, month
)

Valid action values in THIS dataset (use these exact tokens; don't invent
others):
'abandoned', 'accepted', 'added', 'answered', 'assessed', 'assigned', 'awarded', 'became
overdue',
'blocked', 'called', 'created', 'deleted', 'disabled', 'downloaded', 'duplicated', 'edi
ted', 'enabled',
'ended', 'epas', 'evaluated', 'exported', 'fail', 'failed', 'graded', 'granted', 'group'
, 'imported',
'joined', 'launched', 'left', 'leveledup', 'locked', 'loggedin', 'loggedinas', 'loggedou
t', 'migrated',
'moved', 'prevented', 'previewed', 'printed', 'published', 'reassessed', 'received', 're
evaluated',
'regraded', 'removed', 'reset', 'restarted', 'restored', 'resumed', 'reviewed', 'saved'
, 'searched',
'sent', 'shown', 'started', 'stream', 'student', 'submitted', 'success', 'switched', 'ta
ken', 'text',
'unassigned', 'unblocked', 'unlocked', 'unpublished', 'updated', 'uploaded', 'viewed'

Canonical event meanings:

```

- Login events: action IN ('loggedin','loggedinas')
- Logout events: action = 'loggedout'
- Views: action = 'viewed'

Time rules:

- Prefer "time" (quote it: "time") for time-of-day questions if it's non-null.
- Otherwise derive a timestamp from the epoch: to_timestamp(timecreated).
- "datetime" may already be a timestamp. Choose the simplest correct source for the task.

CTE & scope rules:

- If you use a CTE (WITH ...), any column referenced later MUST be selected in that CTE.
- Only reference columns that exist in the current scope. Do not invent columns.

Formatting rules:

- Use only standard PostgreSQL functions (EXTRACT, COUNT, SUM, AVG, date_trunc, make_interval, floor, etc.).
- Return exactly ONE complete SQL statement ending with a semicolon, then the token <END>.
- No comments, no markdown, no prose.
- If the request cannot be answered with the available columns, return: CANNOT_ANSWER;

Examples (study patterns; do NOT echo unless asked):

-- Example A: first 100 rows

```
SELECT * FROM moodle_data ORDER BY id LIMIT 100;
```

-- Example B: most common 15-minute login window (using "time"; fallback to epoch if needed)

```
WITH logins AS (
  SELECT COALESCE("time"::time, (to_timestamp(timecreated))::time) AS t
  FROM moodle_data
  WHERE action IN ('loggedin','loggedinas')
),
binned AS (
  SELECT make_time(
    EXTRACT(HOUR FROM t)::int,
    (FLOOR(EXTRACT(MINUTE FROM t)/15)::int)*15,
    0
  ) AS window_start
  FROM logins
)
SELECT
  window_start,
  window_start + INTERVAL '15 minutes' AS window_end,
  COUNT(*) AS login_count
FROM binned
GROUP BY window_start
ORDER BY login_count DESC
```

```

LIMIT 1;

Task:
{input}

SQL:
"""
)

def _cleanup(sql: str) -> str:
    return (
        sql.replace(", FROM", " FROM")
        .replace("HAVG", "AVG")
        .replace("TIMESTAMP TO STRING", "to_char")
        .strip()
    )

def extract_sql(text: str) -> str:
    """Extract a single SQL query (supports CTEs with WITH ...)."""
    if "CANNOT_ANSWER" in text:
        raise ValueError("Model refused: CANNOT_ANSWER (missing columns)")
    text = text.split("<END>", 1)[0]
    lower = text.lower()
    if "with" in lower:
        start = lower.find("with")
        end = text.rfind(";")
        if start != -1 and end != -1 and end > start:
            return _cleanup(text[start:end + 1])
    m = re.search(r"(?is)\bselect\b[\s\S]*?;", text)
    if m:
        return _cleanup(m.group(0))
    m2 = re.search(r"(?is)\bselect\b[\s\S]*$", text)
    if m2:
        sql = m2.group(0).strip()
        if not sql.endswith(";"):
            sql += ";"
        return _cleanup(sql)
    raise ValueError("No SQL query found in LLM output")

def summarize_db_error(err: Exception) -> Optional[str]:
    msg = str(err)
    m = re.search(r'column "[^"]+" does not exist', msg, flags=re.IGNORECASE)
    if m:
        col = m.group(1)
        return (
            f'You referenced column "{col}" that is not in scope. '
            f'If you used a CTE, include "{col}" in that CTE\'s SELECT list or '
            f'reference the correct column name. '
            f'Only reference columns that are selected in the current scope.'
        )
    if "relation" in msg.lower() and "does not exist" in msg.lower():

```

```

        return "You referenced a table or alias that does not exist. Use only
the table moodle_data."
    if "syntax error" in msg.lower():
        return (
            "Your SQL has a syntax error. Ensure each CTE and SELECT is properly
closed, "
            "lists do not end with dangling commas, and the final statement ends
with a semicolon."
        )
    if "timestamp" in msg.lower() or "time zone" in msg.lower():
        return (
            "You likely applied a timestamp function to a non-timestamp column.
"
            "Use to_timestamp(timecreated) for epoch; use datetime directly if
it is a timestamp; "
            "avoid string concatenation with timestamps."
        )
    return None

def _serialize_rows(rows, columns, max_rows=5, str_max_len=200):
    out = []
    for r in rows[:max_rows]:
        obj = {}
        for c, v in zip(columns, r):
            if hasattr(v, "isoformat"):
                try:
                    obj[c] = v.isoformat()
                    continue
                except Exception:
                    pass
            if isinstance(v, str):
                obj[c] = (v if len(v) <= str_max_len else v[:str_max_len] +
"...")
            else:
                obj[c] = str(v) if v is not None else None
        out.append(obj)
    return out

def summarize_result(question: str, sql: str, columns, rows):
    total_rows = len(rows)
    sample_rows_cap = 2 if total_rows > 20 else 5
    sample = _serialize_rows(rows, columns, max_rows=sample_rows_cap,
str_max_len=160)

    cols_list = list(columns)
    if len(cols_list) > 12:
        cols_list = cols_list[:12]

    payload = {
        "question": question,
        "sql": sql[:4000],
    }

```

```

        "columns": cols_list,
        "total_rows": total_rows,
        "sample_rows": sample,
    }
    payload_str = json.dumps(payload, ensure_ascii=False)

    if len(payload_str) > 8000:
        payload["sample_rows"] = _serialize_rows(rows, columns, max_rows=1,
str_max_len=120)
        payload["columns"] = cols_list[:8]
        payload_str = json.dumps(payload, ensure_ascii=False)
    if len(payload_str) > 8000:
        payload["sample_rows"] = []
        payload["columns"] = cols_list[:5]
        payload_str = json.dumps(payload, ensure_ascii=False)

    instructions = (
        "You are a precise analyst. Based ONLY on the provided SQL result JSON,
"
        "answer the user's question in ONE concise sentence. "
        "Do not invent values not present in the result. "
        "If there are zero rows, say that no rows matched the criteria. "
        "If the result is a single aggregated row, state the key value(s). "
        "If multiple rows, summarize the key takeaway."
    )
    messages = [
        {"role": "system", "content": instructions},
        {"role": "user", "content": payload_str},
    ]
    resp = client.chat.completions.create(
        model=MODEL_NAME,
        messages=messages,
        temperature=0.1, #Επιλογή θερμοκρασίας
        top_p=0.9,
        max_tokens=100,
        seed=42,
    )
    return resp.choices[0].message.content.strip()

def ask_question(question: str, retries: int = 4):
    start = time.time()
    schema = db.get_table_info()

    messages = [
        {"role": "system", "content": "You are a precise PostgreSQL SQL
generator. Output only SQL."},
        {"role": "user", "content": sql_prompt.format(input=question,
table_info=schema)},
    ]

    for attempt in range(1, retries + 1):

```

```

response = client.chat.completions.create(
    model=MODEL_NAME,
    messages=messages,
    temperature=0.0,
    top_p=0.9,
    max_tokens=350,
    seed=42,
    stop=["<END>"],
)

llm_output = response.choices[0].message.content.strip()
print(f"\n[LLM Raw Output]\n{llm_output}")

try:
    sql = extract_sql(llm_output)
    print(f"\n[Final SQL to execute]\n{sql}")

    with engine.connect() as conn:
        result = conn.exec_driver_sql(sql)
        rows = result.fetchall()
        columns = result.keys()

    print("\n[Query Results]\n", rows)

    if len(rows) > 20:
        print("\n[Answer]\nReturned", len(rows), "rows. (Summary skipped
for large result.)")
    else:
        human = summarize_result(question, sql, columns, rows)
        print("\n[Answer]\n", human)

    elapsed = time.time() - start
    print(f"\nSuccess in {elapsed:.2f} seconds (attempt {attempt})")
    return
except (SQLAlchemyError, Exception) as e:
    print(f"Attempt {attempt} failed: {e}\n")

    if attempt >= retries:
        print("Query completely failed after retries.")
        return

    guidance = summarize_db_error(e)
    correction = (
        "Correct the SQL.\n"
        "- Only one complete SQL statement ending with a semicolon.\n"
        "- If you used a CTE, only reference columns selected by that
CTE; add any needed columns to the CTE output.\n"
        "- Use only columns that exist in moodle_data. Do not invent
column names.\n"
        "- Prefer \"time\" when non-null; otherwise use
to_timestamp(timecreated).\n"

```

```

        "- For logins, use action IN ('loggedin','loggedinas'); for
logout action='loggedout'.\n"
    )
    if guidance:
        correction += f"\nSpecific error to fix: {guidance}\n"

    messages.extend([
        {"role": "assistant", "content": llm_output},
        {
            "role": "user",
            "content": (
                f"The SQL above failed with error:\n{e}\n\n"
                f"{correction}\n"
                f"Database schema for reference:\n{schema}\n\n"
                "Return only the corrected SQL statement ending with a
semicolon, then <END>."
            ),
        },
    ])

if __name__ == "__main__":
    q = input("Enter your question: ")
    ask_question(q)

```

Βιβλιογραφία

1. Daniela Rotelli, Giuseppe Fiorentino, Anna Monreale (2021) "Making sense of learning log data" [arXiv : 2106.11071]
2. Gökhan Akçapınar, Alper Bayazit (2019) "Data Mining Analysis Tool for Moodle Learning Management System" [DOI: 10.17051/ilkonline.2019.527645]
3. Ashish Vaswani (2017) "Attention Is All You Need"

- [arXiv : 1706.03762]
4. George Siemens, Ryan S J.d. Baker (2012) "Learning Analytics and Educational Data Mining: Towards Communication and Collaboration"
[DOI: 10.1145/2330601.2330661]
 5. Fei Yu, Hongbo Zhang, Prayag Tiwari, Benyou Wang (2023) "Natural Language Reasoning, A Survey"
[arXiv:2303.14725]
 6. Marius Mosbach, Tiago Pimentel, Shauli Ravfogel, Dietrich Klakow, Yanai Elazar (2023) "Few-shot Fine-tuning vs. In-context Learning: A Fair Comparison and Evaluation"
[arXiv:2305.16938]
 7. Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, Aman Chadha (2024) "A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications"
[arXiv:2402.07927]
 8. Andrew Kean Gao (2023) "Prompt Engineering for Large Language Models "
[https://ssrn.com/abstract=4504303]
 9. Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, Dragomir R. Radev (2018) "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task"
[arXiv:1809.08887]
 10. Ruoxi Sun, Sercan Ö. Arik, Alex Muzio, Lesly Miculicich, Satya Gundabathula, Pengcheng Yin, Hanjun Dai, Hootan Nakhost, Rajarishi Sinha, Zifeng Wang, Tomas Pfister, Cloud AI Research Team, Google DeepMind (2023) "SQL-PaLM: Improved large language model adaptation for Text-to-SQL"
[arXiv:2306.00739]
 11. Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan (2025) "A Survey of Text-to-SQL in the Era of LLMs"
 12. LM Studio Documentation (n.d. / Ημερομηνία πρόσβασης 18/06/2025)
[https://lmstudio.ai/docs/app]
 13. NousResearch Hermes 3 (Llama 3.2 3B) model card
[https://huggingface.co/NousResearch/Hermes-3-Llama-3.2-3B]
 14. Microsoft Phi-4-mini-instruct model card
[https://huggingface.co/microsoft/Phi-4-mini-instruct]
 15. Deepseek-ai DeepSeek LLM / Chat 7B model card
[https://huggingface.co/deepseek-ai/deepseek-llm-7b-chat]
 16. PostgREST Documentation: API reference, Configuration, Row Security Policies (RLS) (n.d. / Ημερομηνία πρόσβασης 26/06/2025)
[https://docs.postgresql.org/en/stable/references/api.html]
[https://docs.postgresql.org/en/v12/references/configuration.html]
[https://www.postgresql.org/docs/current/ddl-rowsecurity.html]
 17. Vasilios Mavroudis (2024) "LangChain"
[https://www.turing.ac.uk/sites/default/files/2024-11/langchain.pdf]
 18. LangChain Documentation: SQL / querying SQL databases
[https://v03.api.js.langchain.com/classes/langchain.chains_sql_db.SqlDatabaseChain.html]
 19. Dave Bergmann, Cole, Stryker (n.d. / Ημερομηνία πρόσβασης 08/07/2025) "What is LangChain?"
[https://www.ibm.com/think/topics/langchain]
 20. Moodle Documentation, Dev Documentation για logging & events (n.d. / Ημερομηνία πρόσβασης 16/05/2025)
[https://docs.moodle.org/501/en/ad-hoc_contributed_reports]
 21. Dask Documentation (n.d. / Ημερομηνία πρόσβασης 16/05/2025)
[https://docs.dask.org/en/stable/generated/dask.dataframe.read_csv.html]
 22. OpenAI API Reference
[https://platform.openai.com/docs/api-reference/chat]
 23. SQLAlchemy Documentation (n.d. / Ημερομηνία πρόσβασης 15/08/2025)

- [<https://docs.sqlalchemy.org/en/20/core/engines.html>]
[<https://docs.sqlalchemy.org/en/20/core/pooling.html>]
[<https://docs.sqlalchemy.org/en/20/core/connections.html>]
24. Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, Dario Amodei (2020) "Scaling Laws for Neural Language Models" [arXiv:2001.08361]
 25. Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, Dario Amodei (2020) "Language Models are Few-Shot Learners" [arXiv:2005.14165]
 26. Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, Yusuke Iwasawa (2022) "Large Language Models are Zero-Shot Reasoners" [arXiv:2205.11916]
 27. Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, Yejin Choi (2019) "The Curious Case of Neural Text Degeneration" [arXiv:1904.09751]
 28. Matthew Renze, Erhan Guven (2024) "The Effect of Sampling Temperature on Problem Solving in Large Language Models" [arXiv:2402.05201]